ساختمان داده ها

سرفصل های درسی

۱- تحلیل الگوریتم ها
۲- آرایه ها و کاربرد های آن
۳- چند جمله ای ها
٤- ماتریس اسپارس
٥- پشته ، صف و کاربرد های آن
٦- رشته و کاربرد های آن
۷- لیست پیوندی و کاربرد های آن
۸- درخت و کاربرد های آن
۹- گراف ها
۱۰- مرتب سازی و ساختار های جستجو

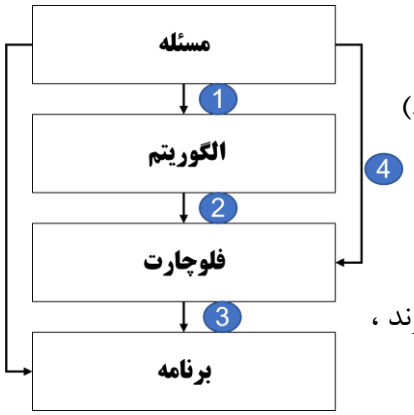
ساختمان داده: عبارت است از مجموعه ای از داده ها و عملگر های مجاز روی این داده ها، به عنوان مثال مجموعه اعداد Integer در زبان هایی که یک نوع داده یا ساختمان را تشکیل می دهند و عملگر های مجاز آن چهار عمل اصلی میباشد.

در هر زبان برنامه سازی یکسری نوع داده وجود دارد (Data Type) ، علاوه بر این برنامه نویس می تواند با به نیاز نوع نوع های داده ی جدیدی را تعریف و پیاده سازی نماید و از آن در برنامه های خود استفاده نماید . مثل : درخت ، صف ، پشته ، لیست پیوندی ....

تحلیل الگوریتم ها : پیچیدگی برنامه (Program Complexity) ، عبارت است از میزان حافظه مورد نیاز برای اجرای برنامه و همچنین مدت زمانی که صرف اجرای آن می‌شود و شمال دو مورد زیر می باشد:

۱- **پیچیدگی حافظه:** میزان حافظه مورد نیاز برای داده ها + میزان حافظه مورد نیاز برای کد اجرایی برنامه
۲- **پیچیدگی زمانی :** زمان اجرای برنامه + زمان کمپایل شدن برنامه

مفاهیم اساسی ساختمان داده ها: اکثر دانشمندان علم کامپیوتر را در مطالعه الگوریتم ها " Study Of Alg " می دانند. <- C.S Study Of Algorithm



۱- ماشین هایی برای اجرای الگوریتم ها (سخت افزار)
۲- نرم افزار هایی برای پردازش الگوریتم ها (دروس و عناوینی که به نرم افزار مربوط هستند.)
۳- تحلیل و بررسی الگوریتم ها
٤- اصول و مبانی الگوریتم ها

تعریف الگوریتم : الگوریتم مجموعه محدودی از دستور العمل هستند که اگر دنبال و پیگیری شوند ، حاصل کار موجب حل مسئله خاصی خواهد شد.

خصوصیات یک الگوریتم (مبانی و اصول یک الگوریتم) :

۱- ورودی : "Input" یک الگوریتم دارای صفر عضو و یا بیشتر به عنوان ورودی خواهد بود.
۲- خروجی : "OutPut" : هر الگوریتمی حداقل دارای یک خروجی باید باشد.
۳- محدودیت :داشتن شرط خاتمه
٤- قطعیت : خالی از ابهام باشد.
٥- کارایی : ۱- زمان لازم (Time: مدت زمان اجرا شدن – Real Time : بلادرنگ) ۲- فضای لازم ۳- پیچیدگی روش

چگونگی و نحوه ایجاد یک برنامه :

۱- تعیین نیازمندی ها / اهداف برنامه Requirment
۲- تحلیل برنامه Analysis
۳- طرح سیستم Design
٤- کد نمودن ، برنامه نویسی Coding
٥- رفع اشکال و تست نهایی Debugging
٦- مستند سازی Document

چگونگی تحلیل یک برنامه :

۱- آیا برنامه به صورت ساختار یافته می باشد
۲- آیا خروجی های برنامه کامل می باشد
۳- آیا برنامه به صورت کامل تست شده است
٤- آیا برنامه به ازای تمام داده ها کامل تست شده است
٥- آیا مستندات کامل برای برنامه تهیه شده است
٦- آیا خوانایی برنامه مناسب می باشد
۷- مستند سازی Document

❖ زمان کمپایل شدن مقدار ثابتی می باشد و علاوه بر این برنامه فقط یکبار کمپایل می شود ، بنابراین در محاسبه پیچیدگی زمان فقط زمان اجرای برنامه را در نظر میگیریم. عوامل زیادی مانند تعداد دستورات برنامه ، نوع دستورات ، ترتیب قرارگرفتن دستورات در کنار یکدیگر آخر در زمان اجرای برنامه تاثیر دارند ولی مهم ترین عامل تعداد دستورات برنامه می باشد لذا معمولا زمان اجرایی برنامه را تابعی از تعداد دستورات در نظر می گیرند.

مثال) تابعی بنویسید که آدرس شروع یک آرایه از اعداد را به عنوان ورودی دریافت (پارامتر) و سپس میانگین عناصر آرایه را برگرداند. برای برنامه پیچیدگی زمانی را محاسبه نمایید.

	غیر اجرایی / اجرایی	تعداد بار اجرا
`float f1(int a[], int n)`	۰	۰
`{`	۰	۰
`int i;`	۰	۰

long int sum = 0;	۱	۱
float avg;	۰	۰
for (i = 0; i < n; i++)	۱	N+۱
sum += a[i];	۱	N
avg = sum / (float)n;	۱	۱
return avg;	۱	۱
}	۰	۰

مجموع دستورات : ۲n+ ٤

زمان اجرا = F (تعداد دستورات)=F(۲n+٤) -> Order -> O(n)

ملاحظه می شود که زمان اجرای تابع متناسب با تعداد داده ها یا همان nمی باشد این مطلب را با O(n) نمایش می دهیم و می گوییم Order یا مرتبه اجرایی الگوریتم n می باشد. در برنامه هایی که با پردازش داده ها سر و کار دارند معمولا زمان اجرا متناسب است با تعداد داده ها

$O(\log_۲ n) < O(n) < o(n \log_۲ n) < o(n^۲) < o(n^۳) <$ ..... $<$ o(n۱) <.....< $o(n^n)$

N=۱۰ ٤ ۱ ٤۰ ۱۰۰ ۱۰۰۰

مثال) برنامه ای بنویسید که آدرس شروع یک آرایه n **عضوی را دریافت نماید (به عنوان پارامتر) و آن را مرتب نماید.**

void sort(int a[], int n)	۰
{	۰
int i, j, t;	$o(n^۲)$
for (i = 0; i < n - 1; i++)	N
for (j = 0; j < n - 1; j++)	N(n-1)
if (a[j] > a[j + 1])	(n-1)(n-1)
{ t = a[j];	(n-1)(n-1)
a[j] = a[j + 1];	(n-1)(n-1)
a[j + 1] = t;	(n-1)(n-1)
}	۰

مجموع دستورات = $۵n^۲ + n - ۴$

در محاسبه order همیشه بزرگترین توان از n را در نظر میگیریم.

❖ مرتبه اجرایی یک الگوریتم با K حلقه تو در تو N^k می شود.

مثال) تعداد دستورات اجرا شده در قطعه کد زیر را به دست آورید؟

```
s = 0;
for (i = 0; i < n; i++)          n+1
```

```
  for (j = 0; j <= i; j++)
    s += i + j;
```

$2+3+\ldots+n+(n+1)=\frac{n(n+1)}{2}+n$

$1+2+3+\ldots+n=\frac{n(n+1)}{2}$

مجموع دستورات = $2+3n+2n^2$ $1+n(n+1)+n+n+1=2n^2+3n+2$

$o(n^2)$

مثال) این الگوریتم چه عملیاتی را انجام می دهد؟

این الگوریتم یک تابع بازگشتی است که عمل ضرب را انجام می‌دهد.

```
int test(int a, int b)
{
  if (b == 1)
    return a;
  else
    return (a + test(a, b -1));
}
```

مثال) مرتبه زمانی تابع زیر را بدست آورید؟

مرتبه زمانی = $O(n) = 2n - 1 = n + \ldots + 4 + 2 + 1$

```
int f (int n) {
    if (n == 1)
        return 0;
    if (n < 2)
        return 1;
    else
        return (f(n/2) + f(n/2));
}
```

مثال) با توجه به رابطه رو به رو مقادیر $p(2,7)$ ، $p(5,3)$ ، $p(15,2)$ **را پیدا کنید.**

```
P ( a , b )= { 5 if a<b
             P(a-b,b+2)+a if a>= b
```

$P(2,7)=5$
$P(5,3)=10$
$P(15,2)=42$
مقادیر خواسته شده

مثال) با توجه به رابطه رو به رو اولاً تابع بازگشتی آن را بنویسید. ثانیاً مقادیر زیر را با توجه به تابع بدست آورید.

F(٤,٢); F(١,٥) , F(٢,٤)

```
F(m,n) = { 1   if m>=n>=1 or m=0
           f(m-1,n) + f(m-1,n-1) Other
```

تابع بازگشتی زیر نوشته شده است.

```
int F(int m, int n) {
    if ((m >= n && n >= 1) || m == 0)
        return 1;
    else
        return F(m - 1, n) + F(m - 1, n - 1);
}
```

F(٤,٢) =١
F(١,٥)=٢
F(٢,٤)=٤
مقادیر خواسته شده

مثال) مرتبه اجرایی قطعه برنامه زیر را بدست آورید ؟

```
for (i = 0; i < n; i++)          n+1
  for (j = 0; j < m; j++)          (n+1)(m+1)
    c[i][j] = a[i][j] + b[i][j];    m*n
```

مجموع دستورات = o(m*n) => ٢*m*n+٢n+١

مفهوم بازگشتی: Recursion Consept : توابعی که خود را دوباره فراخوانی نمی نمایند.

١- مفهوم بازگشتی یکی از مباحث مهم در مسائل کامپیوتری است.
٢- پیاده سازی الگوریتم ها و مسائل مختلف کامپیوتر توسط مفهوم بازگشتی ساده ، قابل اجرا می‌باشد.
٣- ردیابی یا پیگیری اجرای مفهوم بازگشتی (Trace) تا حدودی مشکل است.

مرتبه زمانی اجرای توابع بازگشتی :

مثال) تابع فاکتوریل

```
long int fact(int n) {
  if (n <= 1)
    return 1;
  else
    return (n * fact(n -1));
}
```

زمان اجرای تابع با عدد n = T(n)

$$T(n)=C_1+T(n-1)$$
$$=C_1+C_1+T(n-2)$$
$$=2C_1+T(n-2)$$
$$=3C_1+T(n-3)$$
$$=(n-1)C_1+T(C_2)=nC_1-C_1+C_2$$

زمان اجرای تابع با عدد n= N C_1 + C => O(n)

مثال) تابع جستجوی دودویی : لیست مرتبی از اعداد داریم میخواهیم داده ای وارد این لیست مرتب شده پیدا کنیم.

مرتبه زمانی تابع مجموع دستورات زیر را بدست آورید.

```
S = 0;
for (i = 0; i < n; i++)    →    n+1
  for (j = 0; j <= i; j++) → ۲+۳+٤+....+n+۱
                                  n(n + 1)
                                  ---------
                                      2

   S += i + j; → 1+2+3+...+n=n(n+۱)/۲
```

مثال) مرتبه زمانی تابع بازگشتی زیر را بدست آورید :

```
int f(int n) {
  if (n == 0)
    return 1;
  else
    return (f(n - 1) + f(n - 1));
}
```

زمان اجرا = T(n)

$$T(n)=C_۱ + ۲T(n-۱)$$
$$= C_۱ + ۲\big(C_۱ + ۲T(n-۲)\big)$$
$$= C_۱ + ۲C_۱ + ۴C_۱ + ۸T(n-۳)$$
$$= C_۱ + ۲C_۱ + C_۱ + \cdots + ۲^m T(n-m)$$
$$= ۲^m = ۲^{(n-۱)} = O(۲^n)$$

مثال) مرتبه زمانی تابع بازگشتی زیر را بدست آورید :

```
int bsearch(int a[], int low, int high, int x) {
    int mid;
    if (low > high)
        return -1;
    mid = (high + low) / 2;
    if (a[mid] == x)
        return mid;
    if (a[mid] > x)
        return (bsearch(a, low, mid - 1, x));
    else
        return (bsearch(a, mid + 1, high, x));
}
```

زمان اجرا = T(n)

$$T(n)=C_۱ + T(\tfrac{n}{۲})$$
$$= ۲C_۱ + T\left(\frac{n}{۴}\right)$$
$$= ۳C_۱ + +T\left(\frac{n}{۸}\right)$$
$$= (۲)\ mC_۱ + T(\frac{n}{۲^m})$$
$$= (۱) \ldots + T(۱)$$

از ۱ و ۲ => $۲^m = n \ \log_۲ ۲^m = \log_۲ n = m \log_۲ ۲ = \log_۲ n = m = \log_۲ n$

زمان اجرا = $\log_۲ n + C_۱ + T(۱) = C_۱ \log_۲ n + C_۲ => O(\log_۲ n)$

نوع داده مجرد (انتزاعی) (Abstract Data Type) : برای مشخص کردن یک نوع داده جدید کافیست مقادر مجاز در آن نوع داده به همراه عملگر هایی که روی مقادیر کار می کند را مشخص کنیم. ADT نوع داده ای است که بدون در نظر گرفتن ملاحظات پیاده سازی نوع زبان برنامه سازی و سایر مسائل معرفی میگردد در حقیقت ADT نشان دهنده الگوی یک نوع داده می باشد . غالب خاصی برای بیان ADT وجود ندارد و بهتر است از ساختار زیر استفاده کنیم.

Structure Natural – Number is
Object : An Orderd SvbRange Of The Integer Starting At Zero And Ending At The Maximum Integer (INT_MAX) On The Computer.

Function : For All x,y, ∈ Nat_Number , True , False , ∈ Boolean And Where + , - , < , And == Are the Usual Integer Operations.

Nat_Not_Zero()::==0

تست صفر بودن `Boolean Is_Zero (x) :: = if(x) Return False Else Return True`

جمع کردن `Not_No Add (x,y) :: = if((x+y) <= INT_MAX) Return x+y else return Int_MAX`

تست برابری دو عدد طبیعی `Boolean Equal (x,y) :: = if(x==y) return True else return False`

عدد بعدی طبیعی Nat_No Successor (x) :: = if (x == INT_MAX) return X else return x+۱

تابع تفریق و تفاضل `Nat _ No Subtract (x,y) :: = if ( x<y ) return 0 else Return X-Y end Natural_Number`

`Structure ADT` نام `IS Object :`

لیست مقادیر مجاز

`Function:`
لیست عملگر های مجاز در نوع داده ها

`End ADT` نام

آرایه: مجموعه ای از عناصر همنام و همنوع می باشد که بصورت پشت سر هم در حافظه اصلی قرار می گیرند.

تعریف ریاضی آرایه : آرایه مجموعه ای از زوج های مرتب بصورت <Index,Value> می باشد ، بطوریکه به ازای هر مقدار ایندکس یک مقدار برای `Value` وجود داشته باشد.

مثال) `{<0,8>,<1,2>,<2,11>,<3,6>}` یا `Int a[4]={8,2,11,6}`

درمورد آرایه ۲ نکته مرتبط است { **۱**- فضای لازم برای ذخیره سازی آرایه ها **۲**- آدرس عنصر خالی از آرایه در حافظه چگونه خواهد بود }

محاسبه تعداد عناصر آرایه : ۱– یک بعدی تعداد عناصر = $U_1 - L_1 + 1$ → A[$L_1 \ldots U_1$]

۲- دو بعدی تعداد عناصر =$(U_1 - L_1 + 1) . (U_2 - L_2 + 1)$ → A[$L_1 \ldots U_1, L_2 \ldots U_2$]

آرایه دو بعدی را می توان بصورت یک جدول یا ، ماتریس در نظر گرفت. L۱ تا U۱ سطر هست و ... L۲ U۲ ستون هست.

۳–سه بعدی تعداد عناصر =$(U_1 - L_1 + 1) . (U_2 - L_2 + 1) . (U_3 - L_3 + 1)$ → A[$L_1 \ldots U_1, L_2 \ldots U_2, L_3 \ldots U_3$]

۴–آرایه n بعدی : تعدادعناصر = $\prod_{i=1}^{n}(U_i - L_i + 1)$ → A[$L_1 \ldots U_1, L_2 \ldots U_2, \ldots\ldots\ldots\ldots, L_n - U_n$]

محاسبه عنصر خاصی از آرایه : چون عناصر آرایه پشت سر هم در حافظه اصلی قرار میگیرند ، با داشتن آدرس اولین عنصر میتوانیم آدرس سایر عناصر را بدست آوریم.

حافظه شروع آدرس =∝

طول هر عنصر حافظه=m

آدرس A[i]=?

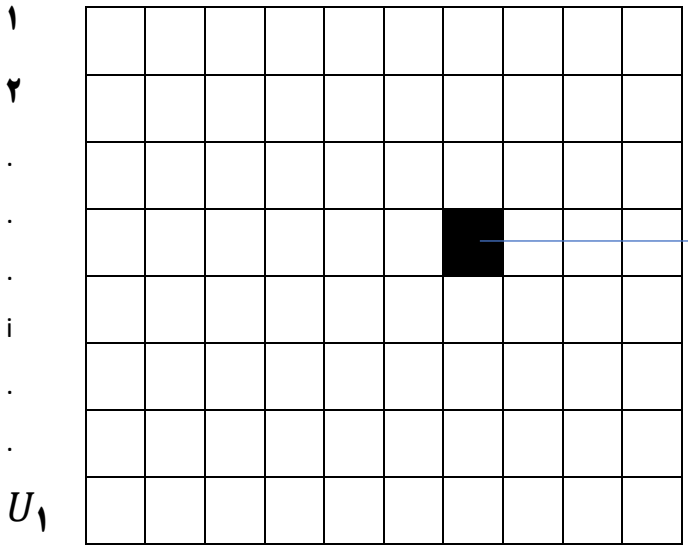
→ آدرس A[i] = $\propto + (i - 1) * m$

مثال) آرایه A[-۱٤,.....,۲۲] تعریف شده است و آدرس شروع آرایه ۱۰۰۰ و طول کلمه ۸ بایت باشد آنگاه عنصر A[۱۸] کدام است؟

آدرس A[۱۸]=۱۰۰۰+(۱۸-(-۱٤))*۸=۱۲٥٦

۲ بعدی

۱ ۲ ۳ ..... j ..... U_2



A[۱...U_1,**۱** ...., U_2]

A[i,j]آدرس= $\propto + [(i - 1)U_2 + (j - 1)]$m

اگر A[$L_1 \ldots U_1, L_2 \ldots U_2$] باشد

A[i,j]= $\propto + [(i - L_1)(U_2 - L_2 + 1) + (j + L_2)]$*m

تعداد عناصر سطر | تعداد سطر

(آدرس به فرم سطری ذخیره شده باشد.)

مثال) اگر آرایه A به فرم A[-۳۰,....,۱٤,٥,....,۱٥] تعریف و به فرم سطری در حافظه ذخیره شده باشد مطلوبست محاسبه آدرس عنصر A[۱۰,۱۲] در صورتیکه آدرس شروع آرایه ۲۰۰۰ و طول کلمه ۸ بایت باشد.

آدرس عنصر A[10,12]=2000+[(10-(-30)(15-5+1)+(15-5)*8]=2000+447=5576

❖ برای محاسبه A[i,j] وقتی به فرم A[$L_{1}....U_{1}, L_{2}....U_{2}$] تعریف شده است و آدرس شروع آرایه $\propto$ و طول کلمه m باشد از رابطه زیر استفاده می شود (آرایه به فرم ستونی ذخیره شده است)

آدرس عنصر A[i,j]= $\propto + [(i - L_{2})(U_{1} - L_{1} + 1) + (i - L_{1})]$*m

❖ تعداد ستون = $(i - L_{2})$

❖ تعداد عضو در ستون = $(U_{1} - L_{1} + 1)$

۳ بعدی آرایه سه بعدی را می توان به صورت یک مکعب با تعدادی صفحه که در هر یک از آنها یک ماتریس وجود دارد تصور کرد :

A[$1...U_{1}, 1 ... U_{2}, 1 .... U_{3}$]

صفحات = $1...U_{1}$

سطر ها = $1 ... U_{2}$

ستون ها = $1 ... U_{3}$

آدرس A[i , j , k]= $\propto + ((i - 1)(U_{2} * U_{3})$+(j-1) U_{3} + (K-1)) * m

❖ نکته اگر آرایه به فرم A[$L_{1}....U_{1}, L_{2}....U_{2}, L_{3}....U_{3}$] تعریف شده باشد آدرس عنصر A[I,j,k] درصورتیکه آدرس شروع آرایه $\propto$ و طول هر کلمه m باشد از رابطه زیر استفاده میکنیم .

❖ تعداد آرایه دو بعدی = $(i - L_{1})$

❖ مقدار عناصر هر آرایه دو بعدی = $(U_{2} - L_{2} + 1)$

A[i , j , k]= $\propto + [(i - L_{1})(U_{2} - L_{2} + 1)(U_{3} - L_{3}) + (j - L_{2})(U_{3} - L_{3} + 1) + (K - L_{3})] * m$

❖ نکته اگر آرایه به فرم A[$L_{1}....U_{1}, L_{2}....U_{2}, L_{3}....U_{3}$] تعریف شده باشد آدرس عنصر A[I,j,k] درصورتیکه آدرس شروع آرایه $\propto$ و طول هر کلمه m باشد از رابطه زیر استفاده میکنیم .

مثال) اگر آرایه Aبه فرم A[۱...۵, ۱...۱۲, ۱....۲۰]تعریف شده باشد، آدرس شروع آرایه ۱۰۰۰ و طول هر خانه ۸ بایت باشد، آدرس عنصر A[۳, ٤, ۲۰]کدام است؟

A[۲۰,٤,۳]=۱۰۰۰+[(۲۰-۱)(۱۲+۱)(۵-۱+۱)+(٤-۱)(۵-۱+۱)+(۳-۱)]*۸= ۱۱۵۷*۸+۱۰۰۰=۱۰۲۵٦

کاربرد آرایه در پیاده سازی چند جمله ای ها:

P(x)=$a_{0}X^{e_{0}}+a_{1}X^{e_{1}}+a_{2}X^{e_{2}}+...+a_{n-1}X^{e_{n-1}}$

$a_{i} \in R$, $a_{i} \in N$

$e_{0}>e_{1}>e_{2}$ ... ... $> e_{n-1} \geq 0$

حال ببینیم چگونه از یک آرایه میتوانیم استفاده کنیم برای ذخیره کردن چند جمله ای برای ذخیره کردن یک چند جمله ای کافی است ، ضرائب و توان های چند جمله ای را ذخیره نماییم.

روش ذخیره کردن چند جمله ای:

$A(x)=۷x^{۵}-۳x^{۴}+۲x^{۲}-۳$

روش اول : Degree درجه چند جمله ای می‌باشد و Coef آرایه است که در آن ضرائب چند جمله ای نگهداری می شود.

مثلا برای مثال فوق داریم :

```
Poly A;

A.Degree = 5;
A.Coef[0]=7;
A.Coef[1]=-3;
A.Coef[2]=0;
A.Coef[3]=2;
A.Coef[4]=0;
A.Coef[5]=-3;
```

```
struct Poly
{
   int degree
   float coef[max] ;
}
```

Degree

Coef [۰]

۵	۷	-۳	۰	۲	۰	-۳

MAX حداکثر تعداد جملات چند جمله ای است.

روش دوم : در این روش Expon توان و Coef ضریب را مشخص می کند.

حال با این آرایه می توانیم ۱۹ جمله را تعریف کنیم.

```
struct Poly {
int expon;
float coef;
};
 Poly A[۲۰];
```

Expon

انتهای چند جمله ای را مشخص می کند

۵	۴	۲	۰	۰
۷	-۳	۲	-۳	۰
۰	1	2	3	4

Coef

جمع دو چند جمله ای :

$A(x)=۷x^{۱۵}-۱۳x^{۱۲}+۸x^{۸}-۶x^{۴}+۱۲x^{۲}-۳$

$B(x)=-4x^{15}-6x^{10}+7x^{9}-8x^{8}+4x^{6}-6x^{4}-3$

$C(x)=A(x)+B(x)=3x^{15}+3x^{12}+6x^{15}+7x^{9}+4x^{6}+12x^{2}-12x^{4}-3x-3$

الگوریتم جمع دو چند جمله ای : فرض کنید چند جمله ای های A(x),B(x) در آرایه A,B ذخیره شده باشند سه متغیر AP,BP,CP را در نظر میگیریم و به آنها مقدار اولیه صفر میدهیم. این متغیر ها به ترتیب جملات جاری دو چند جمله ای های A(x),B(x),C(x) را نشان می دهند. عملیات زیر را تا زمانی که به انتهای یکی از دو چند جمله ای ها برسیم تکرار میکنیم.

```
if(A[AP].Expon==B[BP].Expon)
{
   if (A[AP].Coef+B[BP].Coef)
   {
      C[CP].Expon=A[AP].Expon;
      C[CP].Coef=A[AP].Coef+B[BP].Coef;
      AP++; BP++; CP++;
   }
   else
   {
      AP++; BP++;
   }
}
if(A[AP].Expon>B[BP].Expon)
{
   C[CP]=A[AP];
   AP++; CP++;
}
else
{
   C[CP]=B[BP];
   BP++; CP++;
}
```

و در نهایت هنگامی که به انتهای یک چند جمله ای رسیدیم ، جملات باقیمانده از چند جمله ای دیگر را در چند جمله ای حاصلجمع کپی میکنیم.

ضرب یک چند جمله ای در چند جمله ای : در این حالت ضریب جمله از کلیه ضرائب چند جمله ای ضرب می شود و توان آن با توان کلیه جملات جمع می گردد.

ضرب دو چند جمله ای : ابتدا تک تک جملات چند جمله ای اول را در چند جمله ای دوم ضرب می کنیم سپس این حاصل ضرب ها را با یکدیگر جمع میکنیم.

تمرین های زیر را انجام دهید ؟

۱- **تابعی برای دریافت یک چند جمله ای بنویسید.**

```
 void get_poly(poly A[])
{
    int degree, i, j = ۰;
    printf("Enter degree: ");
    scanf("%d", &degree);
    for (i = degree; i >= ۰; i--)
    {
        float coef;
        printf("coef x^%d: ", i);
        scanf("%f", &coef);
        if (coef != ۰)
        {
            A[j].Expon = i;
            A[j].Coef = coef;
            j++;
        }
    }
}
```

۲- **تابعی برای جمع دو چند جمله ای بنویسید .**

```
void AddPoly(poly A[], poly B[], poly C[])
{
    int AP = 0, BP = 0, CP = 0;
    if (A[AP].Expon == B[BP].Expon)
    {
        if (A[AP].Coef + B[BP].Coef != 0)
        {
            C[CP].Expon = A[AP].Expon;
            C[CP].Coef = A[AP].Coef + B[BP].Coef;
            CP++;
        }
        AP++;
        BP++;
    }
    else if (A[AP].Expon > B[BP].Expon)
    {
        C[CP] = A[AP];
        AP++;
        CP++;
    }
    else
    {
        C[CP] = B[BP];
        BP++;
        CP++;
    }
}
```

۳- **تابعی برای نمایش یک چند جمله ای بنویسید .**

```
Void Print_Poly(Poly A,Poly B,Poly C)
```

٤- **تابعی برای ضرب یک دو جمله ای در چند جمله ای بنویسید.**

```
Void Single_Mult (Poly A,int expon,float coef);
```

٥- **تابعی برای ضرب دو چند جمله ای بنویسید.**

```
Void Mult_Poly (Poly A,Poly B,Poly C);
```

٦- **برنامه ای برای کار با چند جمله ای ها بنویسید.**

```
void Single_Mult(poly A[20], int Expon, float Coef)
{
    int i;
    for (i = 0; A[i].Expon != -1; i++)
    {
        A[i].Expon += Expon;
        A[i].Coef *= Coef;
    }
}
```

❖ ضرب دو چند جمله ای هم ترکیبی از ضرب تک تک جمله است یک چند جمله ای در چند جمله ای دیگر و در نهایت جمع جملات ایجاد شده است.

ماتریس اسپارس (تهی پراکنده ، خلوت) : ماتریس مجموعه از عناصر است که بصورت سطری و ستونی سازماندهی شده است و ماتریس اسپارس ، ماتریسی است که تعداد عناصر غیر ۰ آن کم است یا اینکه بیشتر عناصرش ۰ می باشند معمولا این ماتریس ها دارای ابعاد بزرگی می باشند و در بسیاری از شاخه های مهندسی کاربرد دارند. در این نوع ماتریس ها برای صرفه جویی در حافظه عناصر غیر صفر ماتریس به همراه محل آنها نگهداری می شود.

$$\begin{matrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 2 & 0 & 0 & 0 & 0 & 4 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 6 & 0 & -2 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \end{matrix}$$

	Row	Col	Value
A[0]	5	8	5
A[1]	2	1	2
A[2]	2	6	4
A[3]	4	4	6

ذخیره ماتریس اسپارس : یکی از روش های ذخیره سازی ماتریس اسپارس استفاده از آرایه فشرده با ساختار زیر است:

```
Stract Sparce
{
Int row ;
Int col;
Float value ;
}
A[20];
```

در این آرایه عناصر غیر 0 ماتریس را ذخیره میکنیم.

A[4]	4	6	-2
A[5]	5	7	1

۱- در عنصر اول آرایه تعداد سطر و ستون و ماتریس و تعداد عناصر غیر صفر را ذخیره مینماییم.

۲- در عناصر بعدی آرایه ذخیره سازی محل و مقدار عناصر غیر ۰ حافظه مورد نیاز برای ماتریس ۵*۸*۴=۱۶۰ byte

حافظه مورد نیاز برای آرایه فشرده شده در حافظه مورد نیاز ۶*۸(byte)=۴۸ Byte

در اینجا صرفه جویی در حافظه داریم ولی پردازش بیشتر نیاز داریم.

الگوریتم جمع دو ماتریس اسپارس ذخیره شده آرایه فشرده : فرض کنیم ماتریس اسپارس A,B به ترتیب در آرایه های فشرده A و B ذخیره شده باشند متغیر های AP و BP و CP را در نظر میگیریم و به آنها مقدار اولیه ۱ می دهیم . این متغیر ها به ترتیب عنصر جاری در آرایه های A,B,C را نشان می دهند. اعمال رو به رو تا هنگامیکه به انتهای یکی از آرایه ها برسیم تکرار میکنیم هنگامیکه به انتهای یکی از آرایه ها رسیدیم ، عناصر باقیمانده از آرایه دیگر را در حاصلجمع قرار میدهیم.

```
if (A[AP].Row == B[BP].Row && A[AP].Col == B[BP].Col)
{
    if (A[AP].Value + B[BP].Value)
    {
        C[CP].Row = A[AP].Row;
        C[CP].Col = A[AP].Col;
        C[CP].Value = A[AP].Value + B[BP].Value;
        AP++;
        BP++;
        CP++;
    }
    else
    {
        AP++;
        BP++;
    }
}
if (A[AP].Row < B[BP].Row || (A[AP].Row == B[BP].Row && A[AP].Col <
B[BP].Col))
{
    C[CP] = A[AP];
    AP++;
```

```
    CP++;
}
else
{
    C[CP] = B[BP];
    BP++;
    CP++;
}
```

صف (Queue) : صف یک لیست ترتیبی است که عناصر از یک سمت وارد صف می شوند و از سمت دیگر از آن خارج می شوند در صف اولین عنصری که وارد می‌شود ، اولین عنصری است که خارج می شود لذا صف را یک ساختمان داده ی FIFO (First In First Out)

روش اول پیاده سازی صف : استفاده از یک آرایه یک بعدی می باشد .

```
Int Q[Max];
Int Front = Rear = -1;
```

Q[۰]	Q[۱]	Q[۲]	Q[۳]	Q[٤]

به این ترتیب که یک آرایه با طول MAX در نظر میگیریم ، نوع آرایه بستگی به نوع عناصری دارد که قرار است در صف قرار گیرند. متغیر های Front و Rear را تعریف میکنیم و به آنها مقدار اولیه ۱- را می دهیم ، Front به ابتدای صف و Rear به انتهای صف اشاره می کند. حال باید توابعی برای اضافه کردن عنصر به انتهای صف و حذف عنصر از ابتدای آن بنویسیم.

```
int AddQ(int x)
{
    if (Rear < Max - 1)
    {
        Rear++;
        Q[Rear] = x;
        return 1;
    }
     return 0;
}
int DeleteQ()
{
    if (Front < Rear)
```

اگر عضو به صف اضافه شود تابع را بر میگرداند.

اگر عضو به صف اضافه شود تابع را بر میگرداند

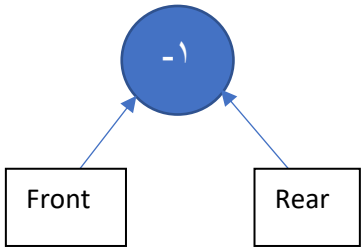
```
    {
        Front++;
        return Q[Front];
    }
    return -1;
}
```

عنصر را بر میگرداند

اگر عنصر نباشد مقدار ۱- را بر میگرداند

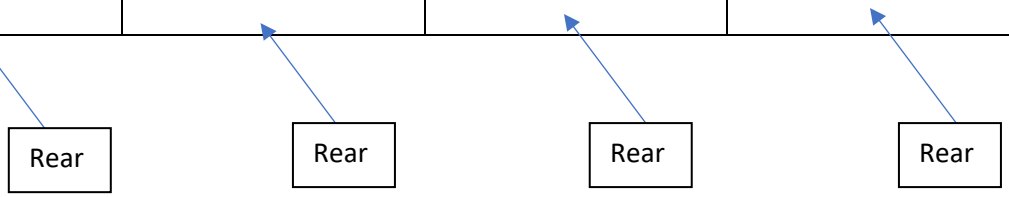
مثال) مثال رو به رو را با توجه به تابع فوق چک کنید.

Add Q (12); AddQ(5); AddQ(20);
Add Q (4); DeleteQ(); DeleteQ();
DeleteQ(); DeleteQ();



- شرط پر بودن صف : `Rear = Max-1`
- وضعیت ابتدای صف : `Front=Rear=-1`
- شرط خالی بودن صف `Rear=Front`

Q[0]	Q[1]	Q[2]	Q[3]	Q[4]
12	5	20	4	



روش دوم پیاده سازی صف : در حالتیکه فقط یک صف در برنامه وجود داشته باشد میتوانیم متغیر های `Front` و `Rear` و `Q` را بصورت سراسری تعریف کنیم در غیر اینصورت باید ساختار زیر را استفاده کنیم.

مثال) تابع `AddQ()` و `DeleteQ()` برای صف با ساختار رو به رو بنویسید؟

```
Sturct Queue
{
  int Front;
  int Rear;
  int Q[Max];
};
Queue A,B;
```

```
int AddQ(struct Queue *A, int x)
{
    if (A->Rear < MAX - 1)
    {
        A->Rear++;
        A->Q[A->Rear] = x;
        return 1;
}
    return 0;
}
```

```
int DeleteQ(Queue * A)
{
    if (A->Front < A->Rear)
    {
        A->Front++;
        return (A->Q[A->Front]);
    }
    return -1;
}
```

صف پویا : در حالتیکه حداکثر تعداد عناصریکه قرار است در صف قرار گیرند از قبل مشخص نباشد می توانیم از تخصیص حافظه بصورت پویا برای ایجاد صف استفاده کنیم ، در این حالت برای صف ساختاری بصورت رو به رو در نظر می گیریم.

Max حداکثر تعداد عناصر صف را مشخص می کند و Q اشاره گری است که به حافظه ای که برای صف اختصاص داده خواهد شد اشاره می نماید.

```
Sturct Queue
{
 int Front;
 int Rear;
Int Max;
 int * Q;
};
Queue A,B;
```

❖ قبل از استفاده از صف باید با استفاده از تابع رو به رو حافظه مورد نیاز را برای آن اختصاص دهیم.

```
Void CreatQ(Queue * A, Int M)
{
  A->Q=(int A)Malloc (۲*m);
  Front = Rear = -۱;
  Max=m
};
```

پشته : یک لیست ترتیبی است که در آن عناصر از یک سمت وارد می شود و از همانطرف نیز خارج می گردند در پشته آخرین عنصری که وارد می شود اولین عنصری است که خارج میگردد . لذا پشته را یک ساختمان داده LIFO (Last In First Out)می گویند.

۱- **پیاده سازی پشته :** استفاده از آرایه می باشد . به این ترتیب یک آرایه یک بعدی در نظر میگیریم و متغیر به نام TOP با مقدار اولیه "۱-" تعریف میکنیم ، Top به عنصر بالا پشته اشاره میکند حال باید توابعی برای اضافه کردن عنصر بالایی پشته و حذف آن را بنویسیم.

```
int Push(int x)
{
    if (top < Max - 1)
    {
        top++;
        S[top] = x;
        return 1;
    }
    return 0;
}
```

```
int Pop()
{
    int X;
    if (top > -1)
    {
        X = S[top];
        top--;
        return X;
    }
    return 0;
}
```

Int S[5]; → MAX

Int Top=-1;

S[4]		TOP
S[3]	۶	TOP
S[2]	۴	TOP
S[1]	۸	TOP
S[0]	۲	TOP
	-1	

Push(2);

Push(8);

Push(4);

Push(6);

Pop (); → 6

Pop(); → 4

Pop (); → 8

Pop (); → 2

- شرط خالی بودن پشته : Top=-1
- شرط پر بودن رشته : Top=Max-1

مثال) فرض کنید پشته ای وجود داشته باشد که در آن بتوان کارکتر ها را قرار داد . برنامه ای بنویسید که یک رشته را دریافت کند و با استفاده از پشته آن را معکوس نماید.

```
void main()
{
    char S[100];
    int i;
    gets(S);
    for (i = 0; S[i]; i++)
```

```
        Push(S[i]);
    for (i = 0; top > -1; i++)
        S[i] = Pop();
}
```

در حالتیکه فقط یک پشته در برنامه وجود داشته باشد S و Topبه صورت سراسری تعریف می شوند در صورتیکه بخواهیم در برنامه بیش از یک پشته داشته باشیم از ساختار زیر استفاده می کنیم.

مثال) توابع Push() **و** Pop() **را برای پشته با ساختار بالا بازنویسی نمایید.**

```
Struct Stack
{
 Int S[Max];
 Int Top;
};
```

```
int Push(Stack *A, int x)
{
    if (A->Top < Max - 1)
    {
        A->Top++;
        A->S[A->Top] = x;
        return 1;
    }
    return 0;
}
```

```
int Pop(Stack *A)
{
    int X;
    if (A->Top > -1)
    {
        X = A->S[A->Top];
        A->Top--;
        return X;
    }
    return 0;
}
```

نکته : در صورتیکه تعداد عناصریکه می خواهند در پشته قرار گیرند از قبل مشخص نباشد از ساختار زیر استفاده می شود. در ساختار رو به رو S به حافظه ای که برای پشته تخصیص داده شده است اشاره می کند قبل از استفاده از پشته باید آن را با استفاده از تابع زیر ایجاد کنیم.

```
Struct Stack{
Int * S;
Int Max;
Int Top;
};
```

```
void Create(Stack *A, int m)
{
    A->S = (int*)malloc(2 * m);
    A->Max = m;
    A->Top = -1;
}
```

کاربرد صف در محیط های چند برنامه ای : در محیط های چند برنامه ای یک کامپیوتر وجود دارد و چندین کاربر که برنامه های خود را برای اجرا آن واگذار میکند. در این حالت کامپیوتر می تواند سیاست های مختلفی را برای پاسخگویی برنامه ها داشته باشد یکی از این سیاست های روش FIFO می باشد در این حالت می توانیم ، برنامه هایی در حال انتظار را در یک صف قرار دهیم.

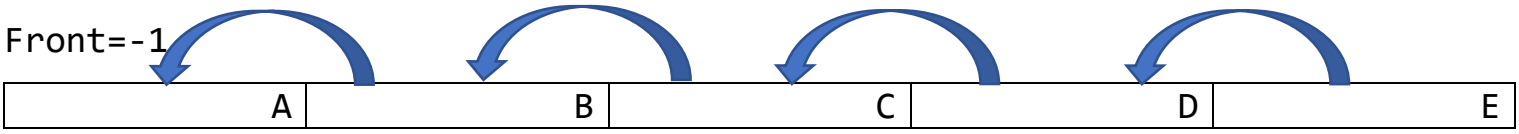
مثال) فرض کنید در یک محیط چند برنامه ای صفی به طول چهار وجود دارد (مثال رو به رو) ملاحظه میشود با وجود اینکه در ابتدای آرایه های خالی وجود دارد ، نمی توان عنصر جدیدی را به صف اضافه نمود چون (Rear = Max-1)

توضیحات	Q[0]	Q[1]	Q[2]	Q[3]	Front	Rear
وضعیت اولیه صف	-	-	-	-	-۱	-۱
برنامه P1 **وارد میشود**	P1	-	-	-	-۱	۰
برنامه P2 **وارد میشود**	P1	P2	-	-	-۱	۱
برنامه P3 **وارد میشود**	P1	P2	P3	-	-۱	۲
برنامه P1 **خارج میشود**	-	P2	P3	-	۰	۲

۳	۰	P4	P3	P2	-	**برنامه** P6 **وارد میشود**
۳	۱	P4	P3	–	–	**برنامه** P2 **خارج میشود**
(rear=max-1) پس صف پر است و نمی تواند عنصر جدیدی را اضافه کند.						**برنامه** P5 **می خواد وارد شود**

❖ برای رفع این مشکل به یکی از دو روش زیر استفاده می شود.

صف شیفتی : در این نوع صف هر گاه عنصری در ابتدای صف حذف می شود عناصر باقیمانده در صف به سمت ابتدای آرایه شیفت داده می شوند در این نوع صف همیشه عنصر صف در Q[0] وجود دارد و Front=-1 است در صف شیفتی هرگاه Rear=Max-1 شود صف واقعا پر است.



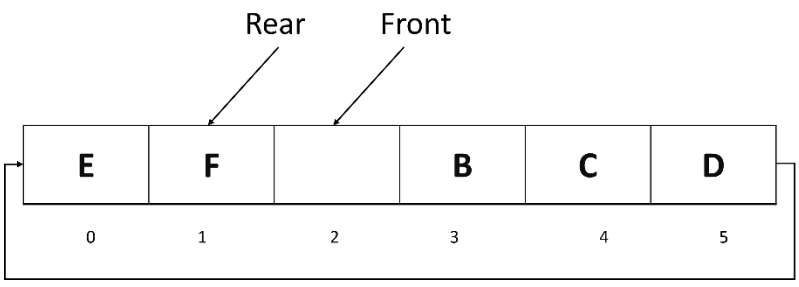
مثال) توابع AddQ()،DeleteQ() را برای صف شیفتی بازنویسی کنید.

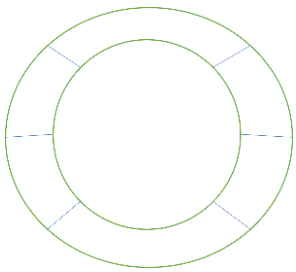
❖ تابع AddQ() که مانند صف معمولی می باشد ولی DeleteQ() به اینصورت است که وقتی عنصری از ابتدای صف حذف شود باید صف یکی به طرف ابتدای آرایه شیفت داده ش

```
int DeleteQ(Queue *A)
{
    int X, i;
    if (A->Rear >= 0)
    {
        X = A->Q[0];
        for (i = 0; i < A->Rear; i++)
        {
            A->Q[i] = A->Q[i + 1];
        }
        A->Rear--;
        return X;
    }
    return -1;
}
```

❖ مشکل اصلی در صف شیفتی این است که شیفت دادن عناصر صف وقت گیر است.

۲- **صف حلقوی :** در این صف هنگامیکه انتهای صف به انتهای آرایه برسد ، در ابتدای آرایه جای خالی وجود داشته باشد عناصر جدید در ابتدای آرایه درج می شوند.





Max=6

Rear=(Rear+1)%Max

باقیمانده = %

مثال) توابع AddQ()، DeleteQ() را برای صف حلقوی بازنویسی کنید.

بطور کلی در صف Front همیشه ، یکی از ابتدای صف جلو تر است ، در صف حلقوی عنصر Q[Front] همیشه یکی از ابتدای صف جلوتر است ، در صف حلقوی عنصر Q[Front] همیشه بلا استفاده باقی میماند (برای تشخیص خالی بودن یا پر بودن صف)

شرط خالی بودن صف : Rear=Front

وضعیف اولیه صف : Front = Rear = 0

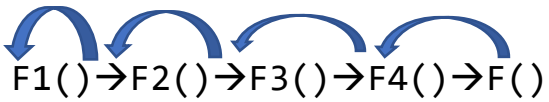
شرط پر بودن صف : (rear+1)%Max=Front

```
int AddQ(Queue *A, int x)
{
    int t;
    t = (A->Rear + 1) % Max;
    if (t != A->Front)
    {
        A->Rear = t;
        A->Q[A->Rear] = x;
        return 1;
    }
    return 0;
}
```

```
int DeleteQ(Queue *A)
{
    if (A->Front == A->Rear)
        return -1;

    int X = A->Q[A->Front];
    A->Front = (A->Front + 1) % Max;
    return X;
}
```

کاربرد پشته در فراخوانی تابع : هنگامی که دستور برای برنامه صادر میگردد ، کد اجرایی برنامه به حافظه اصلی منتقل می شود و برنامه دستور به دستور اجرا میگردد هر یک از دستور های برنامه در آدرس مشخص از حافظه قرار میگیرد ، هنگامیکه یک تابع فراخوانی می شود ، کنترل برنامه به تابع منتقل میگردد ، دستورات تابع اجرا می گردند ، سپس کنترل برنامه به دستور بعد از فراخوانی باز میگردد. پس قبل از فراخوانی تابع باید آدرس بازگشت فرا خواننده را در محلی نگهداری کنیم ، ضمناً توابع را میتوان بصورت تو در تو فراخوانی نمود ، ترتیب بازگشت از توابع عکس ترتیب فراخوانی آنها می باشد بنابراین می توانیم از ساختمان داده پشته برای نگهداری آدرس های بازگشت استفاده نماییم.



کاربرد پشته در ارزیابی عبارت: یک عبارت مجموعه ای از عملوند ها ، عملگر ها و حایل ها می باشد . که به شکل مناسبی در کنار یکدیگر قرار گرفته اند.

حایل ( )	عملگر (+)	عملوند=A,B	عبارت=A+B*7/(A-B*C)

انواع عبارت :

۱- Infix (میانوندی) (A+B) : در این نوع عبارت عملگر بین دو عملوند آن می آید.
۲- Prefix (پیشوندی) (+AB) : در این نوع عبارت ابتدا عملگر و بعد از آن در عملوند آن می آیند.
۳- PostFix (پسوندی) (AB+) : در این نوع عبارت ابتدا دو عملوند سپس عملگر مربوطه می آید.

- ما عبارات را بصورت Infix مینویسیم ، اما محاسبه عبارات بصورت PostFix یا PreFixبرای کامپیوتر راحت تر می باشد لذا از کامپایلر ها ، عبارات را ابتدا به فرم PostFix یا PreFix تبدیل میکنند و سپس مقدار عبارات را محاسبه می نمایند.

InFix	PreFix	PostFix
A+B	+AB	AB+
A+B*C	+A*BC	ABC*+
A*B-C	-*ABC	AB*C-
A+B-C	-+ABC	AB+C-
A*B-C/D	-*AB/CD	AB*CD/-
A*B/C-F*G+H*K	+-/*ABC*FG	*HKAB*C/FG*HK*+
A*(B-C/(F+G))	*A-B/C+FG	ABCFG+/-*

- **نکته :** در تبدیل عبارات اولویت عملگر ها را باید در نظر گرفت .
- **نکته :** یک عملگر هنگامی اعمال می شود که عملوند آن محاسبه شده باشند.
- **نکته :** ترتیب عملوند ها هنگام تبدیل عبارات تغییر نمیکند.

محاسبه عبارت Prefix : از ابتدای عبارت شروع می کنیم و آنقدر جلو میرویم که به یک عملگر که پشت سر آن دو عملوند باشد برسیم عملگر داده ی دو عملوند اعمال میکنیم ، حاصل خود یک عملوند است . مجدداً از ابتدا این کار را تکرار میکنیم تا در نهایت ، عبارت محاسبه گردد. *AB=X /CD=Y

مثال: -*AB/CD → -XY → X-Y → A*B-C/D

محاسبه عبارت PostFix : از ابتدا عبارت شروع میکنیم ، آنقدر جلو میرویم تا به دو عملوند که پشت سر آنها یک عملگر باشد برسیم ، عملگر ها را روی دو عملوند اعمال میکنیم. این عمل را تکرار میکنیم تا در نهایت عبارت محاسبه شود.

مثال: ABCFG+/-*→A*Z→A*(B-Y)→A*(B-C/(F+G))

تبدیل عبارت Infix به PerFix با استفاده از پرانتز گذاری :

در این روش ابتدا عبارت را با توجه به اولویت عملگر ها ، پرانتز گذاری میکنیم بطوریکه در نهایت به ازای هر عملگر یک جفت پرانتز وجود داشته باشد سپس به جای پرانتز های باز عملگر مربوط را قرار میدهیم و در نهایت پرانتز های بسته را حذف مینماییم.

۱- A + B → (A + B) = +AB

۲- A + B – C → ((A + B – C) = -+ABC

۳- A + B * C → (A + (B * C)) → +A*BC

۴- A + B / C – F * G + H * K → ((((A * B) / C – (F * G)) + (H * K))

+-/*AB*FG*HK

۵- (A*(B-(C/(F+G)))) * A – B / C + FG

تبدیل عبارت InFix به PostFix با استفاده از پرانتز گذاری : در این حالت نیز ابتدا عبارات را پرانتز گذاری میکنیم ، سپس پرانتز های باز را حذف مینماییم و به جای پرانتز های بسته عملگر مربوط را قرار میدهیم.

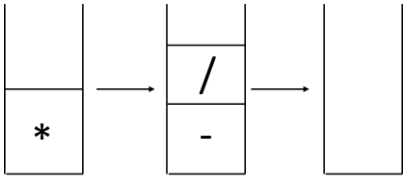
((((A * B) / C) - (F * G)) + (H * K)) → AB* C/FG *-HK * +

تبدیل عبارات InFix به PostFix با استفاده از پشته (روش کمپایلر) : در این روش ، پشته ای در نظر میگیریم در این پشته عملگر ها قرار میگیرند ، عبارت را از چپ به راست بررسی میکنیم هر عملوند ، عملگر یا حایل خوانده شده را یک Token میگوییم و بر حسب Token اعمال زیر را انجام میدهیم.

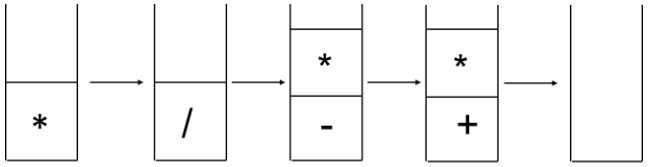
۱- اگر Token عملوند بود مستقیماً آن را به خروجی می بریم.

۲- اگر Token عملگر بود ، باید در پشته قرار داده شود قبل از قرار دادن ، کلیه عملگر هایی که در بالای پشته قرار دارند و اولویتشان بیشتر یا مساوی Token بیشتر یا مساوی Token خوانده شده است . POP می کنیم و در خروجی قرار میدهیم هنگامیکه به انتهای عبارت رسیدیم ، عملگر های باقیمانده در پشته را POP می نماییم و به خروجی منتقل می کنیم.

۱- A*B-C/D → خروجی: AB*CD/-



۲- A*B/C-F*G+H*K → خروجی : AB*C/FG*-HK*+



تبدیل عبارت پرانتز دار Infix به postfix (با استفاده از پشته) : در این روش پشته ای برای عملگر ها و پرانتز های باز در نظر میگیریم ، عبارت را از چپ به راست بررسی میکنیم و بر حسب نوع Token اعمال زیر را انجام میدهیم:

۱- اگر Token عملوند بود ، آن را مستقیماً به خروجی می بریم.

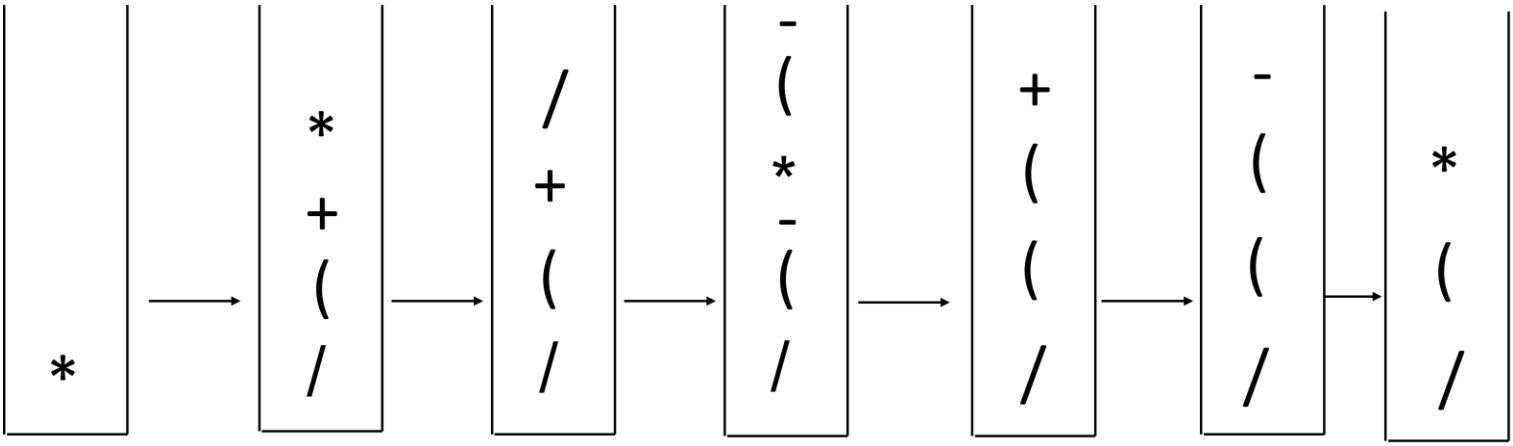
۲- در صورتیکه Token عملگر بود باید در پشته قرار گیرد ، قبل از آن کلیه عملگر هاییکه در بالای پشته قرار دارند و اولویتشان بیشتر یا مساوی Token خوانده شده است Pop می کنیم و به خروجی منتقل می نماییم.

۳- اگر Token پرانتز بود ، آن را در پشته قرار می دهیم ، پرانتز باز هنگامیکه می خواهد در پشته قرار گیرد بالاترین اولویت را دارد و هنگامیکه در پشته قرار گرفت پایین ترین اولویت به آن داده می شود.

٤- اگر Token پرانتز بسته باشد کلیه عملگر هایی که در بالای پشته قرار دارد تا زمانیکه به یک پرانتز باز برسیم POP می کنیم و به خروجی منتقل میکنیم . پرانتز باز نیز POP می شود ولی در خروجی قرار نمیگیرد هنگامیکه به انتهای عبارت رسیدیم عملگر های باقیمانده در پشته را POP میکنیم و در خروجی قرار می دهیم.

A*B/(C+F*G/H-K*(1-M))/((R+S+P+Q)*D*X)

خروجی: AB*CFG*H/+1CLM-*-/RS+P+Q-D*X*



محاسبه عبارت PostFix: همانطور که گفته شده محاسبه عبارات به فرم PostFix راحت تر می باشد برای این منظور از یک پشته استفاده میکنیم در پشته عملوند ها قرار میگیرند عبارت PostFix را از چپ به راست بررسی میکنیم .

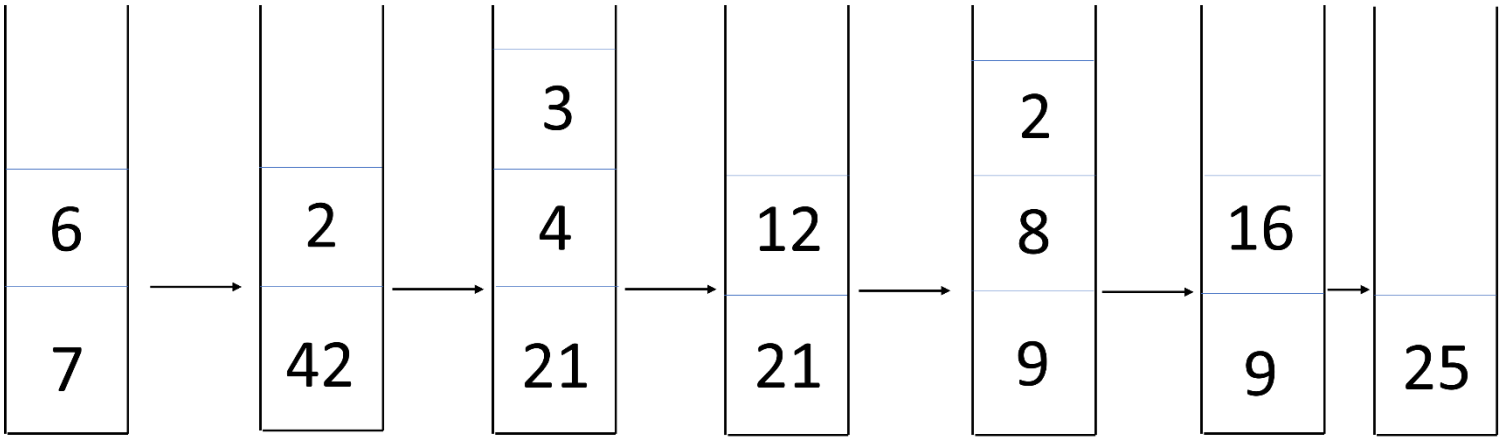
۱- اگر Token عملوند بود آن را در پشته قرار میدهیم.

۲- اگر Token عملگر بود ، دو عملوند را از بالای پشته POP می کنیم . عملگر داده ی دو عملوند اعمال میکنیم وحاصل را مجددا در پشته قرار میدهیم.

هنگامیکه به انتهای عبارت برسیم یک عدد در پشته قرار دارد که حاصل عبارت می باشد.

۷	٦	۲	٤	۳	۸	۲

A * B / C – F * G + H * K → PostFix=AB*C/FG*-HK*+



توجه : برای ساده شدن متغیر ها را یک فرمی در نظر بگیرید . الگوریتم کار به اینصورت است که ابتدا باید Token ها را از هم جدا نماییم سپس متغیر ها را تشخیص دهد و مقدارشان را بگیرد . سپس عبارت را به PostFix تبدیل می کنیم در نهایت آنرا محاسبه نماییم.

۰	$\propto$ $\propto$+۱	MAX-۱

صف و پشته چندگانه : ” Multi Stack & Queue” :

S1	S2

۱- Stack :

S1 { ۱- شرط خالی بودن Top1=-1 ۲- شرط پر بودن Top1=$\propto$

S2 { ۱- شرط خالی بودن Top2=$\propto$ ۲- شرط پر بودن Top2=MAX-1

برای از بین بردن خاصیت خالی بودن یکی و پر بودن دیگری :

S1 { ۱- شرط خالی بودن Top1=-1 ۲- شرط پر بودن |TOP1-TOP2|=1

S2 { ۱- شرط خالی بودن Top2=MAX ۲- شرط پر بودن |TOP1-TOP2|=1

❖ در صف و پشته چندگانه از یک آرایه برای ذخیره کردن چندین صف یا پشته استفاده میکنیم در این حالت در صف چندگانه آرایه را به چند بخش تقسیم می کنیم و هر یک از صف ها را در یکی از بخش های آرایه ذخیره میکنیم.

مثال : دارای ۷ عنصر می باشد.

۰	۱	۲	۳	٤	٥	٦

TOP1	TOP2	
۱- لحظه شروع	۷	
۰	۷	ADS1('K')
۰	۶	ADS2('M')
۱	۶	ADS1('P')
۲	۶	ADS1('J')
۲	۵	ADS2('T')
۳	۵	ADS1('B')
۴	۵	ADS1('D')
		ADS1('R')
		ADS2('X')



Stack Full

در حالت کلی برای n تا Stack می توان داشته باشیم:

۰	$\propto$ $\propto$+۱	B B+۱	MAX-۱
S1	S2	----------	Sn

و برای صف نیز داریم

۰	$\propto$ $\propto$+۱	MAX-۱
Q1	Q2	

❖ وضعیت اولیه : F1=R1=-1

Q1 { ۱- شرط خالی بودن F1=R1 ۲- شرط پر بود R1 = $\propto$ }

❖ وضعیت اولیه F2=R2=$\propto$

۱- شرط خالی بودن F2=R2

۲- شرط پر بودن R2=Max-1

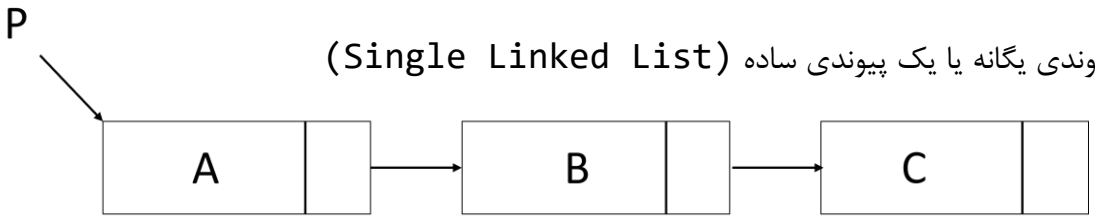
❖ حالت دو تیکه کردن صف را نیز میتوانیم داشته باشیم کافی است در مسئله قبل |R1-R2|=1 باشد و می توان بوجود آورد .

❖ و بطور مشابه برای n تا صف نیز می توانیم پیاده سازی نماییم.

۰ ∝ ∝+۱ B B+۱ MAX-۱

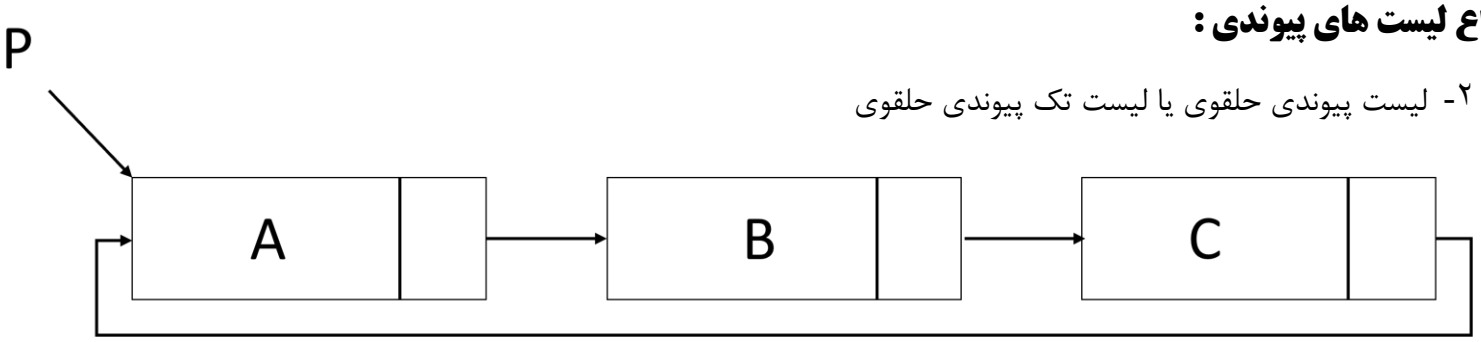
Q1	Q2	----------	Qn

لیست پیوندی : Linked Lists :

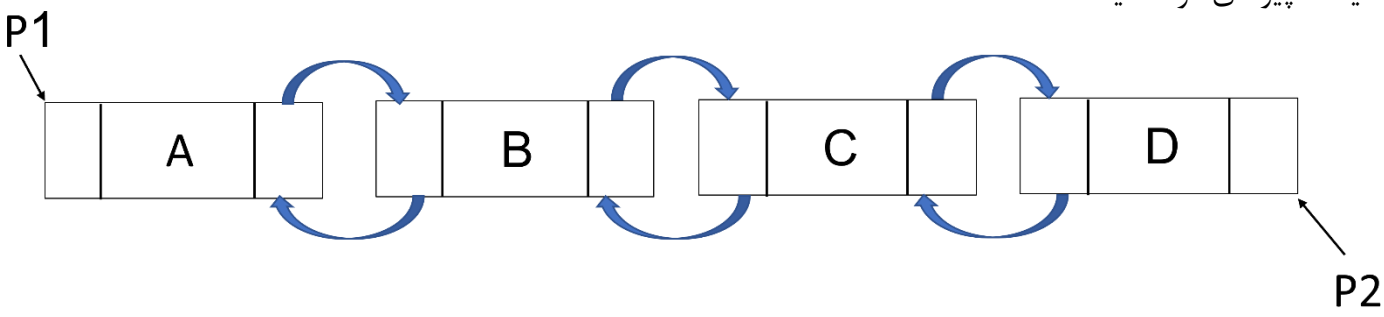


۱- لیست پیوندی یگانه یا یک پیوندی ساده (Single Linked List)

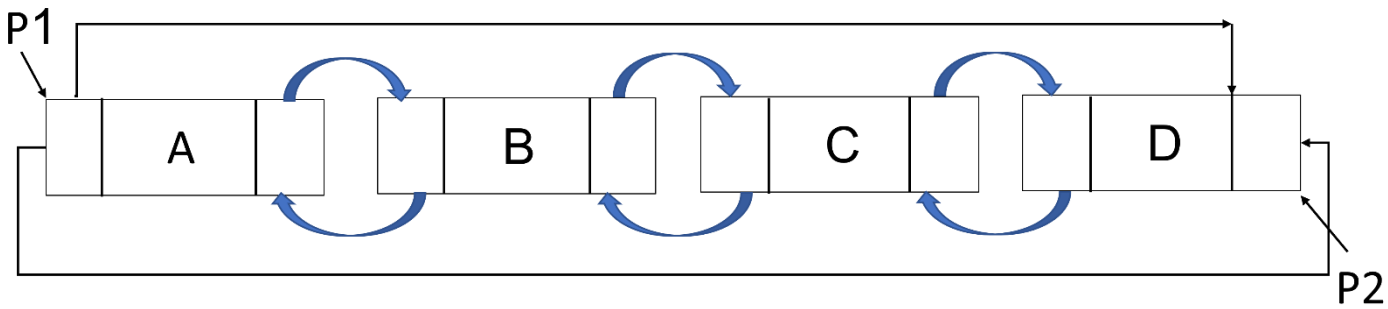
انواع لیست های پیوندی :



۲- لیست پیوندی حلقوی یا لیست تک پیوندی حلقوی

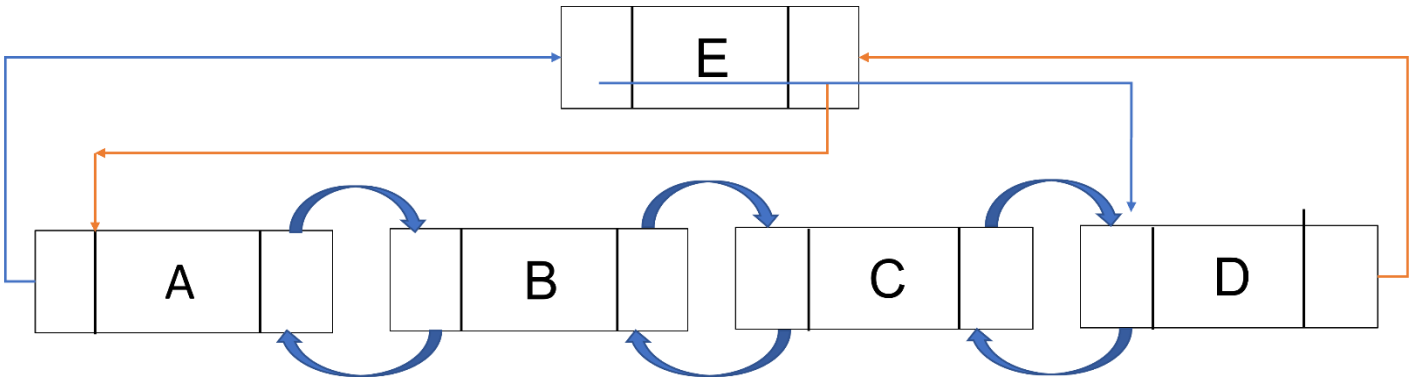


۳- لیست پیوندی دوگانه یا مضاعف:



٤- لیست پیوندی دوگانه / مضاعف حلقوی:

۵- لیست پیوندی دوگانه / مضاعف حلقوی با گره Head



تخصیص حافظه پویا: در برنامه هایی که میزان حافظه مورد نیاز برنامه از قبل مشخص نیست از مکانیزم تخصیص یافته حافظه بصورت پویا استفاده می کنیم با استفاده از این مکانیزم برنامه در حال اجرا می تواند حافظه مورد نیاز خود را از سیستم کسب نماید و پس از استفاده آنرا به سیستم بازگرداند.

تابع Malloc (): این تابع برای تخصیص حافظه بصورت پویا استفاده می شود ، تابع Malloc در حافظه اصلی به دنبال تعداد بیت مشخص شده می گردد در صورتیکه این تعداد بایت بصورت آزاد و پشت سر هم وجود داشته باشد آن را به برنامه اختصاص میدهد و آدرس شروع آنرا باز میگرداند ، این آدرس معمولاً در یک اشاره گر ذخیره میشود و ازطریق این اشاره گر به حالت حافظه اختصاص یافته دسترسی پیدا میکنیم.

(تعداد بایت مورد نظر) Malloc: اگر تابع Malloc نتواند حافظه مورد نیاز را به برنامه اختصاص دهد مقدار Null باز میگرداند.

تابع Free() : پس از اینکه کار ، با حافظه اختصاص یافته پایان یافت باید آنرا آزاد کنیم برای این منظور از تابع Free استفاده میکنیم.

Free(آدرس شروع حافظه مورد نظر)

تابع New: در C++ از دستور New به جای Malloc برای تخصیص حافظه استفاده می کنیم :

نام اشاره گر = New نوع اشاره گر→ P=New int;

دستور بالا به اندازه دوبایت (طول Integer) حافظه به برنامه اختصاص می دهد و آدرس شروع آن را در اشاره گر P قرار میدهد.

دستور Delete: در C++ دستور فوق حافظه اختصاص داده شده به برنامه توسط New را آزاد می کند.

Delete نام اشاره گر ;→ Delete P;

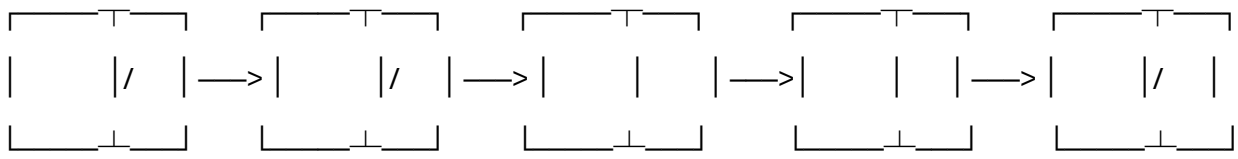
در برنامه هایی که با تخصیص حافظه به صورت پویا سر و کار دارند ، معمولاً روال کار به این صورت است که هرگاه برنامه حافظه جدیدی نیاز داشت به آن اختصاص داده می شود ، به عنوان مثال در یک سیستم اطلاعات دانشجو هرگاه که بخواهیم اطلاعات دانشجویی جدیدی را دریافت کنیم به اندازه رکورد دانشجو حافظه در نظر میگیریم ، و اطلاعات دانشجویی جدید را در آن ذخیره می نماییم طبیعتاً قطعات تخصیص یافته به برنامه در محل های مختلفی از حافظه قرار دارند.

برای کار با آن قطعات باید آدرس‌های شروع قطعات را داشته باشیم پس راه حل مناسب استفاده از لیست پیوندی می‌باشد به این ترتیب که در هر قطعه یا گره آدرس بعدی را نگهداری می‌کنیم در این حالت با داشتن آدرس اولین گره می‌توانیم به تک تک گره‌ها دسترسی پیدا کنیم.

Start

|

v



ساختار مناسب برای گره‌های لیست پیوندی، بصورت صفحه بعدی باشد: *دقت داشته باشید که قسمت next آخرین گره به null اشاره می‌کند.

Struct link_list

{

//اطلاعات اصلی گره

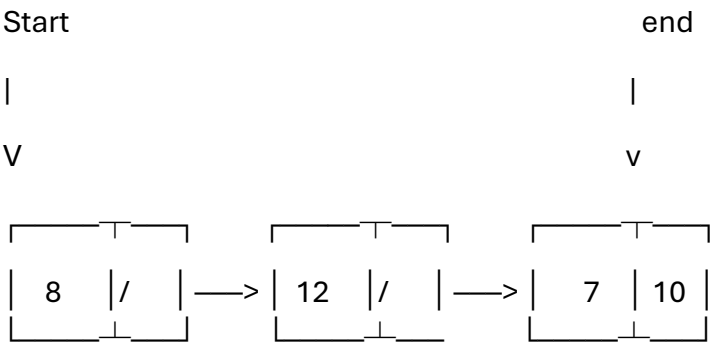
}

مثال: برنامه‌ای بنویسید که تعداد نامشخص عدد صحیح را دریافت کند و از آن‌ها یک لیست پیوندی بسازد وارد شدن عدد صفر به معنی تمام شدن اعداد می‌باشد.

```
Struct Link_list
{
int data;
Link_list *next;
};
void main()
{
Link_list *p , *start = NULL, *end = NULL;
cin>>a;
while(a)
```

```
{p=new Link_list;
if(!p)
}puts("به حافظه کافی وجود ندارد ")
exit(0);
}
p->data = a;
p->next=NULL;
if(!start)
start = end=p;
else
{ end->next=p;
}end=p;
```

Start و End به ترتیب به اولین گره و آخرین گره لیست اشاره میکند هنگامیکه List گره ای ندارد برابر با NULL می باشد.



مثال: اگر یک لیست پیوندی با ساختار بالا وجود دشاته باشد تابعی بنویسید که آدرس شروع لیست را به عنوان پرامتر دریافت کند و محتوایات گره ها را چاپ نماید؟

```
Void print-List(link-list * start){
Link_list *p= start;
While(p)
{
Cout<< p ->data;
```

```
P=p->next;

}

}
```

مثال: فرض کنید ییک لیست پیوندی با ساختار بالا وجود داشته باشد تابعی بنویسید که آدرس شروع لیست پیوندی را به عنوان پارامتر دریافت کند و گره های آنرا آزاد کند؟

```
Link-List *free-list(link-list * start)

{Link-list *p= start;

While*(start)

{

P= start;

Start =start -> next;

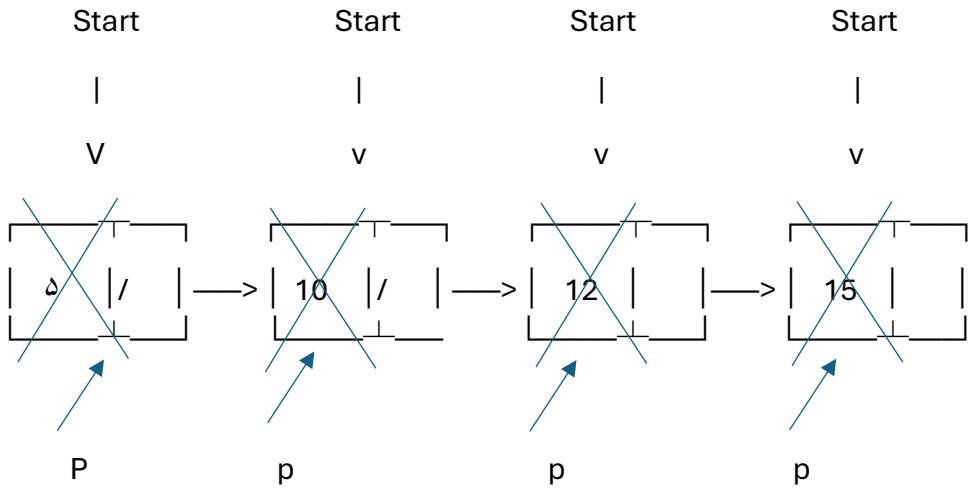
Delete p ;

}

Return start;

}
```

*طبیعتا این تابع باید همیشه مقدار NULL را برگرداند که نشان میدهد لیست خالی شده است.



تمرین: تابعی بنویسید که آدرس شروع یک لیست پیوندی از اعداد را به عنوان پارامتر دریفت کند و لیست را مرتب نماید(با جابجایی محتویات گره ها)

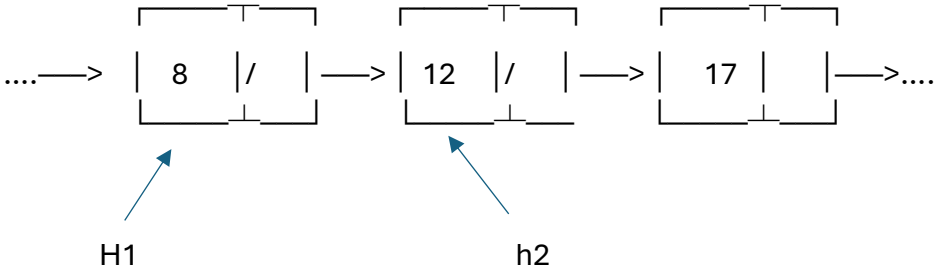
*تمرین: فرض کنید دو لیست مرتب شده از اعداد وجود داشته باشد تابعی بنویسید که آدرس شروع این دولیست را به عنوان پرامتر دریافت کند و لیستها را در یکدیگر ادغام کند (بدون استفاده از لیست کمکی)

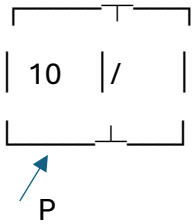
توضیح: تابع باید آدرس شروع لیست را بر گردانه در این تمرین در واقع باید یکی درگیری درج شود

*تمرین:تابعی بنویسید که آدرس شروع یک لیست پیوندی را به عنوان پرامتر دریافت کند و لیست را معکوس نماید با جابجا کردن اشاره گرهای گره ها

تمرین:تابع بازگشتی بنویسید که آدرس شروع یک لیست پیوندی را به عنوان پارامتر دریافت کند و لیست را به صورت معکوس چاپ نمایید(بدون ایجاد تغییر در لیست)

لیست پیوندی مرتب: در لیست پیوندی مرتب گرهای لیست بر اساس یکی زا اقلام اطلاعاتی موجود در گره مرتب می باشد، برای تشکیل لیست پیوندی مرتب کافی است هرگاه می خواهیم گره جدیدی به لیست اضافه کنید، ابتدا محل مناسب آن در لیست را پیدا کنیم و سپس گره را در آن محل درج نماییم.





فرض کنید P به گره ای که می خواهند اضافه شود

اشاره کند و همچنین h1 به گره قبلی و h2 به گره بعد از آن در لیست اشاره نماید هنگام اضافه کردن گره، حالتهای زیر ممکن است اتفاق افتد.

۱اولین گره لیست (start=Null):

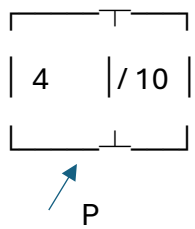
{p-> next =null; (p->next=start);

{start=p;

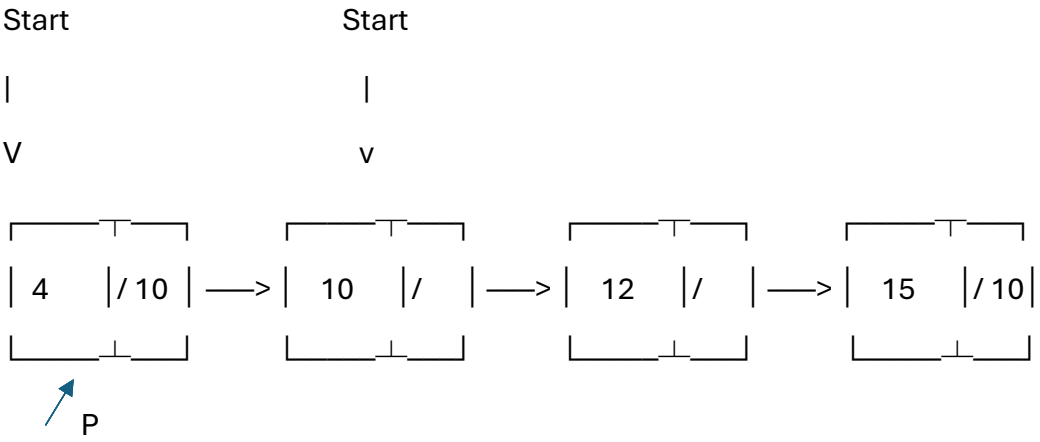
Start

|

v



۲ اضافه کردن گره به ابتدای لیست:..



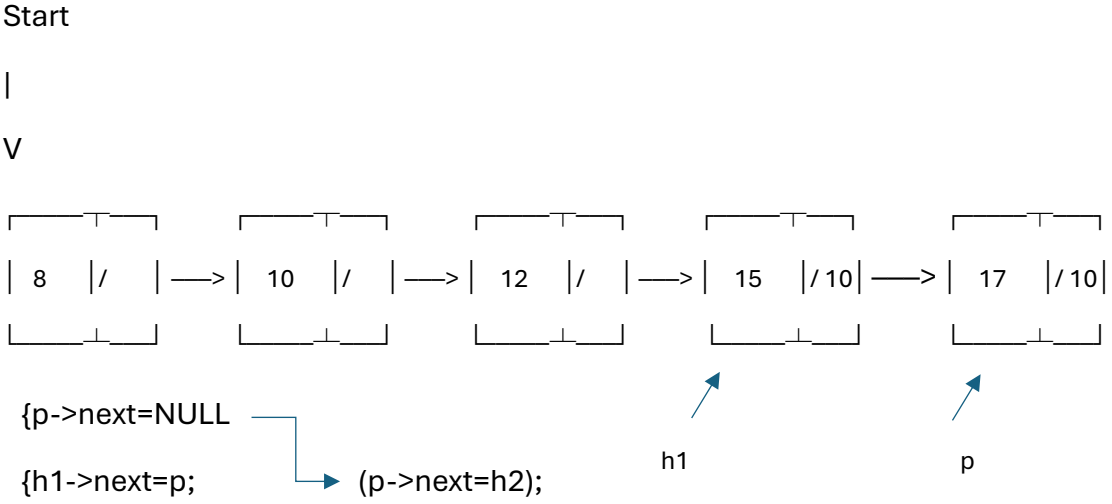
{p-> next =null; (p->next=start);

{start=p;

ملاحظه شود که در حالتهای یک و دو

دستورات یکسانی اجرا می شوند بنابراین: این حالت را می توان در یکدیگر ترکیب نمود:

۳ اضافه کردن گره به انتهای لیست:

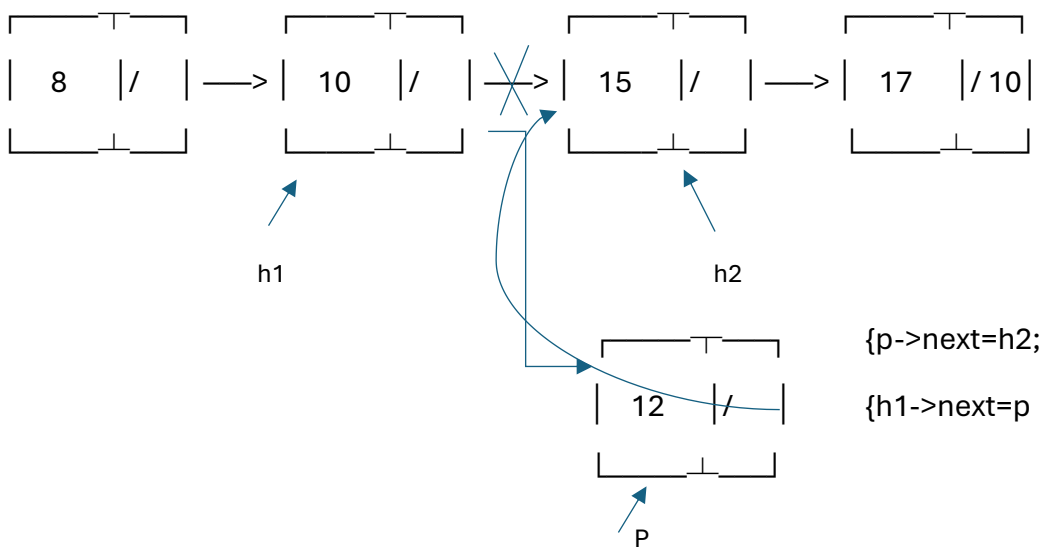


4 اضافه گردن گره به وسط لیست:

Start

|

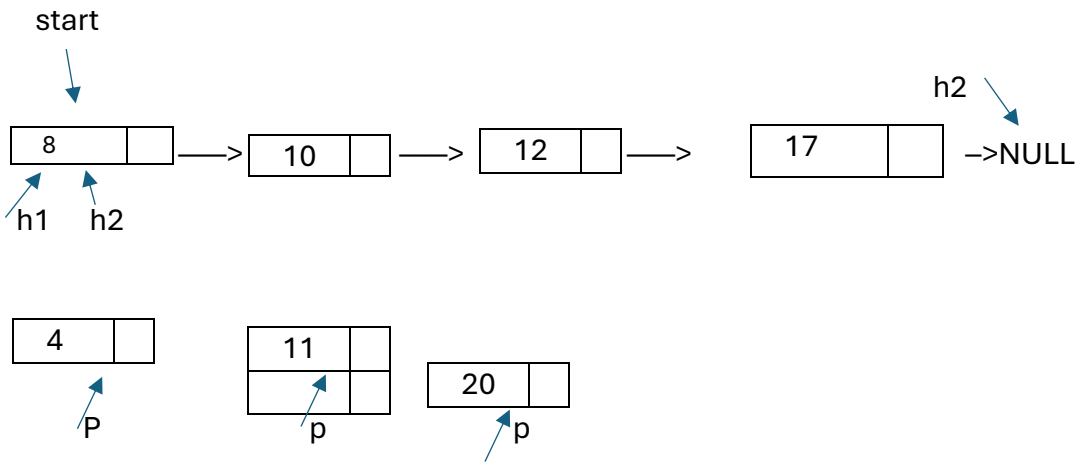
V



*حالتهای ۳ و ۴ را نیز می توانیم در یکدیگر ترکیب کنیم

مثال تابعی بنویسید که آدرس شروع یک لیست پیوندی و یک عدد را به عنوان پارامتر دریافت کند و پس از تشکیل گره برای عدد آنرا در محل مناسب درج نماید:

```
Link-list *add_node(link-list *start, int x)
{
Link_list *hi, *h2,*p;
P = new Link-list;
 P->data = x;
H2=start;
While(h2 && h2 ->Data<x)
{1
h1=h2;h2=h2->next
}
If(h2==start)
{
p->next=start;
```



```
start=p;

}

else

{

p->next=h2;

h1->next=p;

}

return start;

}
```

تمرن:فرض کنید که h آدرس گره ای از لیست پیوندی باشد، p به یک گره جدید اشاره می کند تابعی بنویسید که مقادیر این دو اشاره گر را به عنوان پارامتر دریافت کند و گره p را لبه از گره h در لیست اضافه کند.

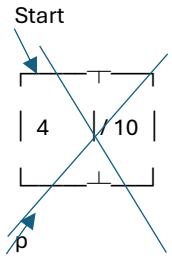
تمرین: تابع بالا را طوری تغییییر دهید که گره p را قبل از گره h اضافه کند تابع باید آدرس شروع لیست را بازگرداند.

حذف گره از لیست پیوندی: فرض می کنیم P به گره ای که می خواهد حذف شود و h به گره قبل از آن در لیست اشاره کند هنگام حذف گره، حالتهای زیر ممکن است پیش آید.

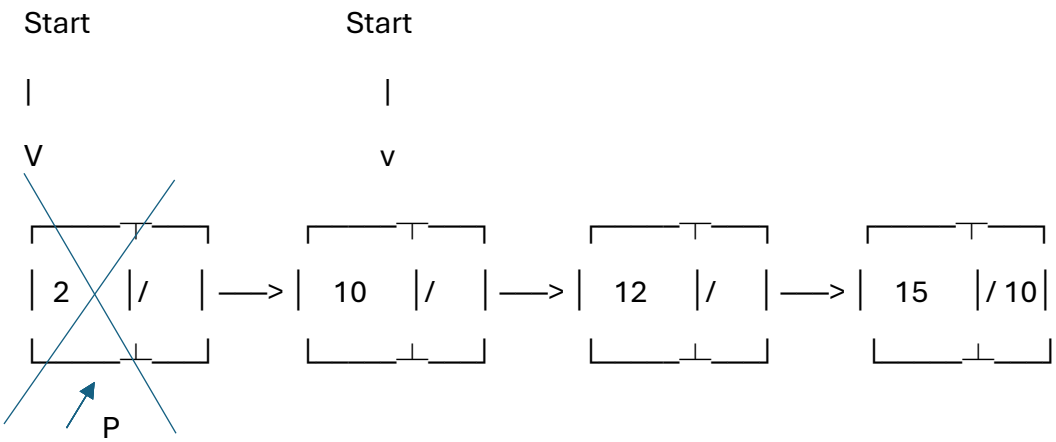
۱ حذف تنها گره لیست: (start=start -> next)

Start=null

delete p;



۲حذف اولین گره لیست:

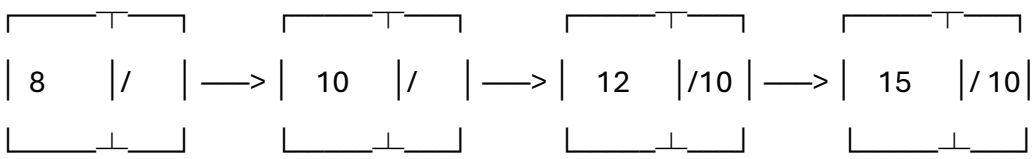



```
Start=start->next;

delete p;
```

با توجه به فوق و حالتهای ۱ و ۲ را می توانید در یکدیگر ادغام نماییم.

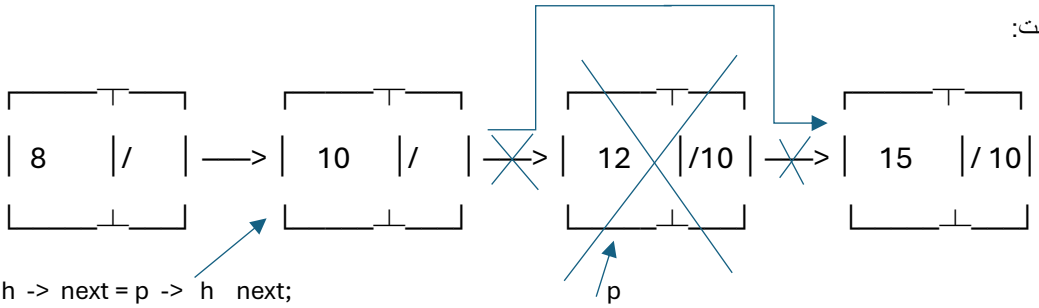
۳. حذف گره از انتهای لیست:



h->next = NULL; h p (h->next=p->next;)

delete p;

۴. حذف گره ازوسط لیست:



h -> next = p -> h next; p

delete p;

*باز با توجه به فوق حالتهای ۳ و ۴ را نیز می توانیم در یکدیگر ترکیب نمایئیم.

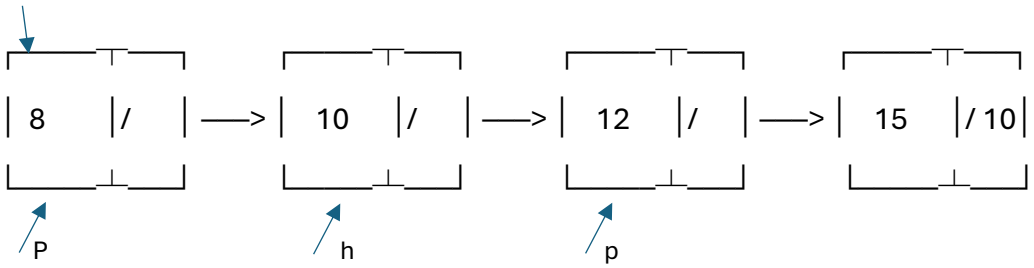
مثال: تابعی بنویسید که آدرس شروع یک لیست پیوندی و یک عدد را بعنوان پارامتر دریافت نماید و گره مربوط به عدد را در صورت وجود از لیست حذف نماید؟

```
Link_list * finded(link-list * start, int x)
{
Link_list * h, *p;
P=start;
While(p && x !=p ->data)
{
h=p;
p=p->next;
}
If(p)
{
If(p==start)
start = start ->next;
delete p;
}
```

return start;

}

start



*شرط if(p) باعث می شود که برنامه چک می کند

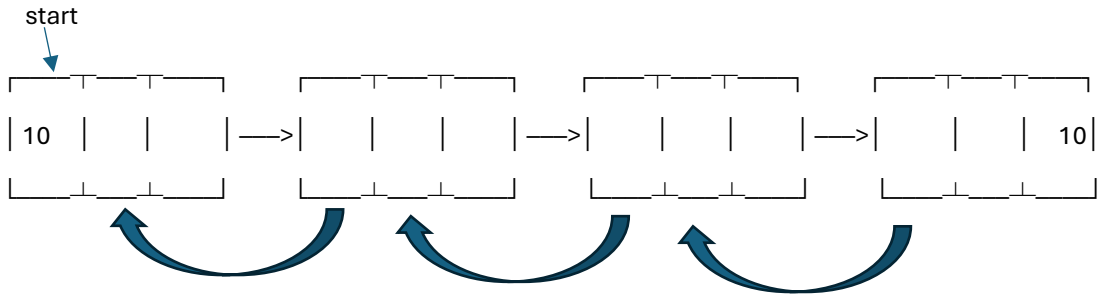
که آیا اصلا گره مورد نظر در لیست هست یا خیر، اگر نباشد p آنقدر جلومی رود تا NULL شود

تمرین مهم: تابعی بنویسید که آدرس شروع لیست پیوندی و آدرس یک گره از لیست را به عنوان پارامتر دریافت کند و این گره را از لیست حذف نماید.

تمرین مهم: فرض کنید یک لیست پیوندی مرتب وجود داشته باشد، در این لیست گره ای موجود است که در محل مناسب خود قرار ندارد، تابعی بنویسید که آدرس شروع لیست را به عنوان پارامتر دریافت کند و این گره را پیدا و حذف نمایید.

تابع باید آدرس روع لیست را بازگرداندد.

لیست دو پیوندی: مشکل لیست یک پیوندی این است که حرکت در آن فقط در یک جهت (از ابتدا به انتها) امکان پذیر می باشد برای رفع این مشکل از لیست دو پیوندی استفاده می کنیم در این نوع لیسیت گره به گره بعدی و گره قبل از خود اشاره میکند.



ساختار مناسب برای گره های لیست دوپیوندی بصورت رو به رو می باشد.

Struct dlink

{

dlink * perior; -> اشاره گر به گره قبلی

//اطلاعات اصلی گره

dlink * next; -> اشاره گر به گره بعدی

}

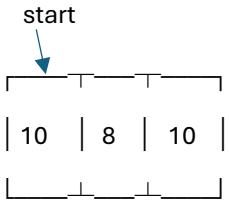
*قسمت perior اولین گره و قسمت next آخرین گره به NULL اشاره می کند.

اضافه کردن گره به لیست دو پیوندی مرتب: فرض می کنیم p به گره ای که می خواهد اضافه شود، h1 به گره قبل از آن و h2 به گره بعد از آن در لیست اشاره نماید هنگام اضافه کردن گره حالتهای زیر ممکن است پیش آید.

۱ اولین گره لیست باشد.

در این حالت چون NULL,start می باشد

تشخیص می دهیم که اولین گره لیست است.



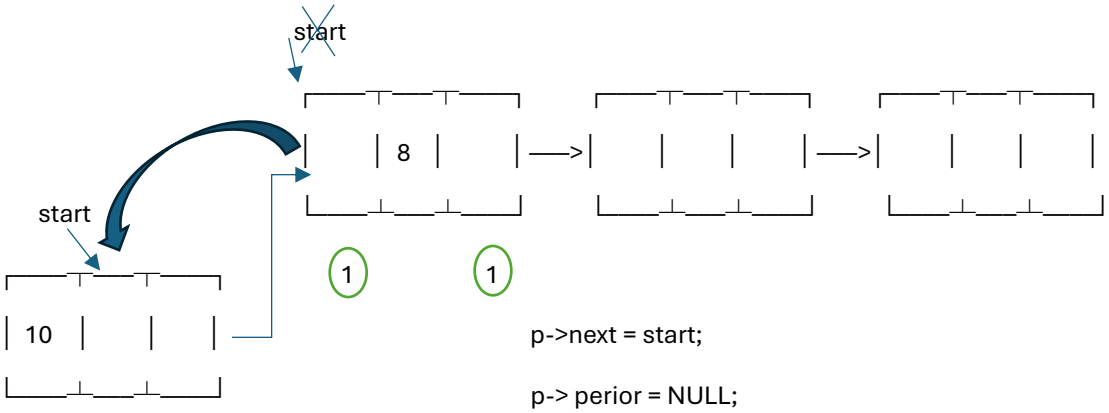
p->next – NULL; p

(p-> next = start;)

p-> perior = NULL;

start = p;

۲. اضافه کردن گره به ابتدای لیست.



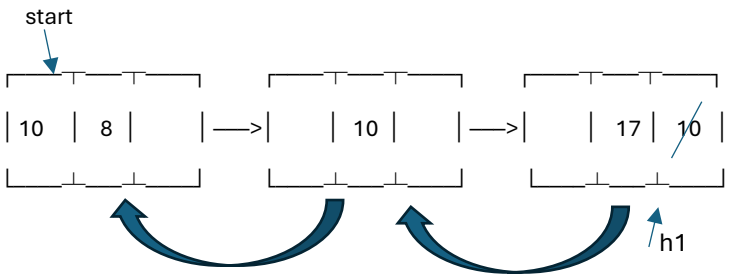
p->next = start;

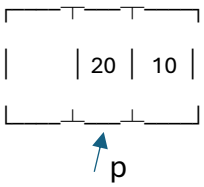
p-> perior = NULL;

Start -> perior = p;

Strt = p;

۳. اضافه کردن گره به انتهای لیست:





p-> next = NULL;

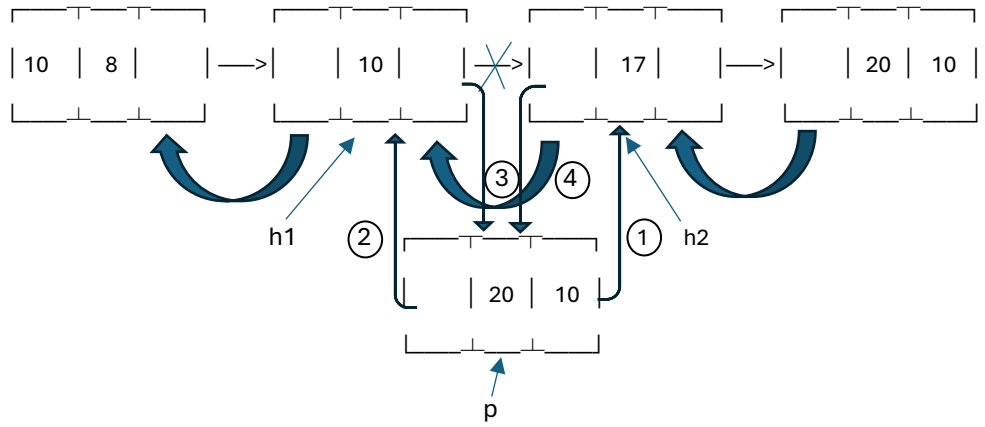
(p -> next=h2);

p-> perior = h1;

h1-> next = p;

۴. اضافه کردن گره به وسط لیست:

start



p-> next = h2;

p->perior = h1;

h1 -> next = p;

h2 -> perior =p;

مثال: تابعی بنویسید که آدرس شروع یک لیست دو پیوندی و یک عدد را به عنوان پارامتر دریافت کند و پس از تشکیل گره برای عدد آنرا در محل مناسب خود درج نماید.

dlink * addnode(dlink * start , int x)

{

dlink *p, *h1,*h2;

p=new dlink;

p->Data = x;

for(h2== start){

p->nest = start;

p->perior = NULL;

if(start)

start->perior=p;

start=p;

}

else{

p->next =h2;

p->perior=h1;

h1->next=p;

if(h2)->پست وسط لیست

H2->perior = p;

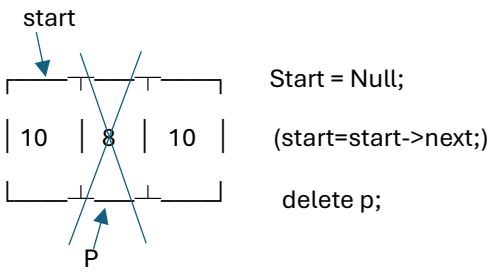
}

return start;

}

حذف گره از لیست دو پیوندی: فرض کنید p به گره ای که می خواهند حذف شود اشاره کند و هنگام حذف گره حالت های زیر ممکن است پیش آید.

۱حذف تنها گره لیست:

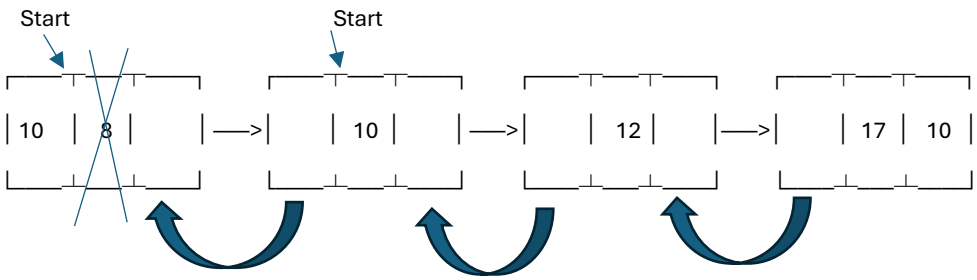


Start = Null;

(start=start->next;)

delete p;

۲. حذف اولین گره لیست:



Start=start->next;

Start->perior = NULL;

delete p;

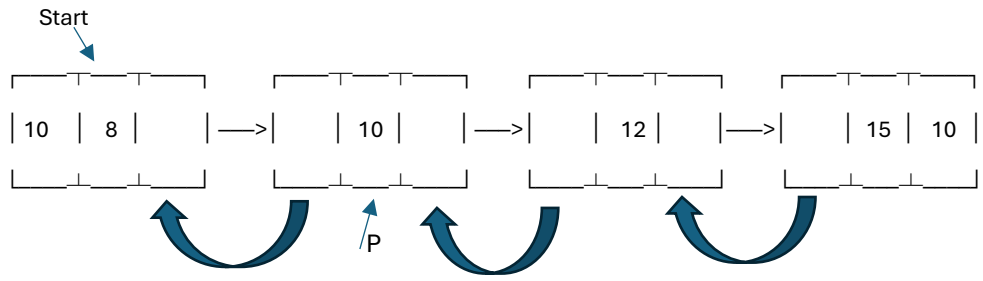
۳. حذف آخرین گره لیست:

p->perior -> next=NULL;

(p->perior->next=p->next;)

delete p;

۴. حذف گره از وسط لیست:



p->perior->next=p->next;

p->next->perior=p->perior;

delete p;

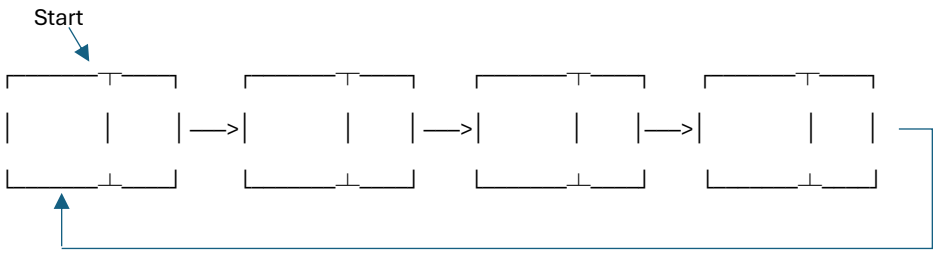
مثال: تابعی بنویسید که آدرس شروع یک لیست دو پیوندی و یک عدد را به عنوان پارامتر دریافت کند و گره مربوط به عدد را در صورت وجود از لیست حذف نماید. (تابع باید آدرس شروع لیست را بازگرداند)

```
dlink *deletenode(dlink * start; int x)
{
        dlink * p;
        for(p=short; p && p->data!=x; p=p->next);
        if(p)
        {
                If(p==start)
                {
                        start= start ->next;
                        if(start)
                                start->perior=NULL;
                        delete p;
                }
                else
                {
                        p->perior->next = p->next;
                        if(p->next)
                                p->next->perior=p->perior;
```

این حلقه کارش این است که جای گره را پیدا می کند

```
                delete p;
            }
        }//if
        return start;
}
```

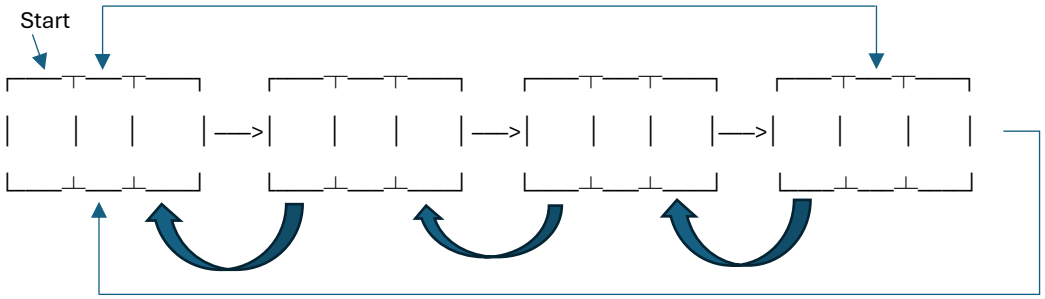
لیست پیوندی چرخشی :



در این نوع لیست آخرین گره به اولین گره لیست اشاره می کند.

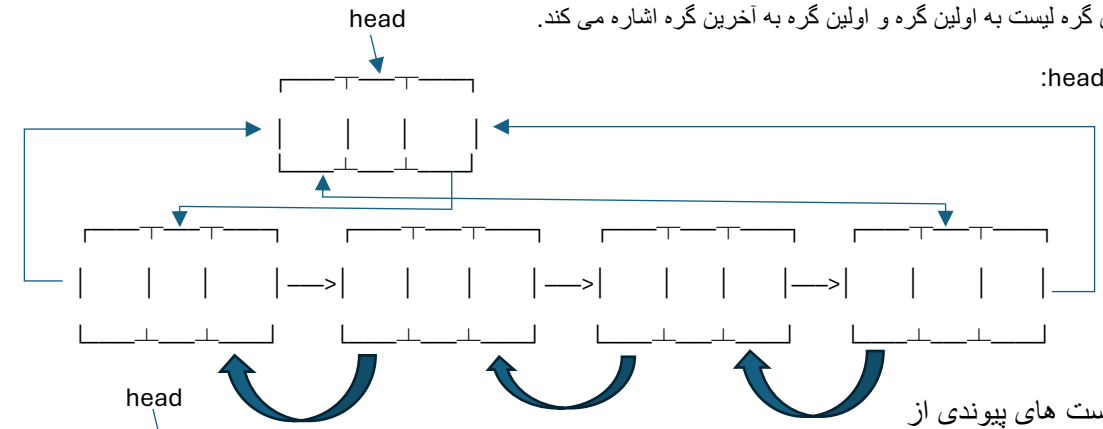
تمرین: توابع اضافه کردن و حذف گره را برای لیست پیوندی چرخشی را بازنویسی کنید.

لیست دو پیوندی چرخشی:

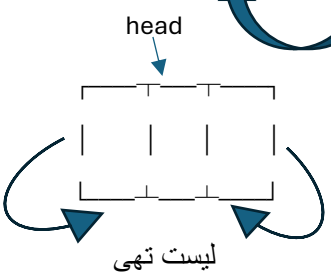


در این نوع لیست آخرین گره لیست به اولین گره و اولین گره به آخرین گره اشاره می کند.

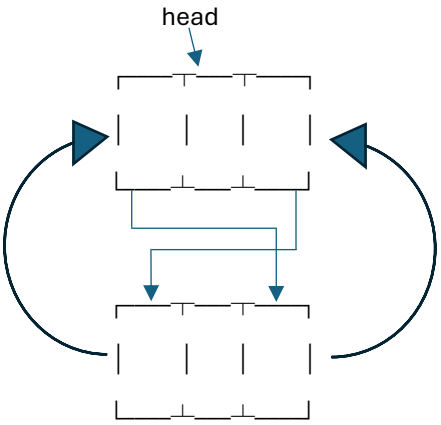
لیست دو پیوندی با گره head:



به منظور ساده سازی کار با لیست های پیوندی از این اسختار استفاده می کنیم در این ساختار یک گره head وجود دارد ساختار این گره مشابه گره های معمولی می باشد قسمت next آخرین گره و perior اولین گره به گره head اشاره میکند.

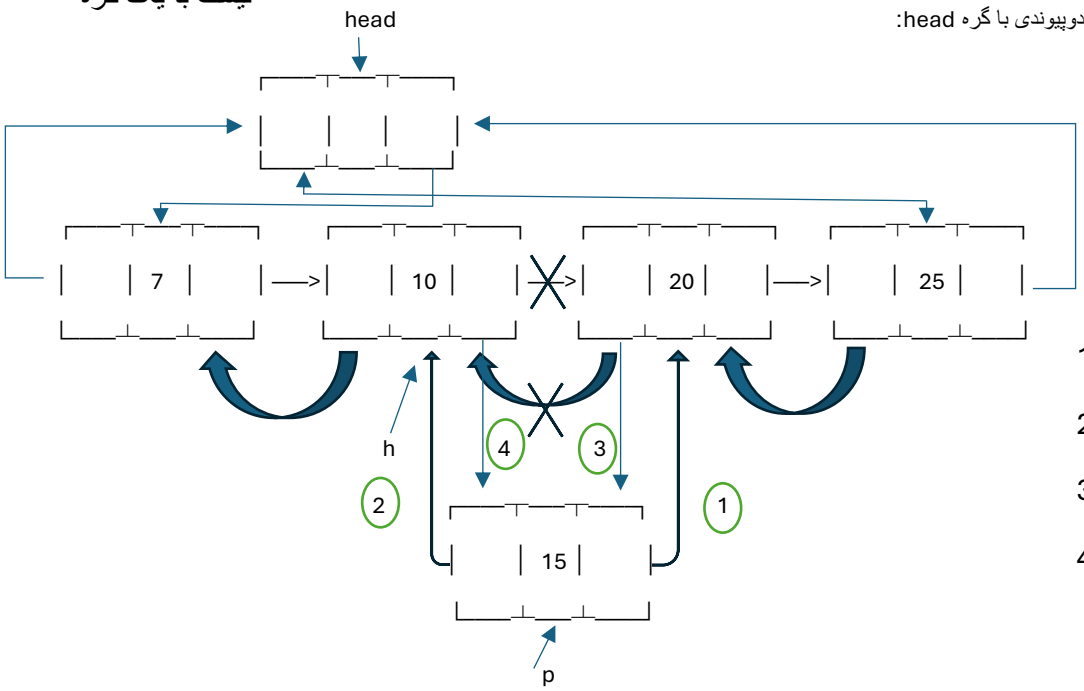


قسمت next گره headبه اولین گره و قسمت perior گره head به آخرین گره
لیست اشاره می نماید



لیست با یک گره

اضافه کردن گره به لیست دوپیوندی با گره head:



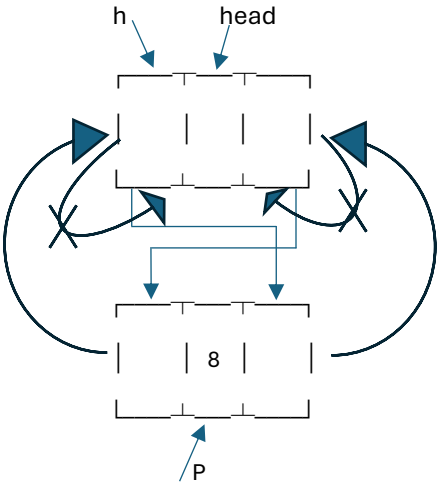
1 p->next=h -> next;

2 p->perior = h;

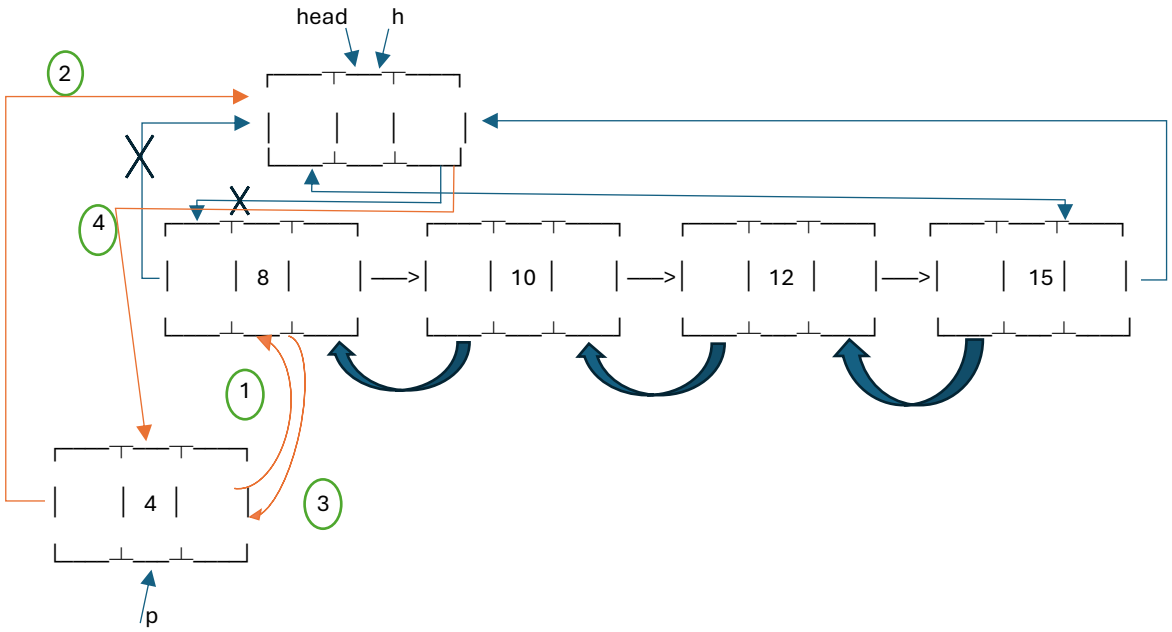
3 h->next->perior=p;

4 h->next=p;

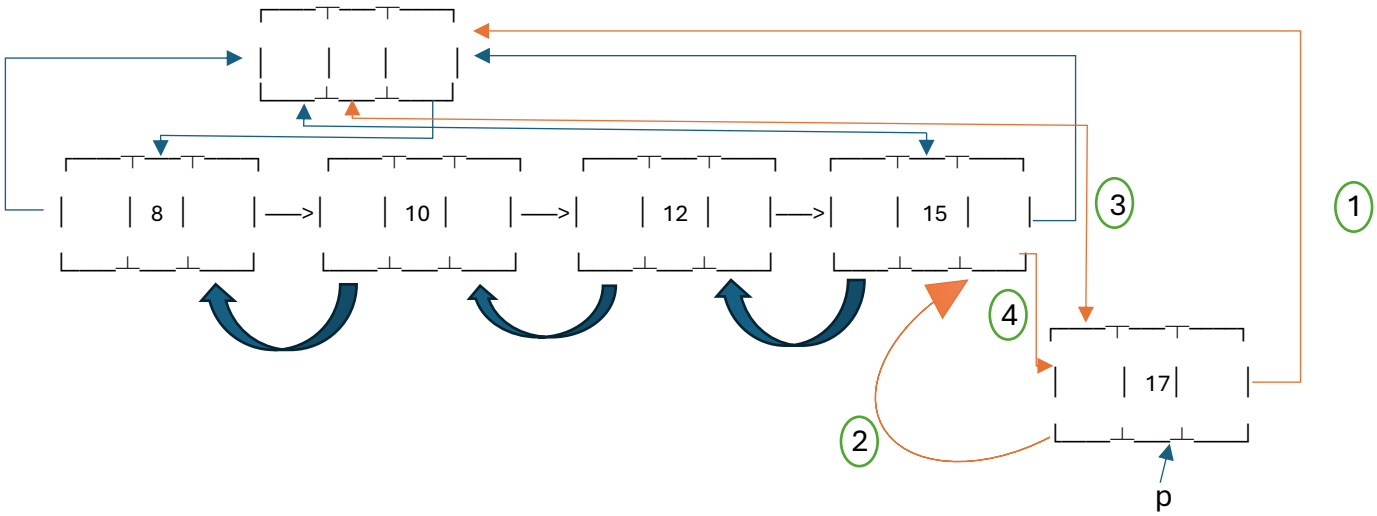
۱ اولین گره:



۲ اضافه کردن گره به ابتدای لیست:

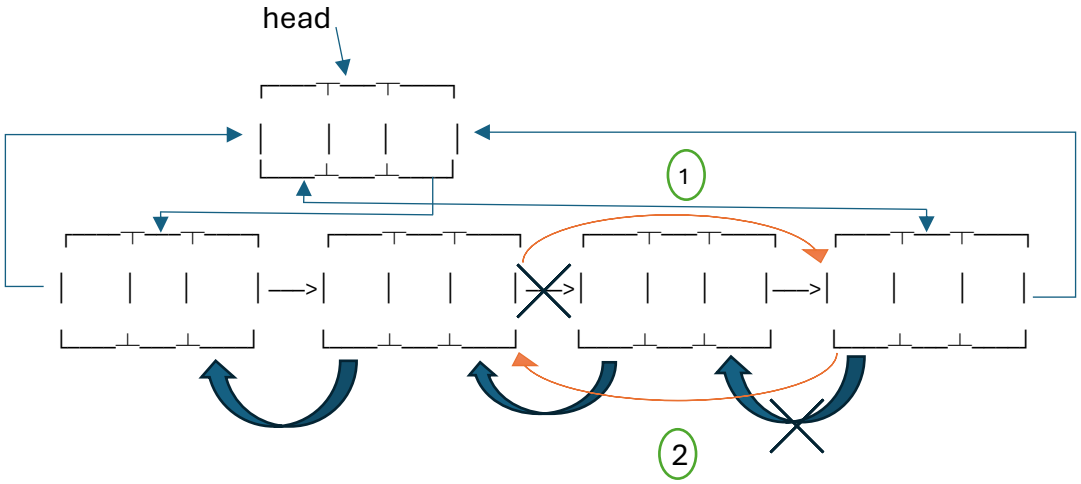


۳ اضافه کردن گره به انتهای لیست:



*ملاحظه می شود که در سهم حالات اجرای چهار دستور فوق گره را به شکل مناسب به لیست اضافه می کند و تنها کافی است که h به گره قبلی اشاره نماید.

تمرین: تابعی برای اضافه کردن گره به لیست پیوندی با گره head بنویسید؟



1

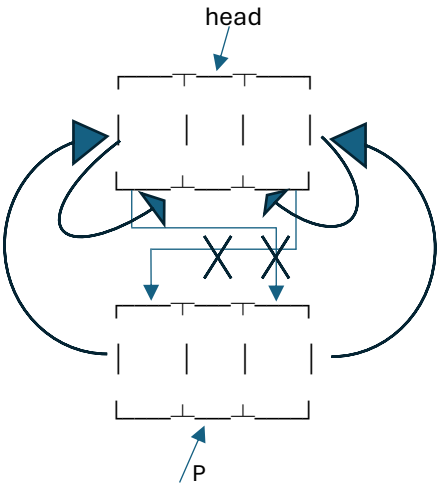
p->perior->next=p->next;

2

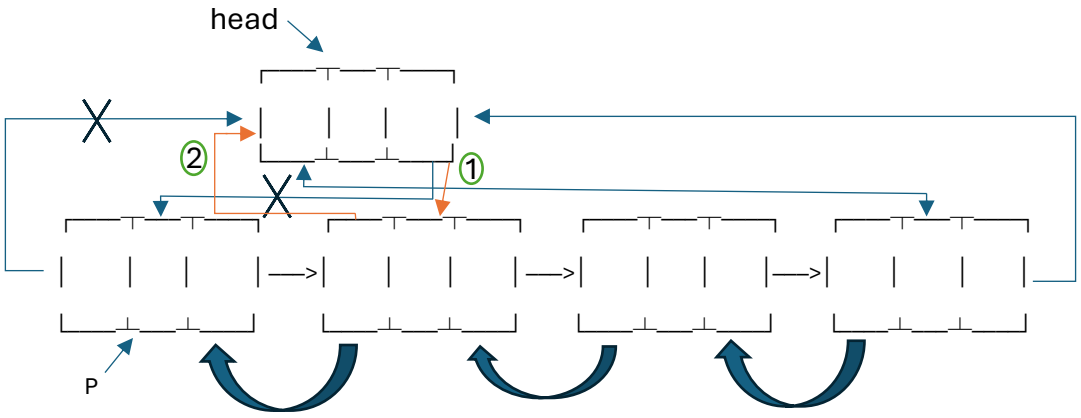
p->next->perior=p->perior

delete p;

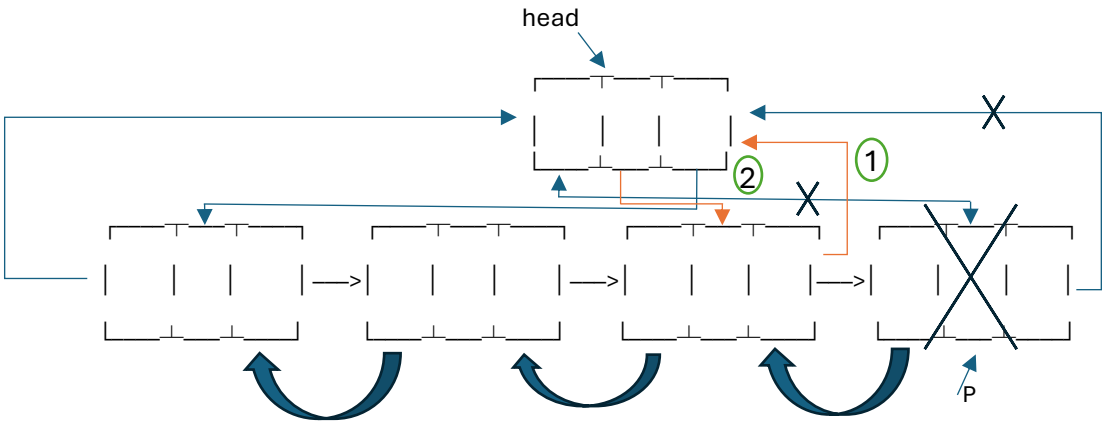
۱ تنها گره لیست باشد



۲ اولین گره لیست باشد



۳ اخرین گره لیست باشد.



ملاحظه می شود که در ۲۸ حالت فوق، اجرای ۳ دستور فوق گره را بطور صحیح از لیست حذف میکند.

تمرین: تابعی بنویسید که آدرس گره head و یک عدد را به عنوان پارامتر دریافت کند و گره مربوط به آن عدد را در صورت وجود حذف نماید.

کاربردهای لیست پیوندی:

۱ در نمایش stack

۲ در نمایش صف queue

۳ در نمایش چند جمله ای

۴ در نمایش ماتریس اسپارس

۵ در مدیریت حافظه

کاربرد لیست پیوندی در پیاده سازی چند جمله ای ها:

در انی مورد میتوانیم از لیست یک پیوندی و یا دو پیوندی استفاده کنیم، هر جمله از چند جمله ای در یک گره از لیست ذخیره میگردد، ساختار گرهای لیست به صورت زیر می باشد

```
Struct poly
{
        Poly *perior;
توان ◄── Int expon;
ضریب ◄── Float coef;
        Poly *next;
};
```

کاربرد لیست پیوندی در پیاده سازی ماتریس اسپارس:

روش اول: در این روش از یک لیست دو پیوندی با ساختار زیر استفاده می کنیم در هرگره از لیست یک عنصر غیر "0" ماتریس را قرار میدهیم.

```
Struct sparse
{
        Sparse * perior;
        float value;
```

```
    int row;
    int col;
    sparse *next;
};
```

روش دوم پیاده سازی ماتریس اسپاری:

$$\begin{bmatrix} 0 & 0 & 11 & 0 \\ 12 & 0 & 0 & 4 \\ 0 & -4 & 0 & 6 \\ 0 & 7 & 8 & 0 \end{bmatrix}$$

با مثال روبه رو بررسی می کنیم

را گره های هد سر و ستون با ساختار

down	row	col	right
value			

down	right
next	

۲ گره های داخل: که برای دخیره کردن عناصر غیر صفر ماتریس استفاده می شود ساختار:

*برای این ساختار یک گره هد اصلی در سطر می گیریم. که مشابه گره های داخل می باشد و برای هر سطر یا ستون یک گره هد در نظر میگیریم.

*next هر گره سطر به گره هد سطر بعدی اشاره می کند next هر گره ستون به گره head ستون بعدی اشاره می کند

*next گره head آخرین سطر به گره head اصلی اشاره می نماید، next گره هد آخرین ستون به گره هد اصلی اشراه می کند

*right هر گره هد سطر به اولین عنصر غیر صفر در آن اشاره میکند Down هر گره هد ستون به اولین هد اصلی اشاره می کند

*right هر گره داخلی به عنصر غیر صفر بعدی در همان سطر اشاره می کند right آخرین گره در هر سطر به گره هد آن سر اشاره میکند Down هر گره داخلی به عنصر غیر صفر بعدی در آن ستون اشاره می کند.

```
#define MAX-Size 50  /*Size of Largest MATRIX*/
Typedef ENUM{head,entry} tagfield;
Typedef struct MAXTRIX_node *MATRIX-Pointer;
Typedef Ehtry-node
{
    Int row;
    Int Col;
    Int value;
};
Typedef Struct MATRIX_node
{
MATRix-pointer down;
MATRix-pointer right;
Tag field Tag;
Union
```

{

MATRIX-Pointer-next;

Entery -node entry;

}U;

};

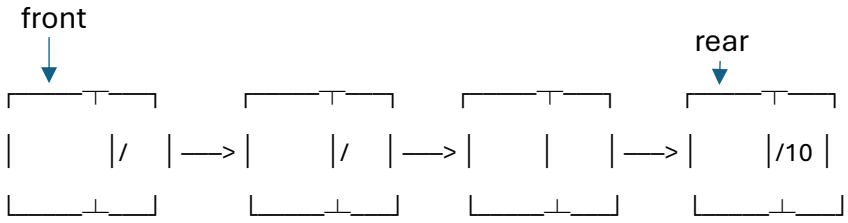
*در ساختار قبلی دو نوع گره وجود دارد هدف این است که از یک struct برای ذخیره سازی هر دو گره استفاده کنیم در دستور دوم برنامه صفحهقبل یک نوع داده شمارشی بنام Tagfield معرفی شده است در متغییر های از این نوع میتوان مقادیر و یا entery را ذخیره کرد، در دستور شماره ۳: نوع داده ای بنام MATRIX-pointer تعریف شده است که در حقیقت اشاره گر به ساختمان MATRIX-node می باشد.

ساختار MATRIX-node برای ذخیر کردن گره های هد، گره های داخل استفاده می شود ساختمان اصلی ساختمان MATRIX-node می باشد.

در هر دو اشاره گرهای down و right وجود دارد جزء Tag می توانی مقادیر head و entery را داشته باشد در این جزء برای گره های head مقدار head و برای گره های داخلی مقدار Entry را قرار دهیم.

در این ساختار از یک Unionاستفاده شده است در گره های head از بخشnext این Union در گره های داخلی از بخش Entry آن استفاده میشود.

*کاربرد لیست پیوندی در پیاده سازی صف:



در این مورد از یک لیست تک پیوندی استفاده می کنیم اشاره گر front به اولین گره و rear به آخرین گره اشاره می کند گره های ریر همیشه به آخر لیست اضافه می شوند و نه فقط از ابتدا آن.

صورت میگرد ساختار گرها بصورت روبرو می باشد

Struct Queue

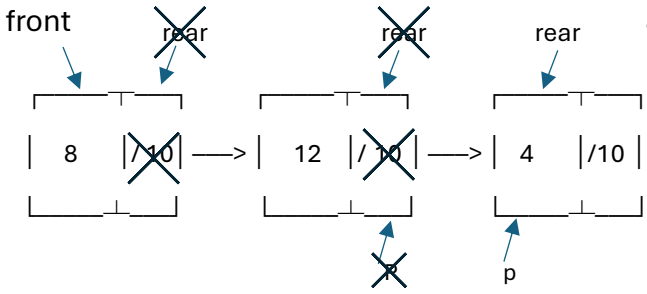
{

int data;

Queue *next;

}

مثال توابع Addq() و delete() را برای صف با لیست پیوندی بسازید؟



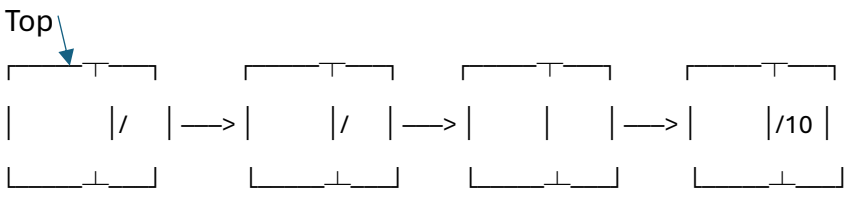
Void Addq(int x) تابع

{

Queue *p;

```
P=(Queue *)moalloc
(sizeof(Queue));
p->data=x;
p->data=x;
p->next=NULL;
if(!front){
front =p;
rear = p;
}
else{
rear ->next=p;
rear = p;
}
Int deletQ()
{
Queue *p;
int x;
if(!front)// اگرNULL باشد یعنی صف خالی است
return -1;
x=front->data;
p=front;
front=front->next;
free(p);
return x;
}
}
```

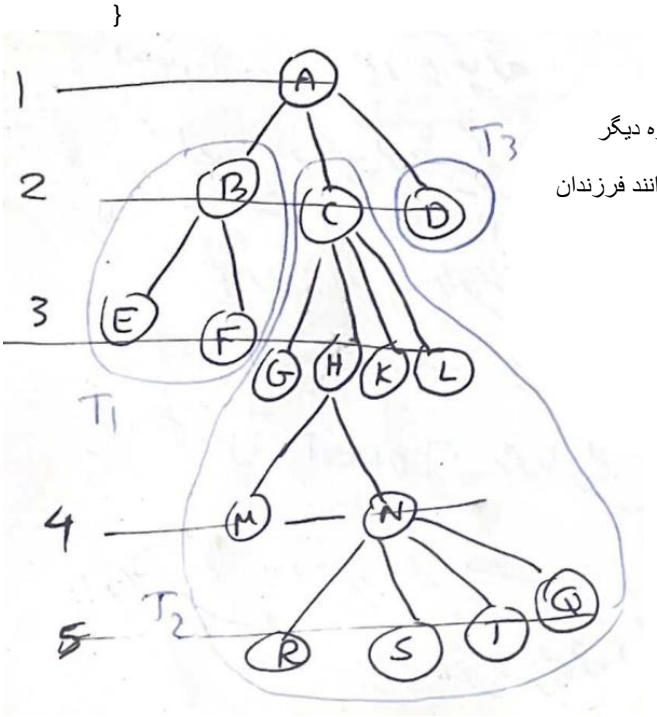
*کاربد لیست پیوندی در پیداه سازی پشته:



بر این منظور از یک لیست پیوندی استفاده می کنیم اشاره گر Topبه اولین گره لیست اشاره می کند حذف و درج عناصر همیشه از ابتدای لیست انجام میشود.

مثال توابع ()push و ()pop را برای پشته ای که توسط لیست پیوندی پیاده سازی شده است بنویسید؟

```
Void push (int x)
{
Stack *p;
P=(stack *)malloc(size of(stack));
p->data = x;
p->data= x;
p->next = Top;
Top= pi
}
Int pop(){
        Int x
        Stack *p;
        If(!Top)
                Return -1;
        x=Top->data;
        free(p);
        return x;
}
```



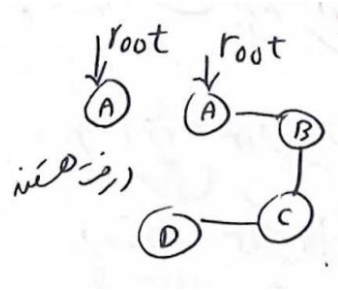
درختها: درخت ساختمان داده ای است که تشکیل یافته است از تعداد محدودی گره، به طوریکه یک گره خاص بنام ریشه وجود دارد به این گره ممکن است تعدادی گره دیگر متصل باشد که به آنها فرزندان ریشه گفته می شود هر یک زا فرزندان خود می توانند فرزندان دیگری داشته باشند.

تعریف بازگشتی درخت:

درخت به مجموعی محدودی از گره ها اطلاق میگردد به طوریکه

۱ یک گره خاص بنام ریشه وجود داشته باشد.

۲ سایر گره ها به n زیر مجموعه مجزا از هم T1...Tn تقسیم می شود بطوریکه هر یک از T1 تا Tn یک درخت می باشد T1 تا Tn را از زیر درختان گره A می گوئیم.



نکته استافده از لغت درخت، در مورد دوم مفهوم بازگشتی بودن را می رساند.

تعاریفی در مورد درخت:

۱ درخت با نام ریشه آن مشخص شود

۲ child (فرزند): به ریشه های زیر درختان یک گره گفته میشود

۳ parent (پدر): هر گره ای که مقداری فرزند داشته باشد هد در این فرزندان نامیده می شود.

۴ درجه گره: به مقدار فرزندان یک گره گفته می شود مثلا در شکل گره A درجه 3 و گره B ۲ می باشد

۵ درجه درخت: به بزرگترین درجه گره موجود در درخت گفته می شود مثلا در شکل فوق درجه درخت چهار می باشد.

"Leafnode Terminal"

۶گره همزاد: گره های هستند که پدر مشترکی دارند

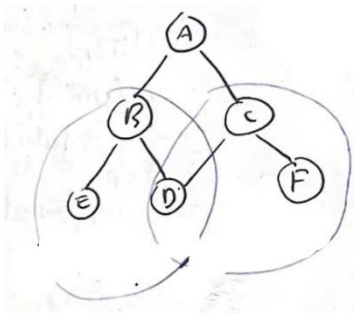
۷ بزرگ یا گره پایانی: گره ای هست که هیچ فرزندی نداشته باشد

۸ گره غیر پایانی: گره ای که حداقل یک فرزند داشته باشد nonterminal node

۹ Grand parent: پدر پدر یک گره

۱۰اجداد: به کلیه گره های که بین ریشه و یک گره خاص وجود دارند اجر و آن گره می گوییم

مثلا A,C,H اجداد M,N هستند



درخت نیست چون مجزا از همه نیستند

۱۱ سطح گره: درخت را می توان سطح بندی نمود بدین ترتیب که ریشه در سطح یک فرزندان آن در سطح ۲ و....

۱۲ عمق یا ارتفاع درخت: به بزرگترین سطح گره موجود در درخت گفته میشود.

۱۳ جنگل Forest: به مجموعه ای از درختان مجزا جنگل مجزا جنگل گفته می شود.

روشهای نمایش و دخیر سازی درخت:

۱. لیست خطی در این روش برای هر گره در داخل پرانتز، فرزندانش را مشخص می کنیم

A(B(E,f)),c(G,H(M,N(R,S,T,Q)),K,L),D)

۲ روش آرایه ای:

در این روش یک آرایه دو بعدی را در نظر می گیریم تعداد ستون های آرایه برابر است با درجه درخت بعلاوه یک در ستون اول آرایه گره ها ی درخت را سطح به سطح قرار میدهیم در ستونهای بعدی حمل فرزندان گره را در آرایه مشخص می کنیم مثال برای نمونه درخت قبلی داریم:

A	1	2	3	-
B	4	5	-	-
C	6	7	8	9
E	-	-	-	-
F	-	-	-	-
G	-	-	-	-
H	10	11	-	-
K	-	-	-	-
L	-	-	-	-
M	-	-	-	-
N	12	13	14	15
R	-	-	-	-
S	-	-	-	-
T	-	-	-	-
Q	-	-	-	-

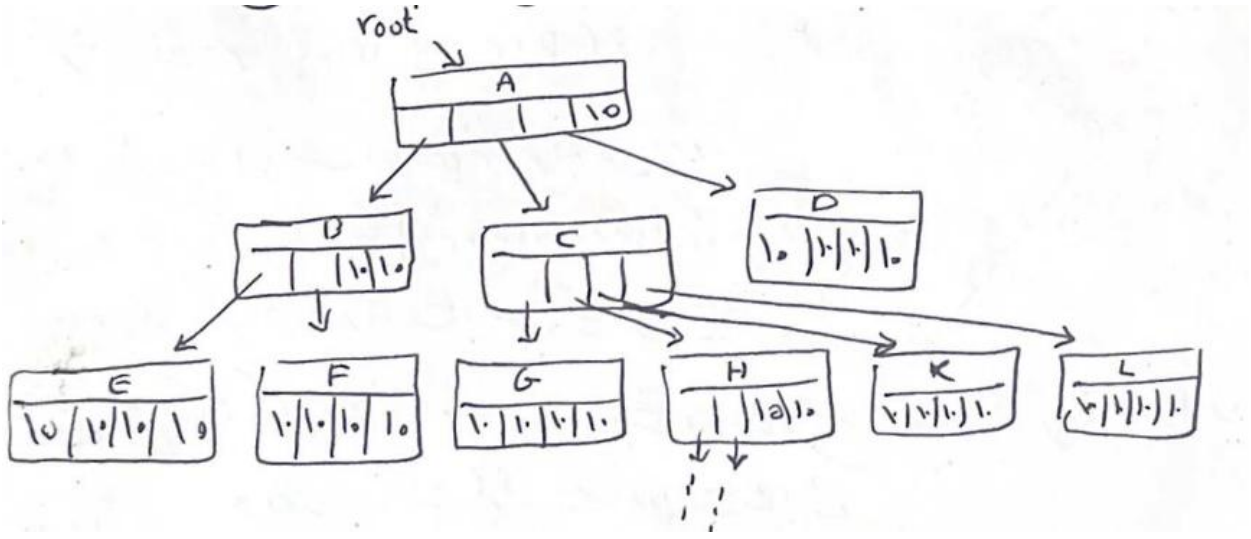
3 روش لیست پیوندی:

در این روش از یک لیست پیوندی که ساختار گره های آن بصورت زیر می باشد استفاده می کنیم

```
Struct tree{
Int data;
Tree *L1;
Tree *L2;
.
.
.
Tree *Ln;
};
```

Data			
L1	L2	.....	Ln

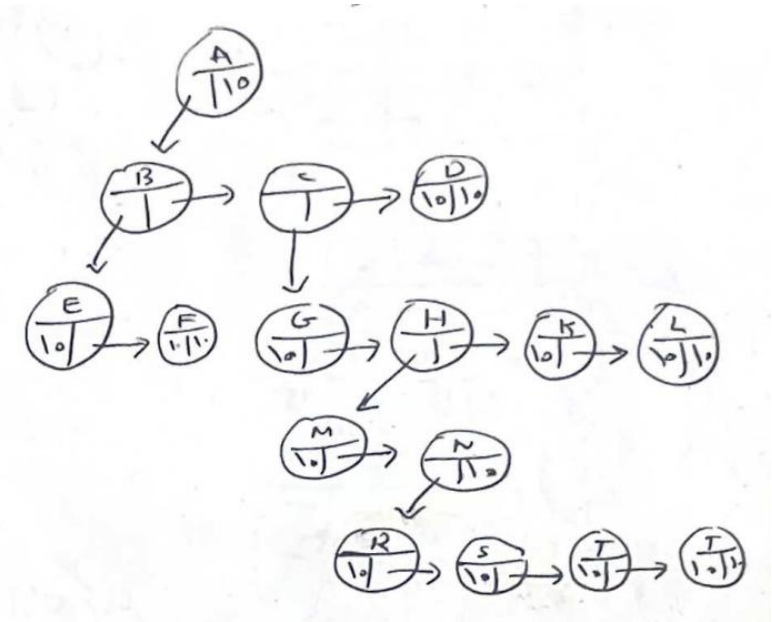
*L1 تا Ln اشاره گرهایی هستند که به فرزندان گره اشاره می کنند n درجه درخت می باشد.

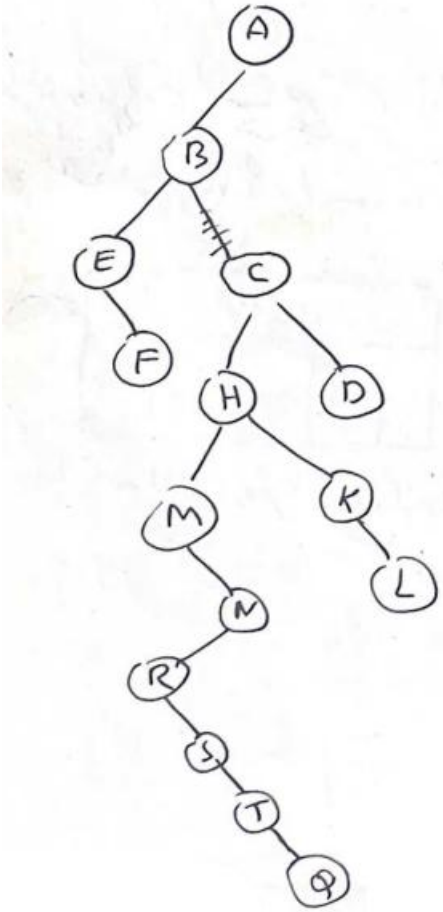


۴ فرزند چپ همزاد است: مشکل روش قبل این است که تعداد زیادی از اشاره گر ها بلا استفاده باقی می مانند همچنین در بعضی از مواقع درجه درخت از قبل مشخص نیست، برای رفع این مشکلات از این روش که یک لیست پیوندی است استفاده می شود.

*ساختار این روش بصورت زیر است:

Data	
اشاره گر به همزاد راست	اشاره گر به سمت چپ ترین فرزند





۴ نمایش دودیی درخت:

ازنمایش فرزند چپ همزاد راست بدست می آید به این ترتیب که همزاد های راست را داریم

در جهت عقربه ی ساعت دوران میدهیم.

درخت دودویی (Binary Tree):

تعریف اول: در درخت دودویی که می تواند تهی باشد یک گره خاصی بنام ریشه وجود دارد هر یک

از گره ها میتواند صفر یا یک فرزند راست وصفر یا یک فرزند چپ داشته باشد.

تعریف دوم: یک درخت دودویی تهی است و یا حاوی مجموعه محدودی از گره ها شامل یک ریشه

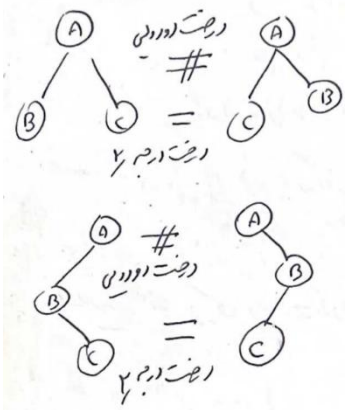
و دو زیر درخت مجزای چپ و راست می باشد

تفاوت های درخت دودویی با درخت درجه ۲:

۱ درخت دودویی و تواند تهی باشد در حالیکه درخت درجه ۲ باید لااقل یک گره داشته باشد.

۲ در درخت دودویی بین فرزندان چپ و راست تمایز قایل هستیم در حالیکه در درخت درجه ۲

فرزندان ترتیب ندارند (شکل زیر)



قضیه ۱: حداکثر تعداد گره ها در سطح i ام درخت دودویی برابر است با 2^{i-1}

مثلا:

سطح یک = ۱ و سطح ۲ =۲ و سطح ۳= ۴ و ...

اثبات) با استفاده از استقراء:

الف – به ازای m=1 درست است چون حداکثر تعداد گره ها در این سطح یک است $2^{1-1} = 2^0 = 1$

---فرض استقراء: فرض می کنیم که قضیه برای m=i-1 برقرار است یعنی

$2^{(i-1)-1} = 2^{i-2}$ = حداکثر تعداد گره ها در سطح (i-1) ام

حکم استقراء: باید ثابت کنیم در سطح (i-1) ام قرار می گیرند و هر گره دارای دو فرزند و باشد سپس:

$2^{i-1} = 2^{i-2} * 2 = 2*$ حداکثر مقدار گره ها در سطح = حداکثر تعداد گره ها

(i-1) ام در سطح i ام

ثابت شد.

قضیه ۲: حداکثر تعداد گره ها در یک درخت دودویی k سطحی برابر است با 2^k-1 می باشد.

اثبات) با استقراء:

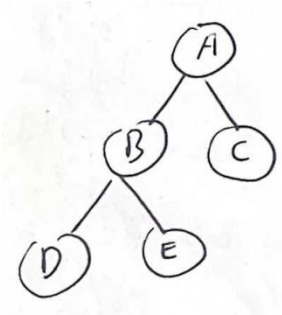
الف. قضیه برازای m=1 صحیح می باشد چون درخت یک سطحی فقط یک گره دارد و طبق فرمول هم همین را داریم $2^1 - 1 = 2-1 = 1$

ب – فرض استقراء: فرض می کنیم قضیه به ازای m=k-1 بر قرار باشد.

$2^{k-1}-1+2^{k-1}$ = حداکثر تعداد گره ها + حداکثر تعداد گره های درخت = حداکثر تعداد گره های درخت

2^k-1 = سطح kام (k-1) سطحی k سطحی

قضیه ۳: در یک درخت دودویی همیشه تعداد برگها یکی بیشتراز تعداد گره های درجه ۲ می باشد.



حکم

n0=n2+1

n = تعداد کل گره ها

n0 = تعداد گره های درجه صفر

n1= تعداد گره های درجه یک

n2= تعداد گره های درجه ۲

B = تعداد اتصالات

اثبات:

$n=n_0+n_1+n_2$

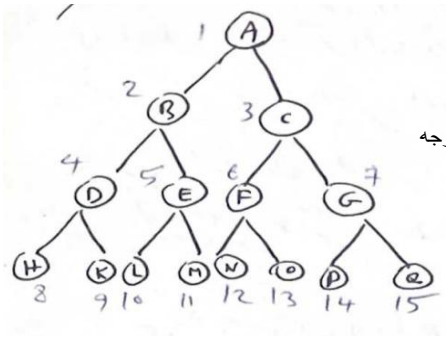
$B=2n_2$

به ازای هر گره درجه ۲ دو اتصال برای هر یک از گره های درجه یک یک اتصال داریم

کلیه گره غیر از ریشه هر کدام دارای یک اتصال n=B+1 می باشد

$n_0+n_1+n_2=2n_2+n_1+1$

$->n_0=n_2+1$



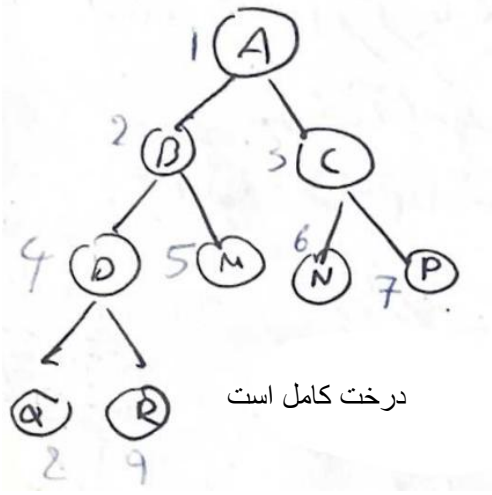
درخت پر: درختی است که حداکثر تعداد ممکن گره ها را داشته باشد

در یک درخت پر k سطحی 2^k-1 گره وجود دارد در درخت پر گره درجه

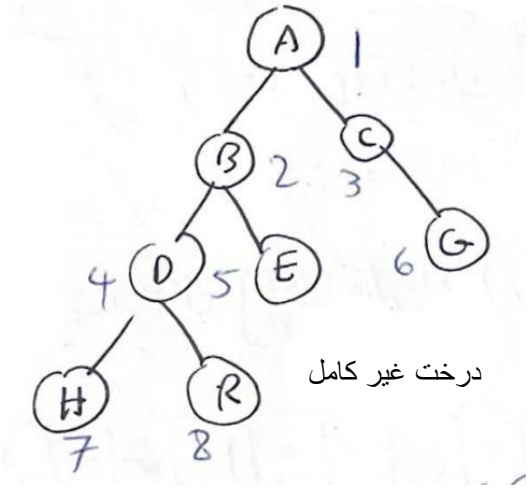
یک وجود ندارد و کلیه برگها در سطح k ام درخت قرار میگیرند.

شماره گذاری گره های درخت: گره های درخت را سطح به سطح از چپ به راست شماره گذاری می کنیم.

درخت کامل: درختی است که اگر گره های آن را شماره گذاری کنیم، این شمارها به شمارهای درخت پر همه عمق خود منطبق شود.



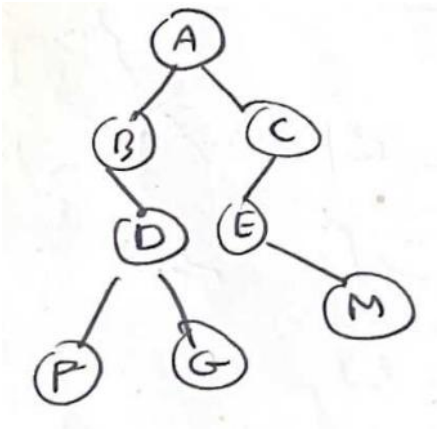
درخت کامل است



درخت غیر کامل

روش های نمایش درخت دودویی:

۱ لیست خطی



A(B(_,D(F,G),C(E(-,M),-))

روش آرایه ای

۳ روش لیست پیوندی: در این روش از یک لیست یک پیوندی که ساختار گره های آن بشکل زیر می باشد استفاده می کنیم

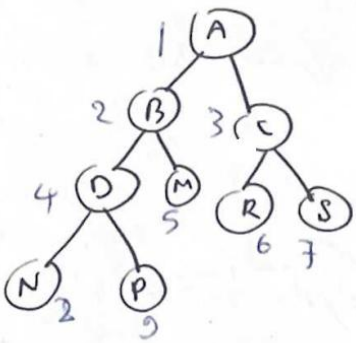
0	A	1	2
1	B	-	3
2	C	4	-
3	D	5	6
4	E	-	7
5	P	-	-
6	G	-	-
7	M	-	-

Data	
Lchild	Rchild

Lchild: اشاره گر به فرزند چپ

Rchild: اشاره گر به فرزند راست

۴ روش آرایه یک بعدی: این روش بیشتر برای درختان پر و کامل کاربرد دارد بر ای نترتیب که گره های درخت را شماره گذاری میکنیم و هر گره را در اندیس مربوط در آرایه قرار می دهیم.



A	B	C	D	M	R	N	N	P	
1	2	3	4	5	6	7	8	9	10

قضیه ۴: اگر گره های یک درخت دودویی کامل به روش بالا در یک آرایه یک بعدی ذخیره نمائیم آنگاه خواهیم داشت:

۱ فرزند چپ گره i در محل 2i قرار دارد اگر (2i<=n) n می شود تعداد گره ها

۲ فرزند راست گره i درمحل 2i+1 قرار دارد اگر (2i+1<=n)

۳ پدر گره i در محل $[\frac{i}{2}]$ قرار دارد ((>))

A	B	C	-	D	E	-	-	-	F	G	-	m
1	2	3	4	5	6	7	8	9	10	11	12	13

تمرین: اثبات قضیه ۴

نکته: با استفاده از قضیه بالا میتوان یک درخت دودویی معمولی را نیز در یک آرایه یک بعدی ذخیره کرد.

پیمایش درختان دودویی: اطلاعات ذخیره شده در گره های درخت، باید به نحوی بازیابی شوند که عمل ملاقات کردن تک تک گره های درخت پیمایش آن گفته می شود منظور از ملاقات کردن گره ها رفتن به سراغ آنها و انجام یک عمل یا پردازش روی آنها می باشد.

به طور کلی با ترکیب سه عملی L و R و V شش پیمایش مختلف را برای درخت میتوان در نظر گرفت

L: رفتن به سمت فرزند چپ

R: رفتن به سمت فرزند راست

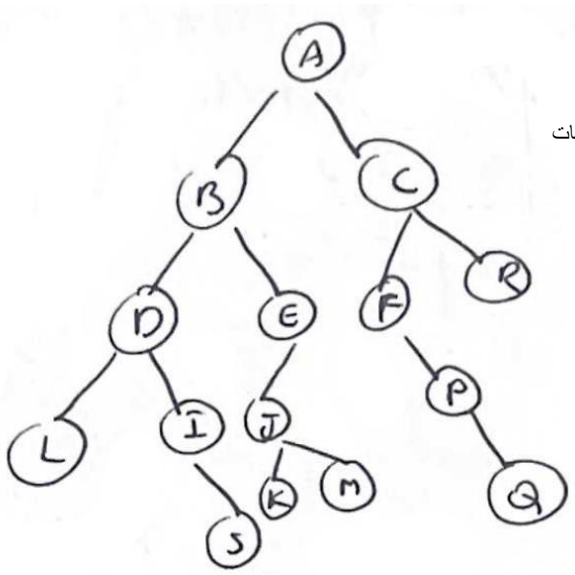
V: ملاقات کردن گره

اصلی: LVR، LRV، VLR

RVL، RLV، VRL

پیمایش LVR و LRV و VLR ، پیمایش های اصلی هستند پیمایش های که اول فرزند چپ و بعد راست پیمایش می شود

پیمایش Inorder (LVR):



در این پیمایش ابتدا فرزند جپ و سپس خود گره و در نهایت فرزند راست آن گره ملاقات

می شود به عبارت دگیر یک گره زمانی ملاقات می شود که کل زیر درخت چپ آن پیمایش

شده باشد و هنگامی سراغ فرزند راست یک گره می رویم که زر درخت چپ خود گره ملاقات

شده باشند

LVR:LDISBKJMEAFPQCR

پیمایش postorder (LRV): در این پیمایش ابتدا فرزند جپ و سپس فرزند راست و در

نهایت خود گره ملاقات می شود به عبارتی هنگامی سراغ یک گره می رویم که کل زیر

درخت چپ و کل زیر درخت راست ان پیمایش شده باشد

LRV:LSIDKMJEBQFRCA

پیامیش preorder (VLR): در این پیمایش ابتدا خود گره ، سپس زیر درخت چپ و در

در نهایت زیر درخت راست آن ملاقات می شود.

VLR:ABDLISEJKMCFPQR

*برای درخت دودویی گره هایی با ساختار رو به رو داریم:

```
Struct btree
{
    btree * Lchild;
```

اشاره گر به فرزند چپ

```
        Int data;
        btree *Rchild;
}
```

اشاره گر به فرزند راست

مثال تابعی بنویسید که آدرس یک درخت دودویی را به عنوان پارامتر دریافت کند و آنرا به روش Inorder پیمایش کنید

1.

```
Void inorder(btree *root)
{
        If(root)
        {
                Inorder(root->Lchild);
                Cout<< root->data;
                Inorder(root->Rchild)
        }
}
```

2.

```
Void postorder(btree *root)
{
        If(root)
        {
                postorder(root->Lchild);
                postorder(root->Rchild);
                cout<<root->data;
        }
}
```

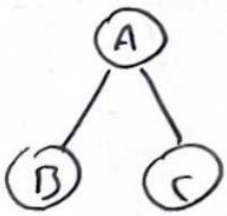
3.

```
Void preorder(btree *root)
{
        If(root)
        {
                cout<<root->data;
                preorder(root->Lchild);
```

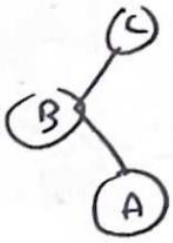
```
            preorder(root->Rchild);
        }
}
```

تابع پیمایشی درخت بصورت غیر بازگشتی(Inorder)

نکته: با داشتن یک پیمایش از درخت نمی توان آنرا رسم کرد تعیین درخت های متفاوت ممکن است، پیمایش های یکسانی را داشته باشند بنابراین برای رسم یک درخت لااقل ۲ پیمایش آن مورد نیاز است.



LVR: ABC



LVR: BAC

تمرین: بدست آورید که چگونه با داشتن پیمایش Inorder و یکی دیگر از پیمایش های درخت، می توان درخت را رسم نمود

الگوریتم inorder بصورت غیر بازگشتی:

۱ ریشه را در متغیر Currоot قرار دهید

*۲ از Currоot شروع کرده و آنقدر به سمت چپ بروید تا به NULL برسید و در بین را کلیه گره ها را در پشته قرار دهید.

۳ یک داده از پشته PoP کنید و آنرا در Currоot قرار دهید چنانچه داده ای وجود نداشت عملیات خاتمه است

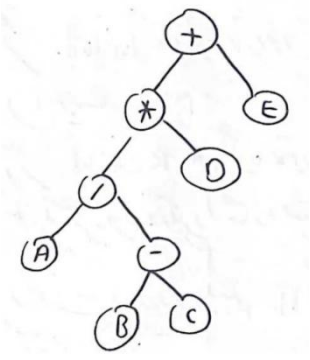
۴ currot-> data را چاپ کرده و curroot->right را داخل curroot قرار دهیم و به دستور ۲ بروید *

مثال با توجه به دو پیمایش مطرح شده درخت اولیه را مشخص کنید آیا درخت نهایی یک درخت منحصر بفرد است؟

Preorder(VLR): ABCDEFGHI

Inorder(LVR): BCAEDGHFI

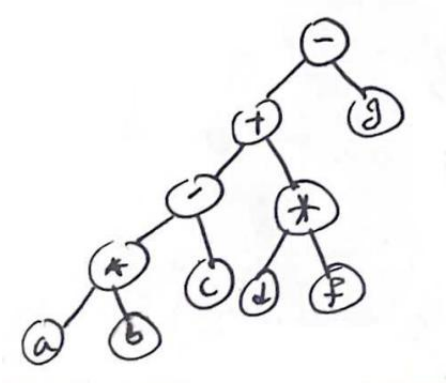
مثال: پیمایشهای درخت زیر را بنویسید



دخیره کردن یک عبارت در درخت دودویی:

یک عبارت را میتوان در درخت دودویی دخیر کرد به این ترتیب که هر عملگر را در یک گره قرار میدهیم و عملوند های عملگر را به عنوان فرزندان چپ و راست گره در نظر می گیریم.

a*b/c+d*f-g



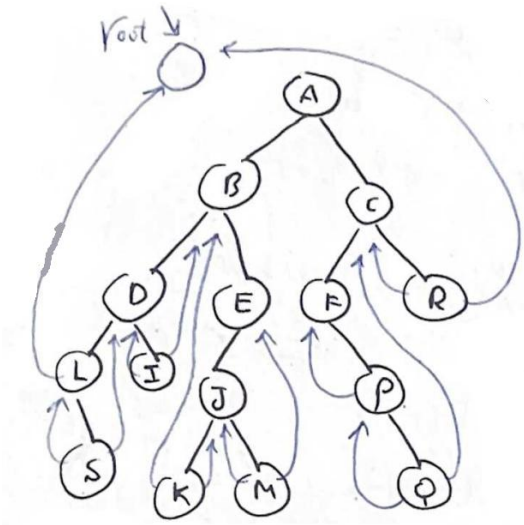
نکته اگر درخت فوق به صورت preorder پیمایش شو معادل prefix عبارت بدست می آید.

VLR:-+/*abc*dfg

*اگر بصورت inorder پیمایش شود معادل infix عبارت بدست می آید.

پیمایش سطحی درخت: در این پیمایش گره های درخت سطح به سطح از چپ به راست ملاقات می شوند

پیمایش سطحی: ABCDEFRLIJPSKMQ



درخت نخی دودویی: در یک درخت دودویی با n گره 2n اشاره گر وجود دارد در این درخت n-1 اتصال داریم بنابراین در درخت (n+1) اشاره گر تهی وجود دارد از این اشاره گرهای تهی میتوان استفاده نمود

به این ترتیب که: ۱ اگر Lchild یک گره تهی بود آنرا به گره های قبلی از این گره در پیمایش Inorder می آید اشاره می دهیم.

۲ اگر Rchild یک گره تهی بود آنرا به گره ای که در پیمایش هدخقیتق بعد از این گره قرار میگرد اشاره می دهیم.

در ساختار گره های درخت نخی دودویی روی شکل قبلی اشاره گر های تهی را اشراه میدهیم.

Heap tree: دلیل استفاده از Heap tree بخاطر مرتب سازی اقلام اطلاعاتی می باشد

۱ ایجاد درخت Heap

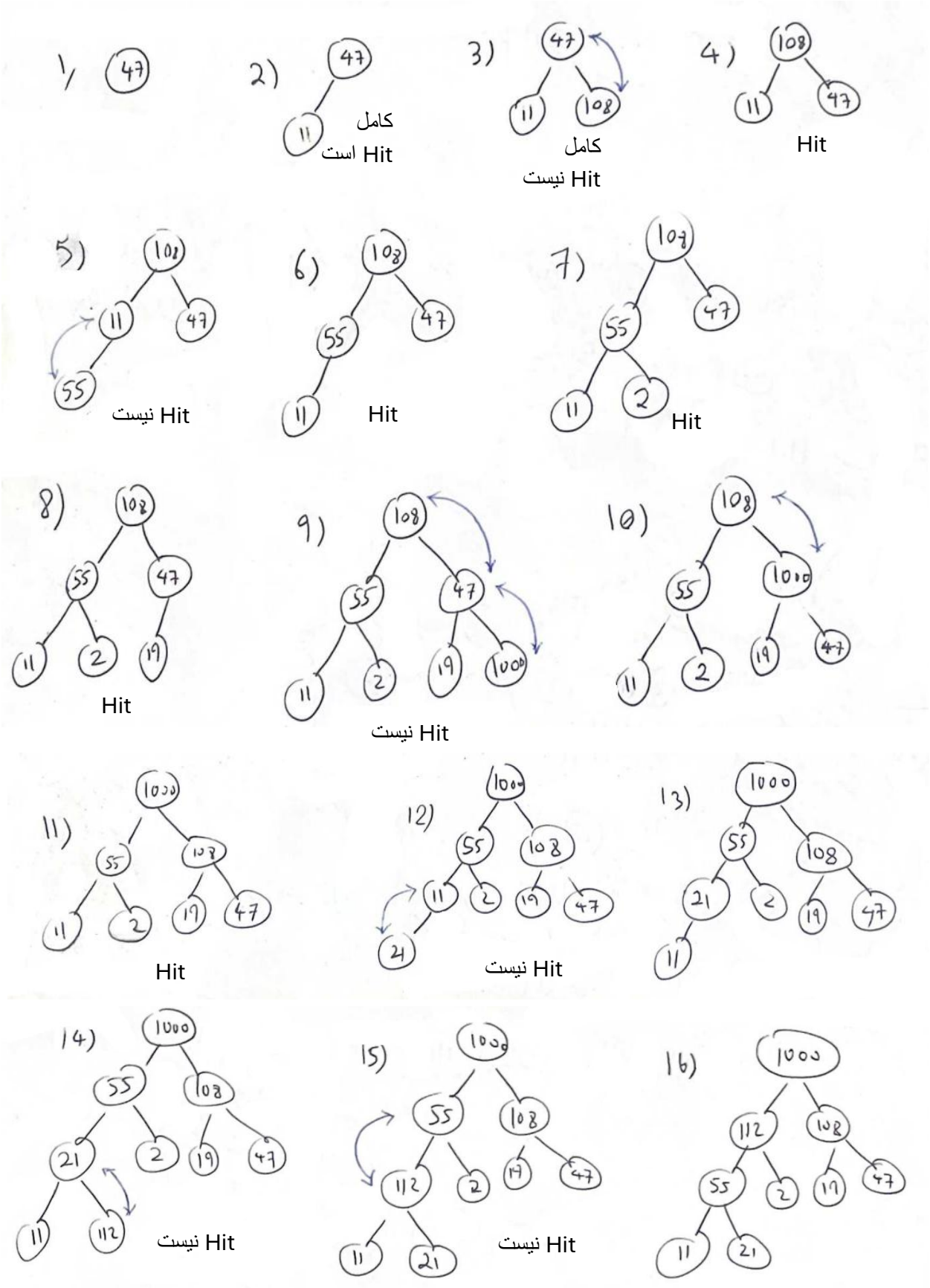
۲ حذف گره ریشه

حالت ۱ الف: کامل باشد Complete Binary Tree

ب: ریشه هر Subtree بیاد نسبت به فرزندانش بزرگتر یا کوچکتر باشد.

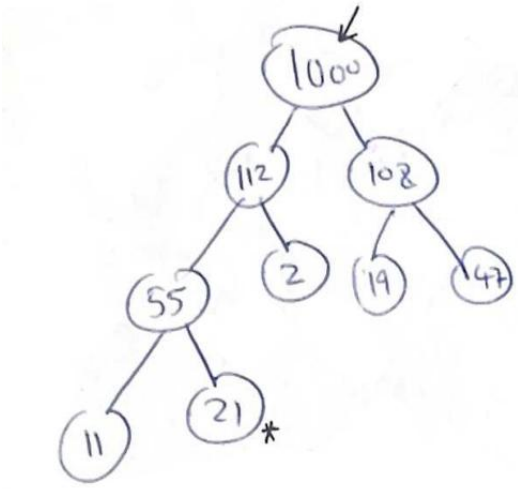
مثال داده های زیر را به صورت یک MaxHeap در آورید.

47,11,108,55,2,19,1000,21,112



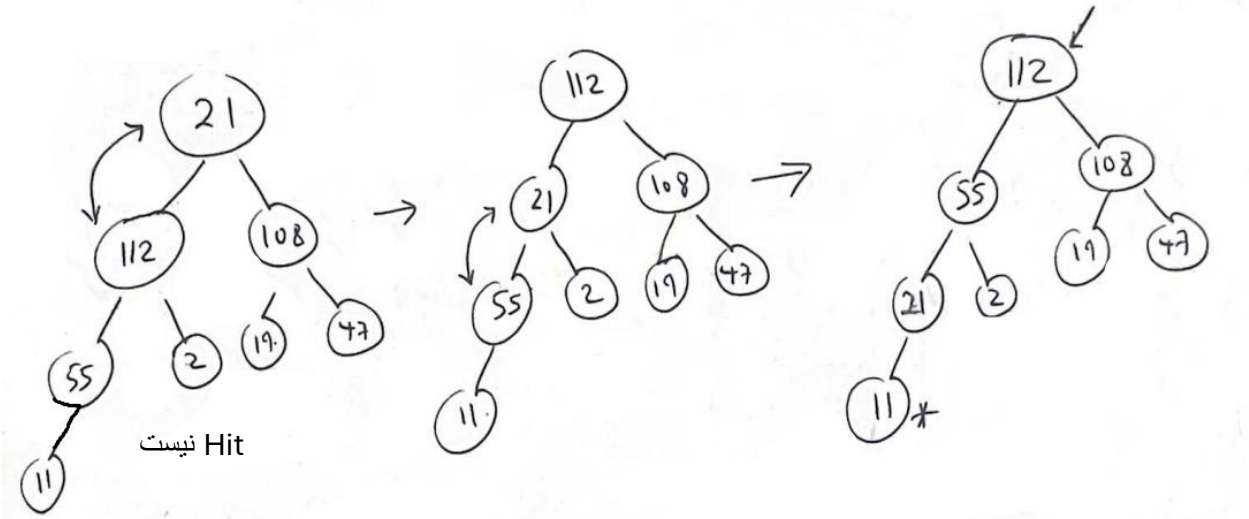
این درخت MAX می باشد ریشه (MAX Heap)است.

مثال حذف گره ریشه در درخت Heap روبه رو داریم:

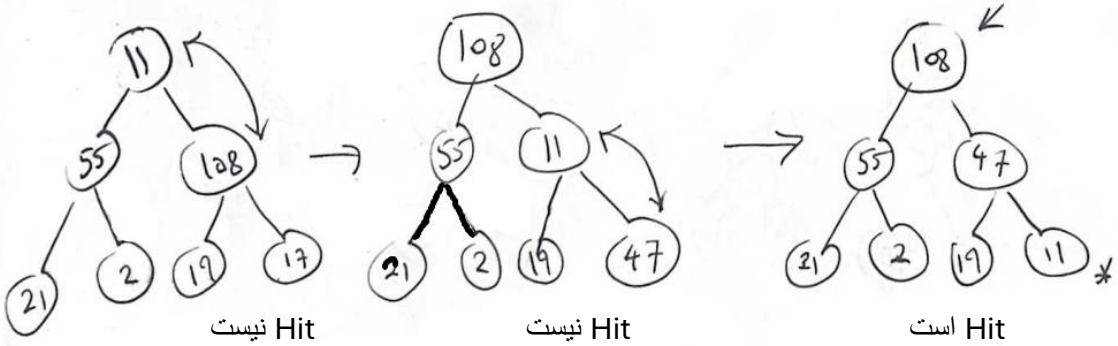


مثال حذف گره ریشه در درخت Heap رو به رو داریم:

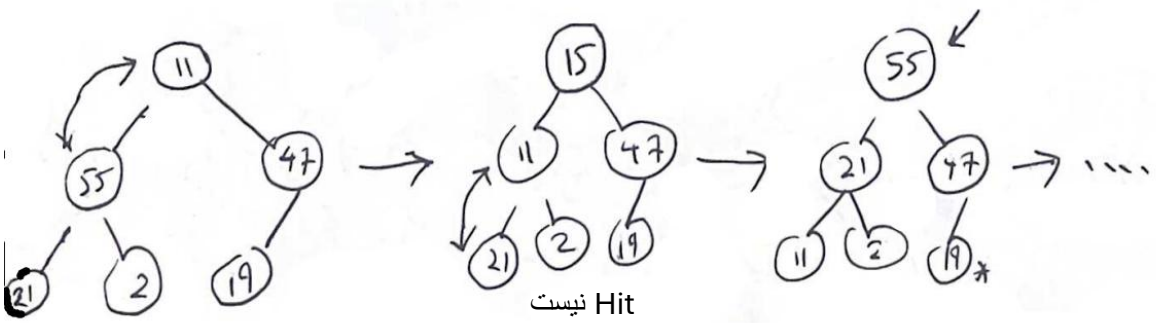
Out put=1000,112,108,55,...



Hit نیست



Hit نیست Hit نیست Hit است

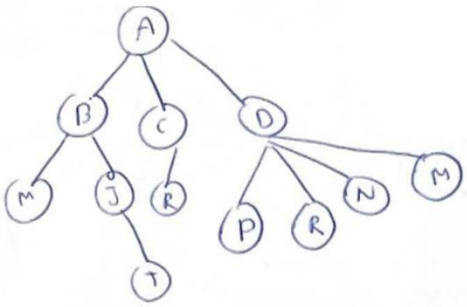


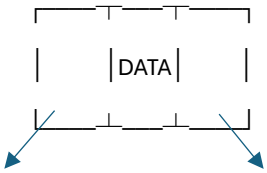
Hit نیست Hit نیست

نمایش دودویی درخت عادی:

علت آن ۱ پیمایش ساده تر درخت دودویی ۲ داشتن اتصالات تهی کمتر

مثال درخت عادی زیر را به یک درخت دودویی تبدیل نمایند.



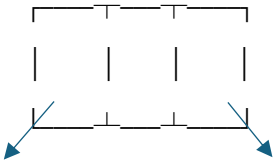


Left child

Left subtree

right child

right subtree

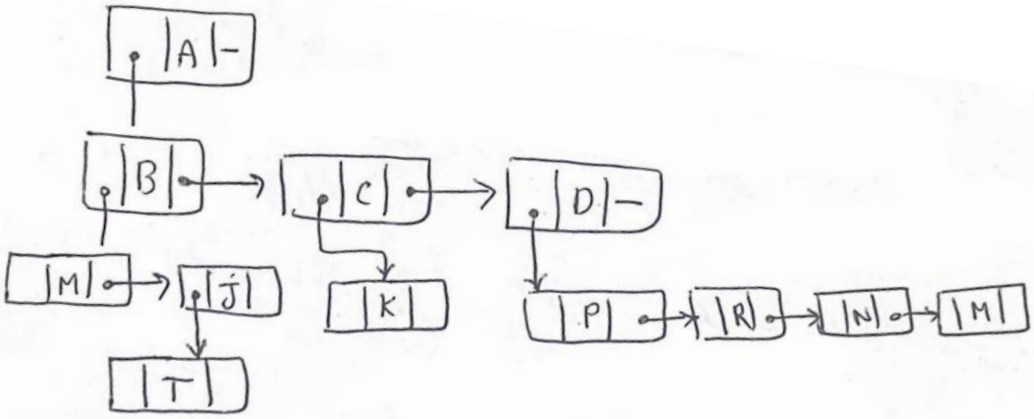


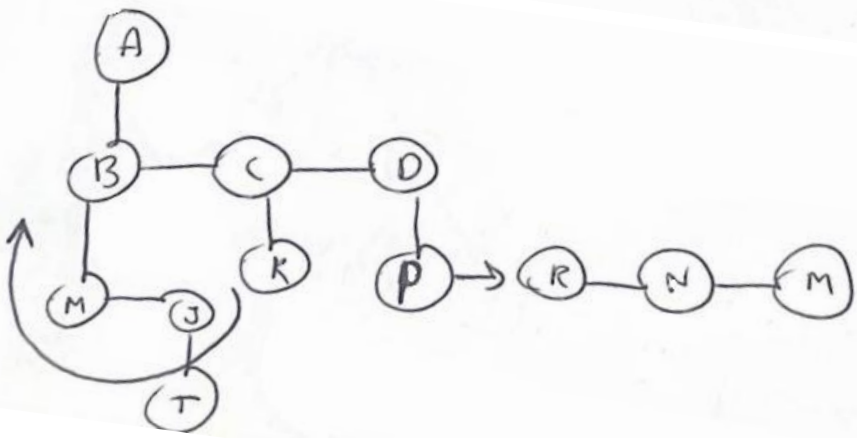
فرزند منتهی الیه سمت چپ

Left most child

همزاد راست

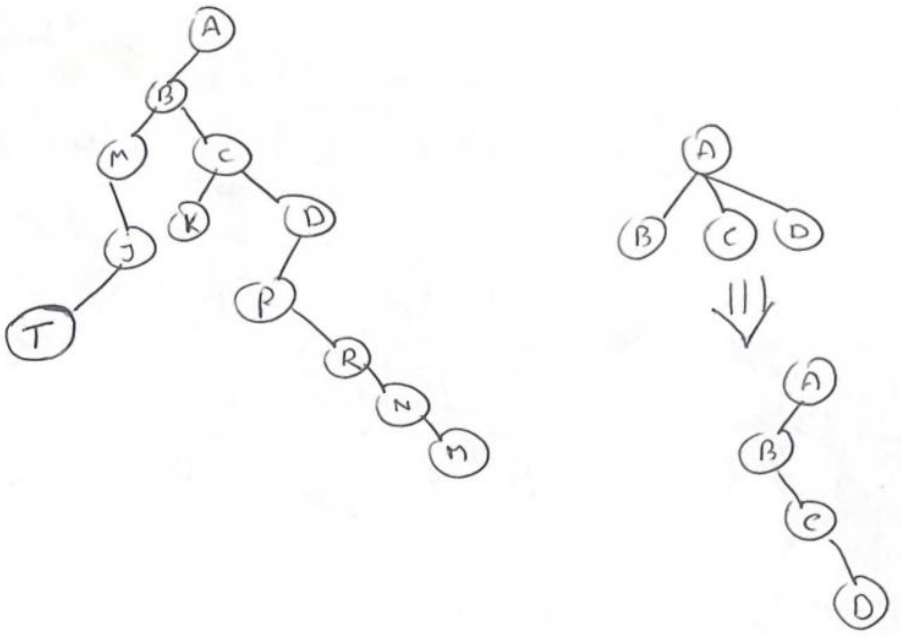
Right sibiling





نمایش دودیی مثال فوق برابر است

مثال



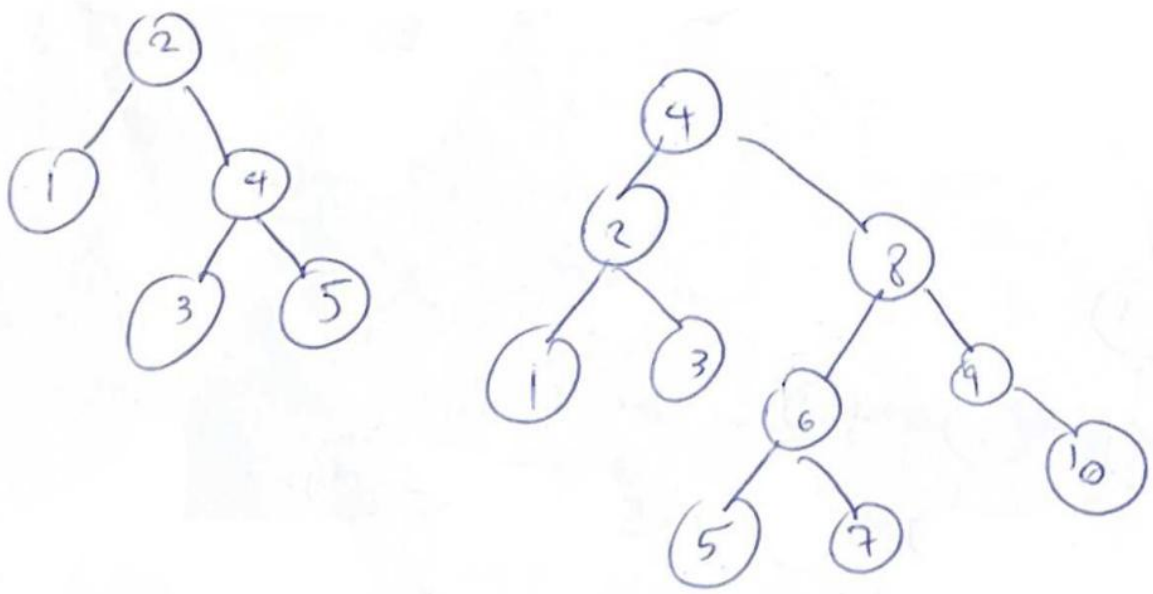
درخت متعادل BALANCEdTree (AVL):

تعریف ۱ * یک درخت دودویی که به ازای تمامی گره های آن حداکثر تفاوت تعداد گره های عمق زیر زیر درخت چپ و راست "۱" باشد

تعریف ۲ * با یک سری جابجایی درخت را حداقل به گونه ای درست میکند که عمق دو طرف یک ریشه بیش از یک واحد اختلاف نداشته باشد.

تعریف ۳ * درخت AVL درخت دودویی جستجویی است که ارتفاع آن متوازن می باشد درخت با ارتفاع متوازن یک درختی است که حداکثر اختلاف دو زیر درخت چپ و راست آن یکسان باشد حداکثر ارتفاع یک درخت AVL با n گره $[\text{Log}_2 n]$ می باشد همچنین اعمال درج و حذف یک عنصر در این درخت با پیچیدگی j(Log n) انجام می شود

*در شکل زیر دو درخت AVL نشان می دهیم در انی درخت ها ترتیب عناصر همان ترتیب برچسب های آنها است.



درخت جستوجوی دودویی:

یک درخت جستجوی دودویی میتواند تهی باشد و اگر تهی نباشد باید شرایط زیر را داشته باشد

۱ در درخت جست و جوی دودویی مقادیر تکراری وجود ندارد

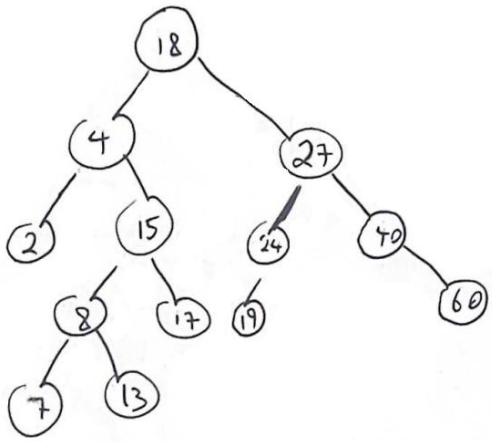
۲ تعداد هر گره از فرزند چپ آن بیشتر است

۳ تعداد هر گره از فرزند راستش کمتر می باشد.

*در مورد هر گره، تعداد گره از کل مقدار زیر درخت چپ آن بزرگتر و از کل زیر درخت راست کوچکتر می باشد.

مثال با استفاده از داده های زیر یک درخت جستچوی دوددویی بسازید

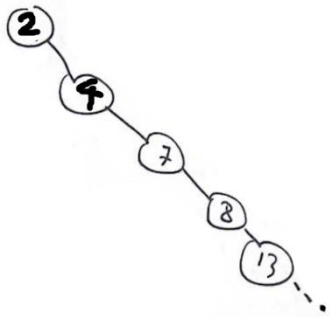
18,4,15,8,17,13,27,24,40,19,60,2,7



*شکل درخت جست و چوی دودویی وابسته به ترتیب ورودی انها می باشد

مثال یک درخت مورب که فقط فرزند راست دارد

2,4,7,8,13,15,18



*جست و جو در درخت دودویی: فرض کنیم به دنبال داده x در درخت می گردیم، x را با ریشه مقایسه می کنیم، در صورت برابری حمل داده در درخت پیدا شده است، در غیر اینصورت اگر x از ریشه کوچکتر باشد به سراغ زیر درخت چپ ریشه می رویم و اگر بزرگتر باشد به سمت زیر درخت راست می رویم این عملیات را تکرار می کنیم تا در نهایت محل داده ی مورد نظر در درخت بدست آید.

مثال تابعی بنویسید که آدرس ریشه درخت جست و چوی دودویی و عدد x را به عنوان پارامتر دریافت کند و آدرس گره ای که در آن مقدار x وجود دارد را بازگرداند.

```
Btree bsearch(btree *root, int x)

{

        If(!root)

                return NULL;

        if(root-> data == x) return root;

        if(root->data>x) return bsearch(root -> Lchild,x );

        else bsearch(root ->  Rchild,x)

}
```

اضافه کردن گره به درخت دودویی: همانطور که گفته شد در درخت جست و جوی دودویی، گره هایی با مقادیر یکسان وجود ندارد بنابراین قبل از اضافه کردن گره جدید باید مطمئن شویم گره ای با این مقدار در درخت وجود نداشته باشد. سپس حمل صحیح درچ گره در درخت را پیدا کنیم و گره را در آن محل اضافه می نمایید.

تابع اضافه کردن گره به درخت جست و جوی دودویی:

```
Void insert-node(tree-Pointer *node, int num)
{
        tree-pointer ptr, Temp = modified-Search(*node, num);
        if(temp  || !(*node)){ptr=(treePointer)malloc(sizeof(node));
                if(Ls-full(ptr))
                {
                        enit(1);
                }
                ptr->data = num;
                ptr-left-child=ptr->right-child=NULL;
                if(*node)
                        if(num<temp->data)
                                temp->left-child=ptr;
                        else
                                temp->right-child=ptr;
                else
                *node = ptr;
}//if
}//
```

Node به آدرس ریشه درخت اشاره می‌کند.

Num مقداری است که باید به درخت اضافه شود.

ptr اشاره‌گری است که آردس گره جدید را در خود نگه میدارد.

Temp آدرس گره‌ای از درخت را مشخص میکند که گره جدید باید به آن متصل شود.

تابع modified_search() آدرس ریشه درخت و مقدار num را به عنوان پارامتر دریافت میکند و آدرس گره‌ای را که گره جدید باید به آن متصل شود را بازمیگرداند، این تابع اگر گره‌ای با مقدار num درخت وجود داشته باشد، مقدار null را بازمیگرداند.

در صورتیکه temp مقدار null نداشته باشد یا درخت هیچ گره‌ای نداشته باشد می‌توانیم گره جدید را به درخت اضافه کنیم که توسط دستور malloc گره جدید تشکیل میشود و آدرس آن در ptr گذاشته میشود.

اشاره‌گرهای چپ و راست گره جدید null میشوند if(*node) به این معناست که درخت در حال حاضر لااقال یک گره در دیگر مقدار num از مقدار ذخیره شده در temp کوچکتر باشد گره جدید فرزند چپ temp میشود در غیر اینصورت فرزند راست.

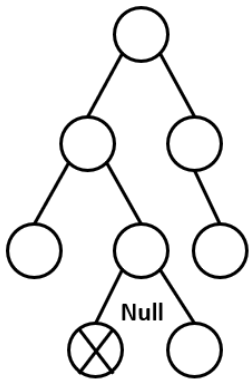
تمرین : تابع Modified_Search() را بنویسید؟

```
tree_pointer modifide_search(tree_pointer*root, int num)
{
   if (!root)
      return null;
   while (١)
   {
      if (root->data == num)
         return null;
      else if (root->data > num)
      {
```

```
        if (!root->left_child)
            return root;
        else root = root->left_child;
    }
    else
    {
        if (!root->right_child)
            return root;
        else root = root->right_child;
    }
  }
}
```

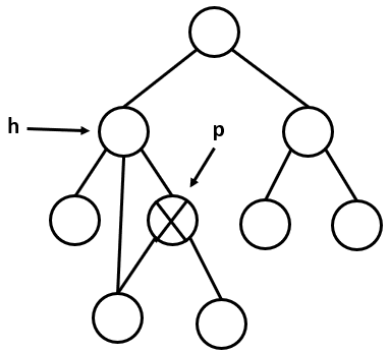
حذف گره از درخت جستجوی دودویی : هنگام حذف یک گره ممکن است یکی از ۲ حالت زیر پیش آید :

۱_ گره موردنظر برگ باشد در این صورت بسادگی گره را حذف میکنیم و در اشاره‌گر مربوطه در گره پدر مقدار null را قرار میدهیم.

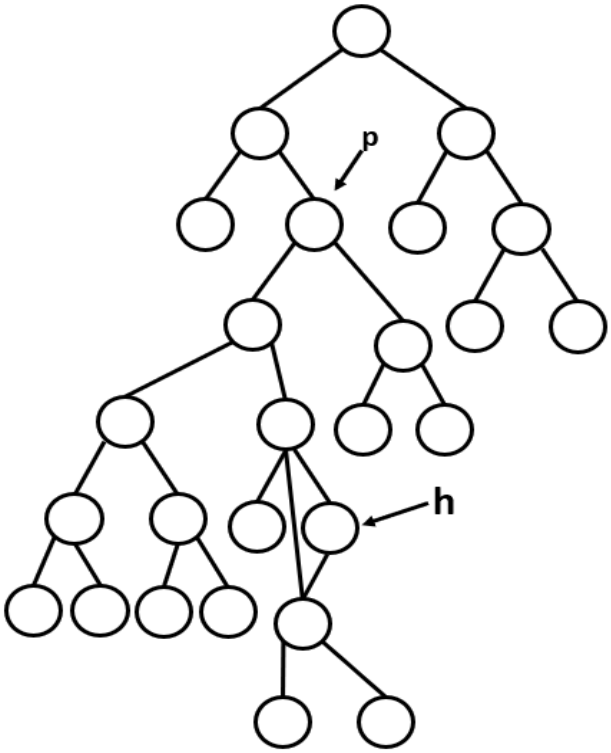


۲_ گره‌ای که میخواهیم حذف کنیم یک فرزند داشته باشد در اینصورت گره را حذف میکنیم و در گره پدر آدرس فرزند، گره‌ای را که حذف کرده‌ایم قرار میدهیم در این وضعیت چهار حالت پیش می‌آید که دو حالت آن بصورت روبه‌رو است :

```
h->rchild=p->Lchild;
h->Rchild=p->Rchild;
```



۳_ گره‌ای که میخوایم حذف کنیم دو فرزند نداشته باشد در اینصورت باید تکلیف فرزندان چپ و راست آن را مشخص کنیم برای رفع این مشکل، به یکی از دو روش زیر عمل میکنیم:

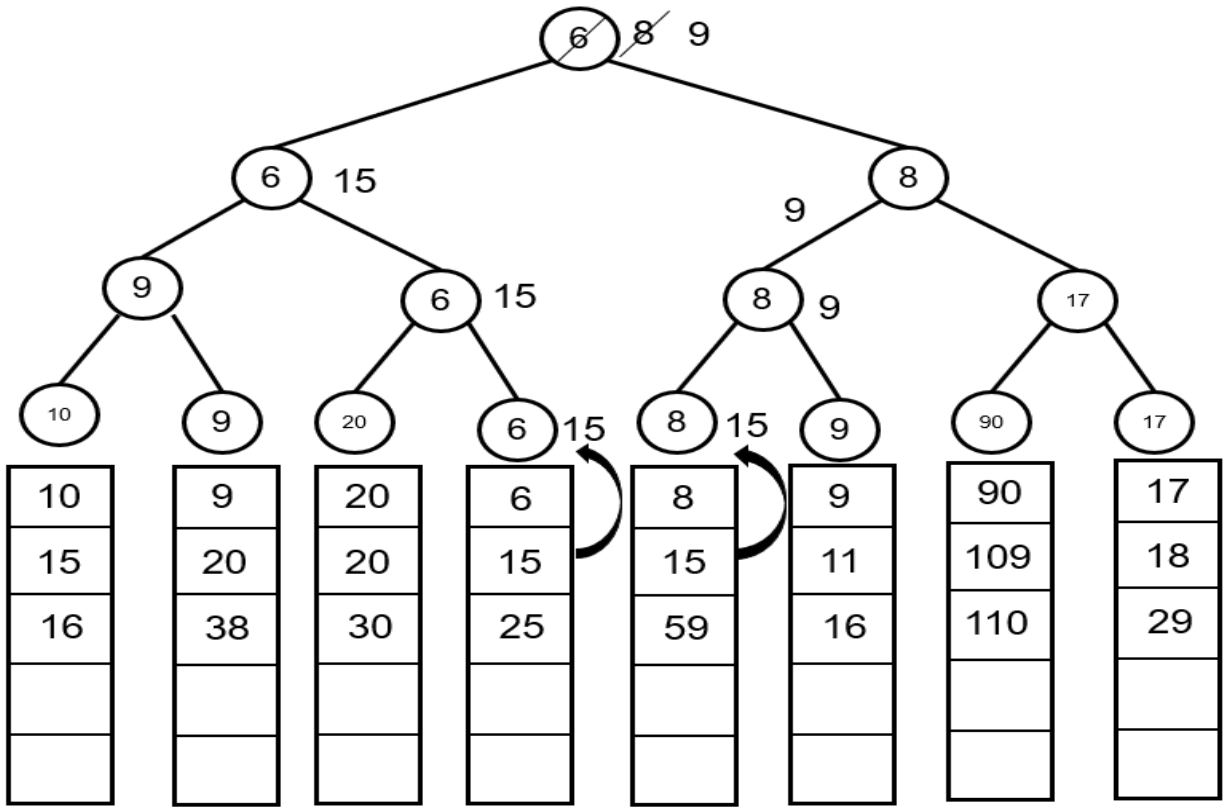


روش اول : فرض کنید در شکل روبه‌رو گره p را میخواهیم حذف کنیم از این گره یک قدم به چپ و سپس تا جایی که امکان دارد به راست می‌رویم به گره h میرسیم، این گره فرزند راست ندارد و بزرگترین مقدار در زیر درخت چپ گره p میباشد، مقدار آنرا در گره p قرار میدهیم و به جای آن گره h را حذف میکنیم.

روش دوم : از گره p یک قدم به راست و سپس تا جایی که تمکان دارد به چپ میرویم، به گره‌ای میرسیم که فرزند چپ ندارد و کوچکترین داره در زیر درخت راست گره p میباشد مقدار آن در گره p قرار میدهیم و گره جدید را حذف میکنیم.

درختان انتخابی : فرض کنید k لیست مرتب شده از اعداد داشته باشیم و میخواهیم آنها را در یکدیگر ادغام کنیم به طوریکه در نهایت یک لیست مرتب شده ایجاد گردد، در ساده‌ترین روش باید تک تک عناصر ابتدای لیست‌ها را با یکدیگر مقایسه کنیم و کوچکترین آنها را انتخاب کنیم. یعنی برای انتخاب هر عنصر (k-۱) مقایسه نیاز میباشد. حال با استفاده از درختان انتخابی میتوانیم تعداد مقایسات را کاهش دهیم.

درخت انتخابی از سمت برگها بدست ریشه ایجاد میگردد، به این ترتیب که بین هر دو برگ مجاور، داده کوچکتر را انتخاب میکنیم و به عنوان پدرشان در نظر میگیریم، در نهایت کوچکترین داده در ریشه قرار میگیرد آنرا به لیست خروجی منتقل میکنیم و سپس یک داده از لیست مربوط به درخت می‌آوریم و مجدداً مقایسات لازم را بین گره‌های درخت انجام میدهیم. در حالت قبل برای انتقال یک داده به لیست خروجی (k-۱) جستجو نیاز بود در این حالت با $[\log^{k}_{۲}]+۱$ مقایسه داده‌ای که باید به خروجی منتقل شود مشخص میگردد.



خروجی = ۶، ۸، ۹، ...

الگوریتم حذف گره از درخت باینری :

اگر q مشخص کننده پدر گرهی که باید حذف شود (پدر p)، rp مشخص‌کننده عنصری که باید جایگزین p شود، f بیانگر پدر rp و نشان‌دهنده فرزند سمت چپ rp است البته تا وقتی که این فرزند سمت چپ وجود داشته باشد.

```
void DeleteBst (treepointer *tree, data x)
{
   p = *tree;
   q = null;
   while(p && p->data != x)
   {
```

```
    q = p;
    if (p->data < x)
        p = p->rlink;
    else
        p = p->llink;
}
if (p == null)
    return;
if (p->llink == null)
    rp = p->rlink;
else
    if (p->rlink == null)
        rp = p->llink;
else
{
    f = p;
    rp = p->rlink;
    s = rp->llink;
    while (s != null)
    {
        f = rp;
        rp = s;
        s = rp->llink;
```

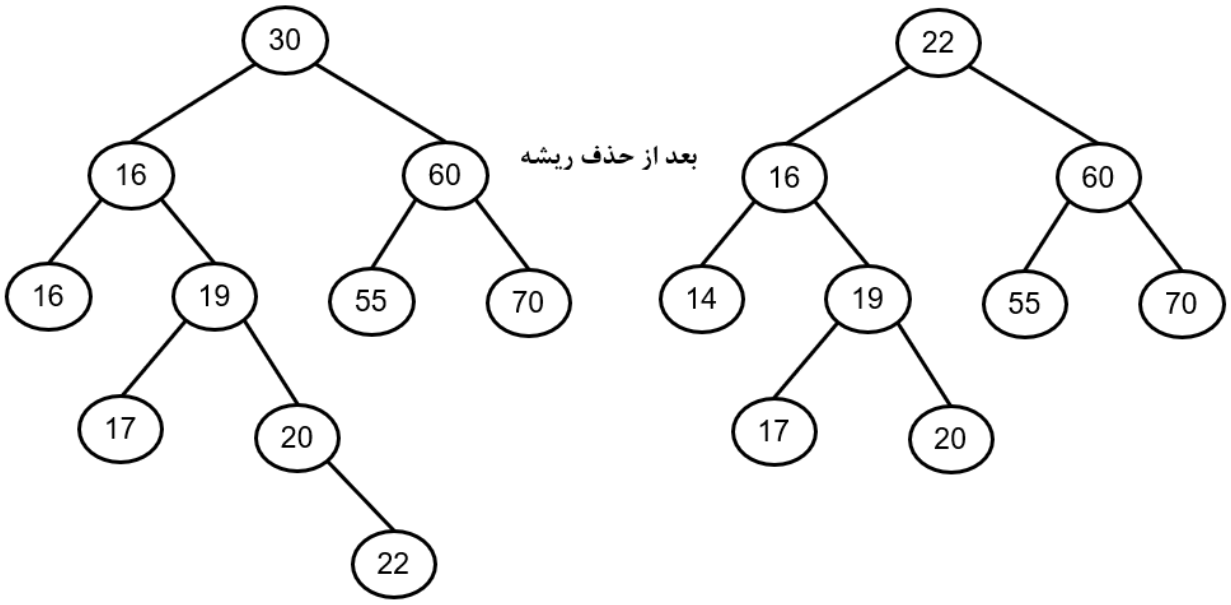
```
        }
        if (f != p)
        {
            f->llink = rp->rlink;
            rp->rlink = p->rlink;
        }
        rp->llink = p->llink;
    }
    if (q == null)
        *tree = rp;
    else
        if (p == q->llink)
            q->llink = rp;
    else
        q->rlink = rp;
}
```

حذف یک عنصر یک درخت دودویی : ابتدا باید توسط الگوریتم جستجو محل گره موردنظر را در درخت پیدا کنیم گره موردنظر ممکن است در یکی از محل‌های زیر قرار داشته باشد. در این صورت دو وضعیت پیش می‌آید:

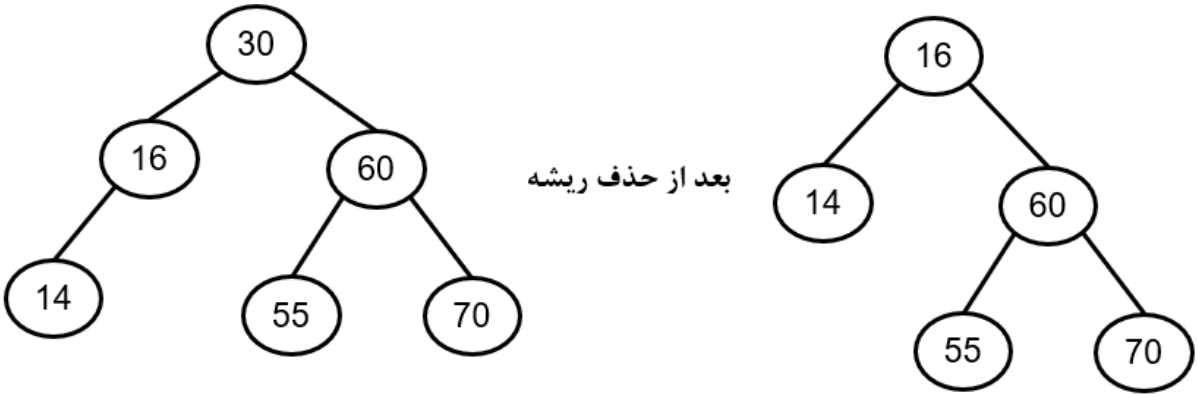
۱_ ریشه درخت باشد.

الف) درخت فقط دارای یک گره است از ریشه لذا ریشه را حذف و اشاره‌گر درخت را null میکنیم.

ب) درخت دارای زیردرخت سمت چپ ریشه فیلد راستش تهی نباشد لذا در زیر درخت سمت چپ راسترین گره را پیدا نمود و آن را به عنوان ریشه جدید در نظر میگیریم، زیرا در زیر درخت سمت چپ راسترین گره بزرگترین گره است.



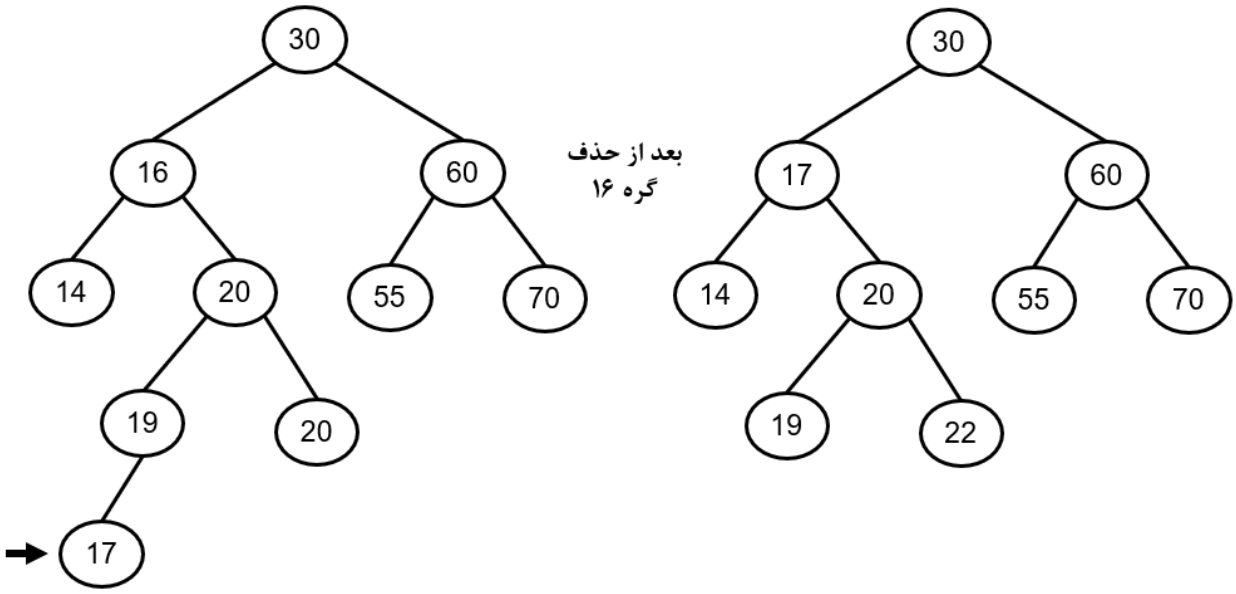
ج) درخت دارای زیر درخت سمت چپ است و اگر سمت چپ ریشه فیلد راستش تهی است در اینصورت گره سمت چپ رگره مورد نظر شه را به عنوان ریشه جدید نظر میگیریم.



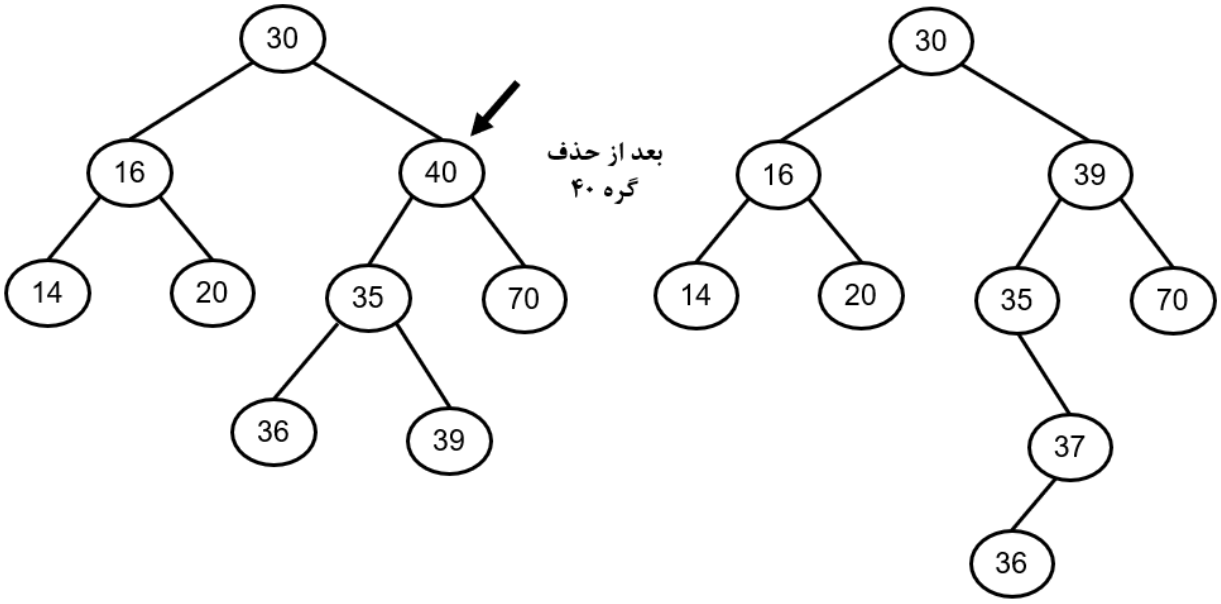
۲_ گره مورد نظر دارای دو فرزند است :

در این وضعیت دو حالت اتفاق می‌افتد:

الف) گره مورد نظر در زیر درخت چپ میباشد آنگاه سمت چپ‌ترین گره زیر درخت راست گره مورد نظر را پیدا نموده و داده آنرا به گره مورد نظر منتقل میکنیم و سپس سمت چپ‌ترین گره را حذف میکنیم.



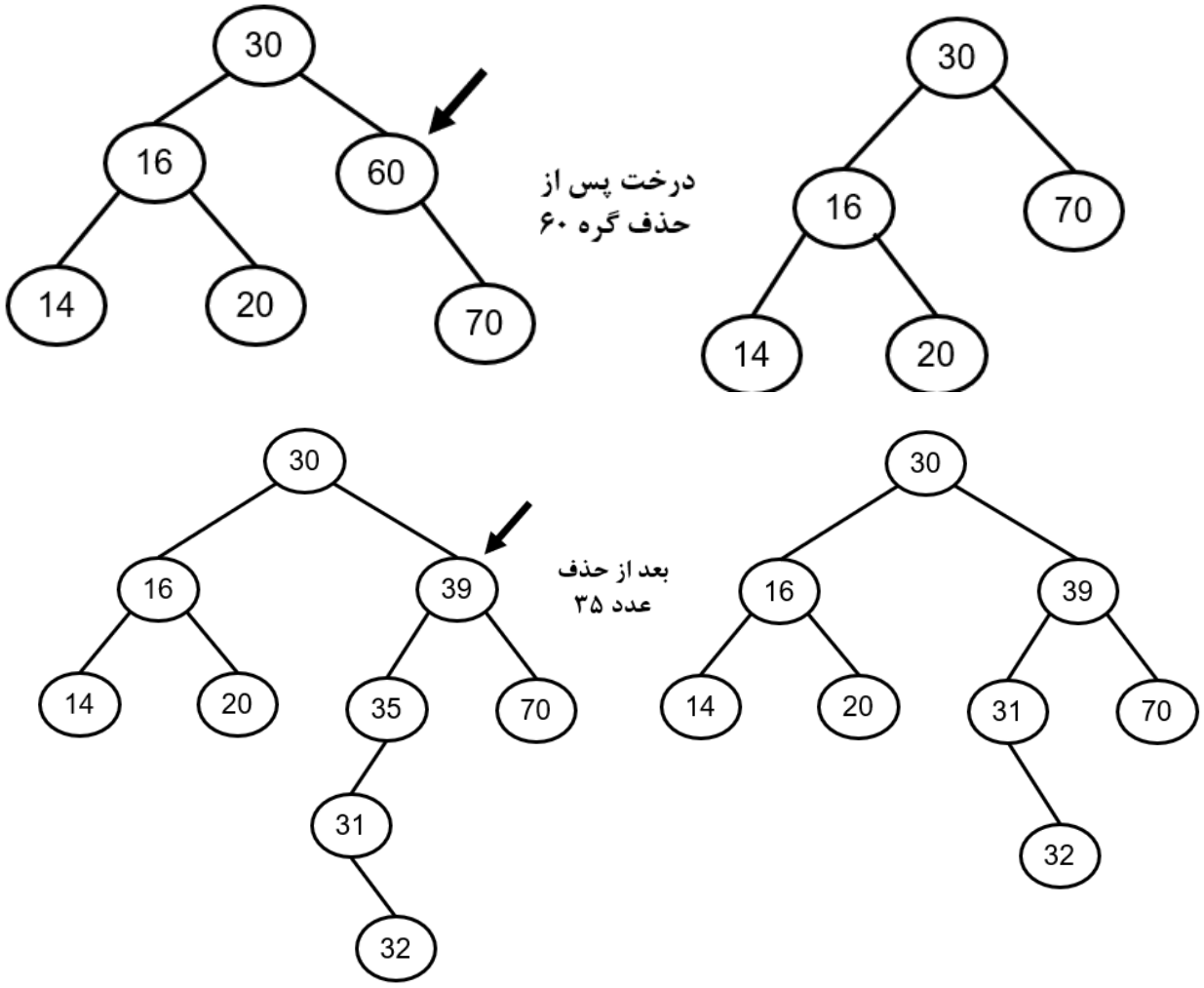
ب) گره مورد نظر در زیر درخت راست باشد آنگاه سمت راسترین گره زیر درخت سمت چپ گره مورد نظر را پیدا نموده و آنرا به گره مورد نظر منتقل میکنیم و سپس سمت راسترین گره را حذف میکنیم.



۳_ گره مورد نظر یک فرزند دارد و یا فاقد فرزند میباشد : در اینصورت دو وضعیت پیش می‌آید :

الف) گره مورد نظر در زیر درخت چپ باشد آنگاه دیگر فیلد چپ گره مورد نظر مخالف تهی باشد در فیلد چپ گره پدر فیلد چپ گره مورد نظر را قرار میدهیم در غیراینصورت فیلد راست گره مورد نظر را در فیلد چپ گره پدر قرار میدهیم.

ب) گره مورد نظر در زیر درخت راست باشد آنگاه اگر فیلد چپ گره مورد نظر مخالف تهی باشد در فیلد راست گره پدر فیلد چپ گره مورد نظر را قرار میدهیم در غیر اینصورت فیلد راست گره مورد نظر را در فیلد راست گره پدر قرار میدهیم.

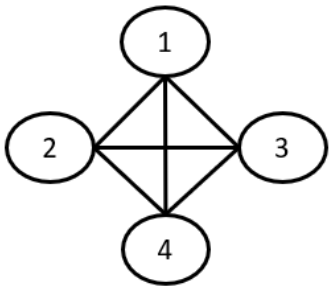


گراف (Graph) : هر گراف متشکل از دو مجموعه V و E میباشد که V یک مجموعه محدود از رئوس است و E یک مجموعه محدود از زوج‌های (Vi, Vj,...) میباشد Vi, Vj,... عضو E که به آنها یال یا لبه می‌گوییم.

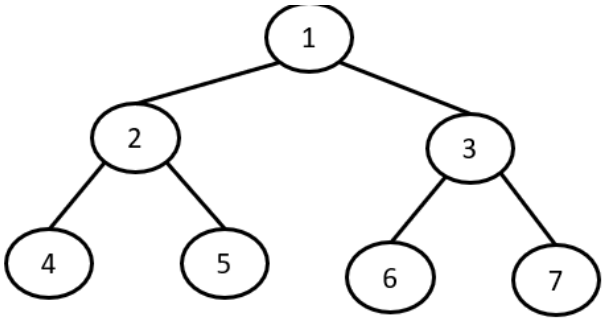
گراف شامل دو مجوعه زیر باشد :

۱ـ مجموعه‌ای از رئوس

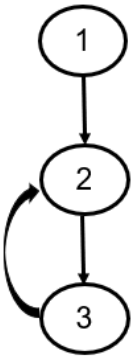
۲ـ مجموعه‌ای از خطوط یا لبه‌ها G = (V,E)



G1



G2



G3

G = Graph (گراف)

V = Vertex (رأس)

E = Edge (لبه)

مثال)

۱_ $G_۱ = (V,E)$

$V(G_۱) = \{۱,۲,۳,٤\}$, $E(G_۱) = \{(۱,۲)(۱,٤)(۲,٤)(۱,۳)(۲,۳)\}$

(Vi, Vj) = (Vj, Vi)

۲_ $G_۲ = (V,E)$

$V(G_۲) = \{۱,۲,...,۷\}$

$E(G_۲) = \{(۱,۲)(۱,۳)(۲,٤)(۲,٥)(۳,٦)(۳,۷)\}$

<Vi, Vj> برای گراف جهت‌دار

<۱,۲> # <۲,۱>

۳_ **$G_۳ = (V,E)$**

$V(G_۳) = \{۱,۲,۳\}$

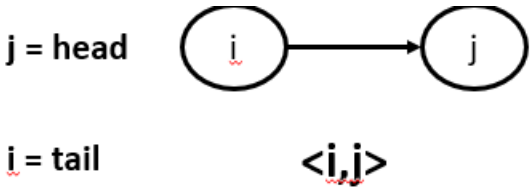
$E(G_۳) = \{<۱,۲> <۲,۳> <۳,۲>\}$

واژه‌های خاص گراف :

۱_ گراف بدون جهت G_1 و G_2

۲_ گراف جهت‌دار G_3

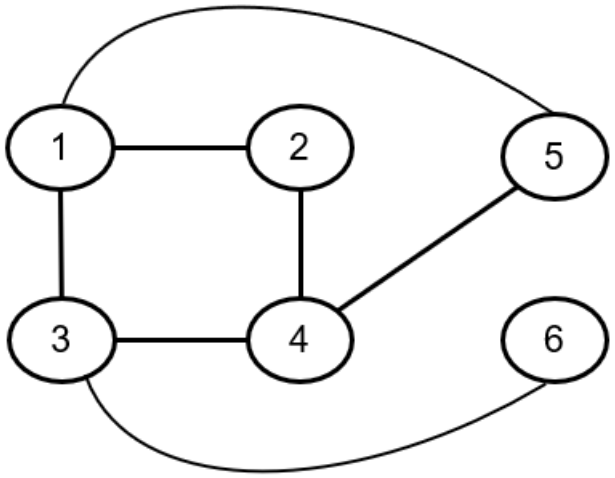
۳_ head



٤_ tail

٥_ گراف کامل

٦_ مسیر : یک مسیر برای دو رأس p و q عبارت است از دنباله‌ای از رئوس بصورت $p\ i_1\ i_2 \ldots i_n\ q$ بصورتی که لبه‌های $((p,i_1)\ (i_1,i_2)\ (i_2,i_3) \ldots (i_n,q))$ وجود داشته باشد.



مثال) در شکل روبه‌رو مسیری از ۱ به ۶ بصورت (۱۲۴۵۱۳۴۲۱۵۶) میباشد و همینطور (۱۳۴۵۶) و نیز (۱۲۴۶)

۷_ مسیر ساده : مسیری است که تمام رئوس آن مجزا باشند و رأس تکراری نداشته باشد بجز احتمالاً ابتدا و انتهای آن، پس مسیری که بار اول گفته شد مسیر ساده نیست اما دو مسیر ساده هستند.

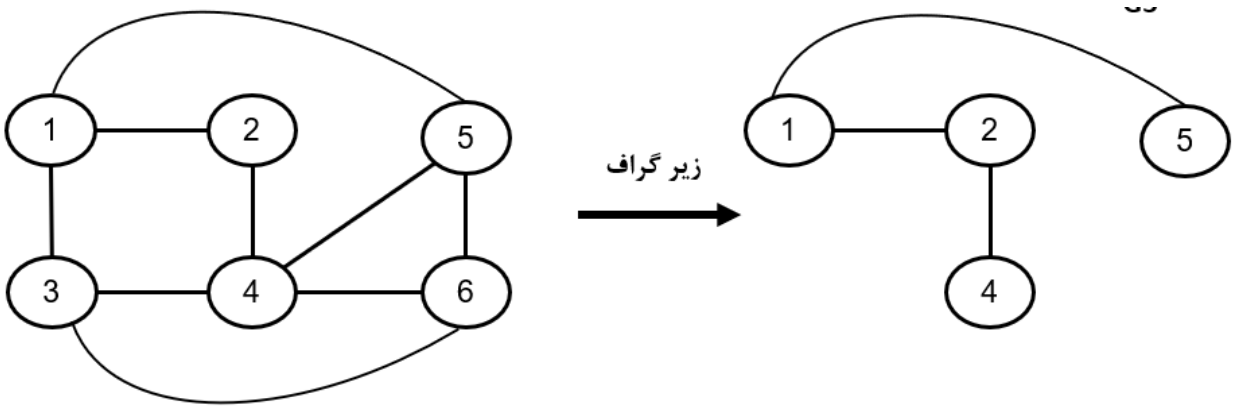
مسیری است که احتمالاً بجز رأس ابتدا و انتهای سایر رئوس قابل تفکیک و غیر تکراری میباشد.

(١ ٣ ٤ ٥ ٦)

(١ ٣ ٢ ٤ ٦)

(٣ ٤ ٦ ٣)

۸_ زیر گراف : اگر $G'(V',E')$ را زیرگراف از $G(V,E)$ می‌گوییم. اگر و تنها اگر V' عضو V و E' عضو E باشد بطوریکه لبه‌های E' فقط با رئوس V' متلاقی باشند.



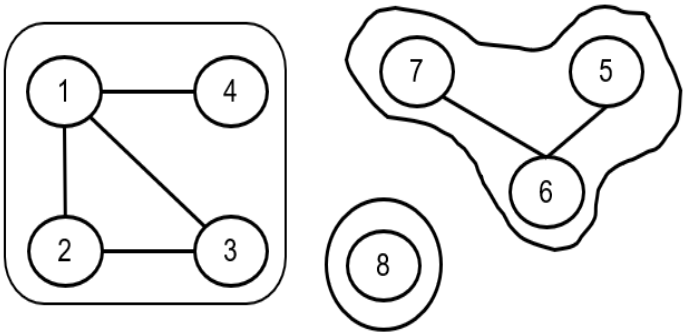
۹_ سیکل (حلقه) : مسیر ساده‌ای است که رأس ابتدا و انتهای آن یکی باشد.

۱۰_ رأس مجاور

۱۱_ گراف همبند (متصل) : گرافی است که در آن بین هر دو گره حداقل یک مسیر داشته باشیم.

۳ گراف داریم و بزرگترین آن مؤلفه همبند است.

به بزرگترین زیرگراف‌های همبند، مؤلفه همبند گویند.

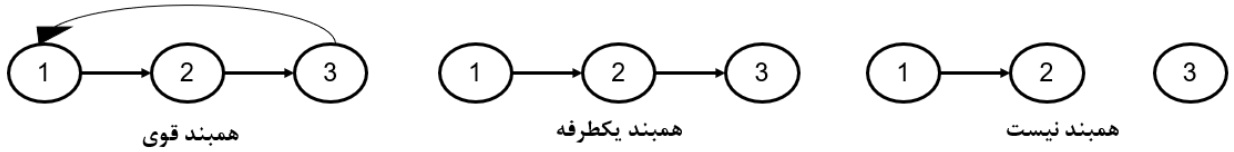


گراف جهت‌دار :

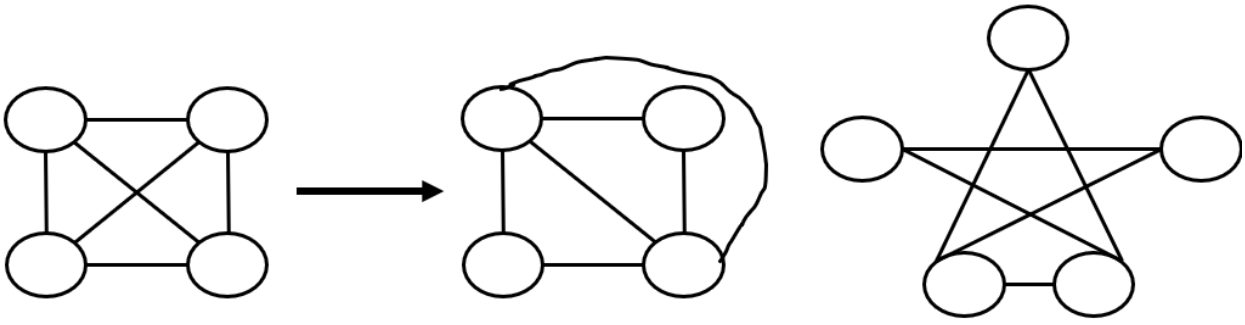
۱_ همبند قوی : وقتی بین دو رأس **j** و **i** یک مسیر از **i** و **j** و هم از **j** به **i** داشته باشیم.

۲_ همیند یکطرفه : بین هر دو رأس دلخواه **i** و **j** یا مسیری از **i** به **j** و یا مسیری از **j** به **i** داشته باشیم.

۳_ همبند ضعیف : هر وقت بین هر دو رأس دلخواه یک مسیر بدون جهت داشته باشیم.



گراف سطح : گرافی که بتوان آنرا در صفحه رسم کرد که لبه‌های آن یکدیگر را قطع نکنند گراف سطح گویند.



این گراف سطح است

این گراف ستاره سطح نیست

درجه رأس : به تعداد رئوس مجاور یک رأس خاص گفته می‌شود.

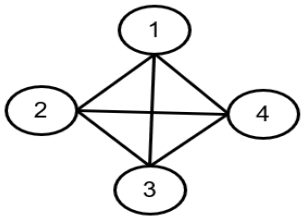
گراف کامل : گراف با n رأس ← تعداد خطوط

گراف بدون جهت $\frac{n(n-1)}{2}$

گراف جهت‌دار $n(n-1)$

اگر شرط ۱ و ۲ برقرار باشد گراف کامل خواهد بود.

مثال) تعداد خطوط مورد نظر



$$\frac{n(n-1)}{2} = \frac{4(4-1)}{2} = 6$$

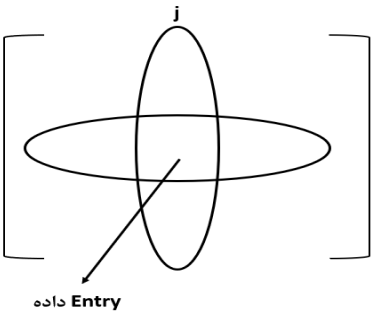
نمایش‌های ممکن گراف :

۱_ ماتریس همجواری مجاورت adjacency matrix

۲_ لیست همجواری یا مجاورت adjacency lists

۳_ لیست‌های چندگانه همجواری یا مجاورت adjacency multi lists

۱_ ماتریس همجواری :



A [i] [j] =

بین رأس i و j اگر لبه باشد "۱"

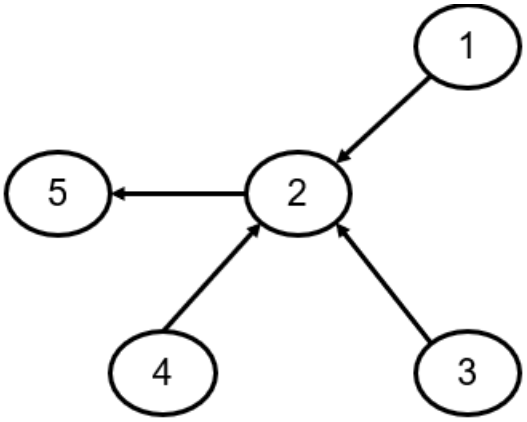
بین رأس i و j لبه نباشد "/"

به مقدار رئوس مجاور یک رأس خاص درجه گفته میشود و برای گراف جهت‌دار دو واژه داریم :

“out_degree” “in_degree”

برای گراف G_1 ماتریس همجواری (صفحه قبل)

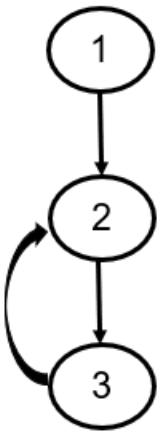
	1	2	3	4
1	0	1	1	1
2	1	0	1	1
3	1	1	0	1
4				



In_degree (2) = 3

out_degree (2) = 1

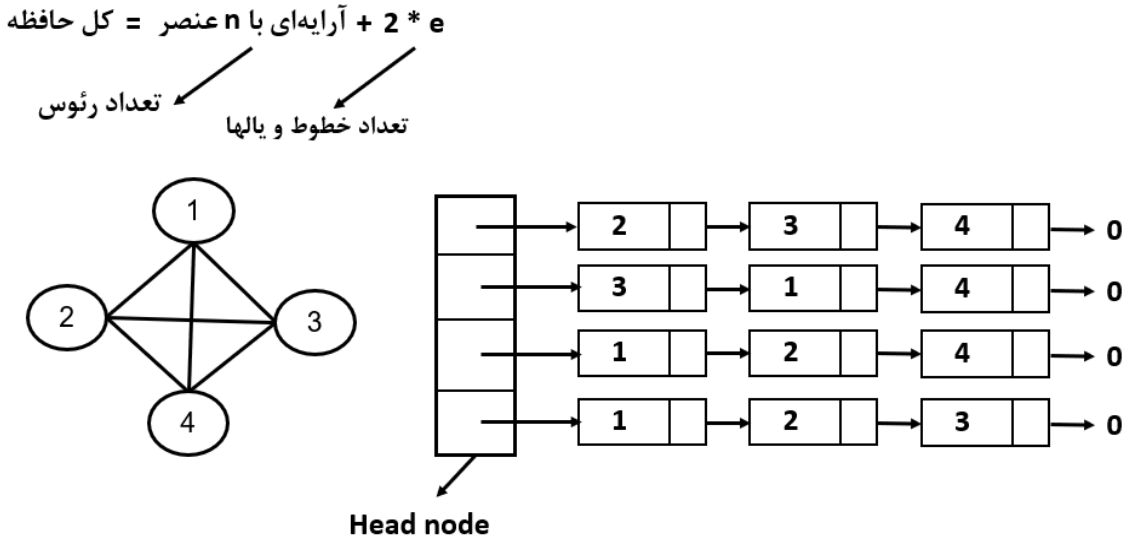
Degree = 1+3 = 4



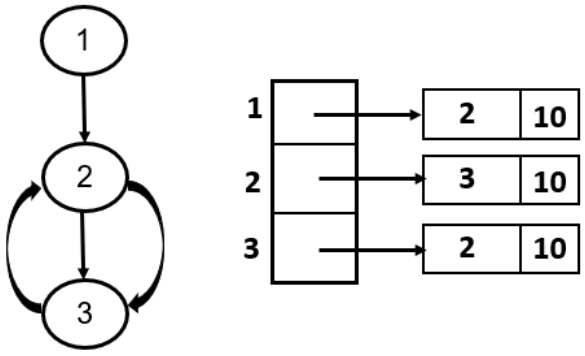
	1	2	3	
1	0	1	0	
2	0	1	1	out_degree
3	0	1	0	
		in_degree		

۲_ لیست همجواری :

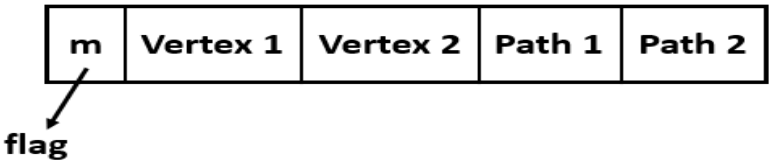
مثال ۱



مثال ۲



۳_ لیست‌های چندگانه همجواری یا مجاورت : ساختار کلی گره در لیست‌های چندگانه



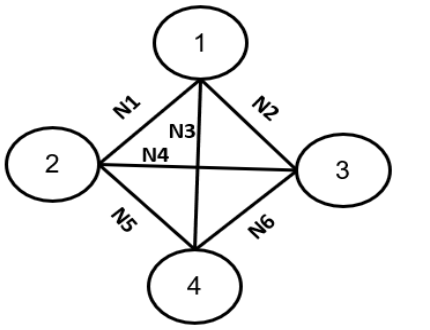
در نمایش لیست مجاورتی هر لبه در یک گراف غیر جهت‌دار مانند (V,V) با دو واژه نمایش داده میشود. یکی در لیست u و دیگری در لیست V همانطور که مشاهده میشود در بعضی موارد لازم است وارده دوم یک لبه خاص را مشخص کرد و علامت بگذاریم تا معلوم شود این لبه قبلاً بررسی شده است این کار به آسانی امکان‌پذیر میباشد

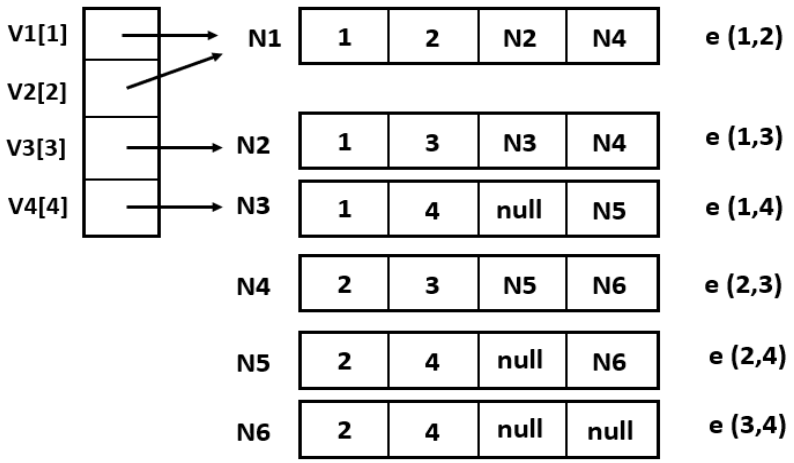
به شرط اینکه لیست بصورت لیست چندگانه درآیند (یعنی، لیستهایی که گره‌هایشان بین چندین لیست مشترک باشند) در حقیقت برای هر لبه بیش از یک گره وجود خواهد داشت اما این گره در دو لیست مشترک خواهد بود (یعنی لیست‌های مجاورتی برای هر دو گروه مجاور وجود دارد).

اگر m فیلدی یک بیتی باشد و نشان میدهد آیا لبه قبلاً بررسی شده است یا خیر؟

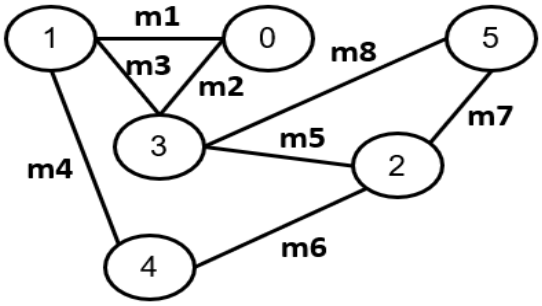
```
struct edge
{
   int m;
   int vertex۱;
   int vertex۲;
   edge * path۱;
   edge * path۱;
};
struct edge headnode[۱][n];
```

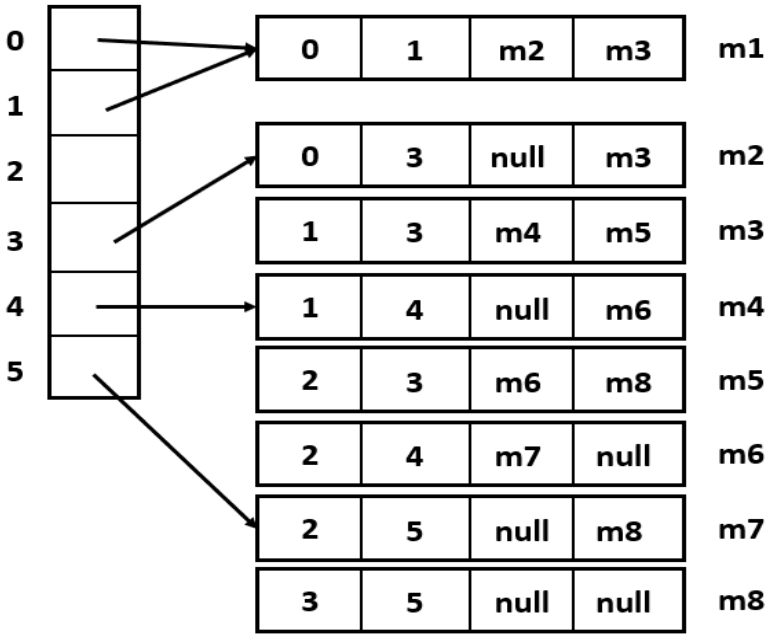
ذخیره‌سازی یک لیست مانند لیست‌های مجاورتی معمولی است با این تفاوت که بیت m برای ۱۴ متدگذاری اضافه گردیده است.





مثال)

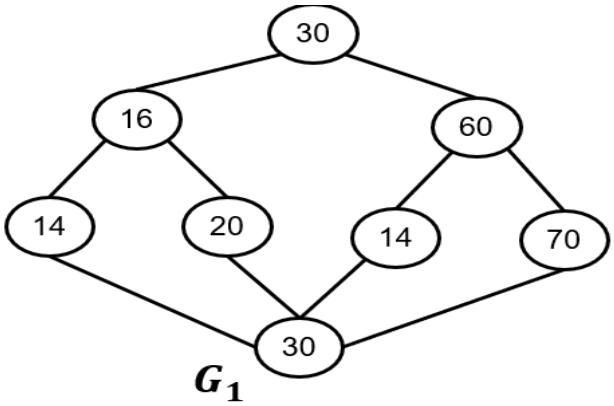




پیمایش‌های مختلف یک گراف

۱_ d.f.s پیمایش عمقی depth first search

۲_ b.f.s پیمایش سطحی یا ردیفی breath first search



G_1

d.f.s $(G_1) = V_1, V_2, V_4, V_8, V_5, V_6, V_3, V_7$

لیست خطی حاصل از پیمایش d.f.s یک پیمایش منحصر به فرد نمیباشد.

d.f.s : رأس شروع ← یکی از رئوس

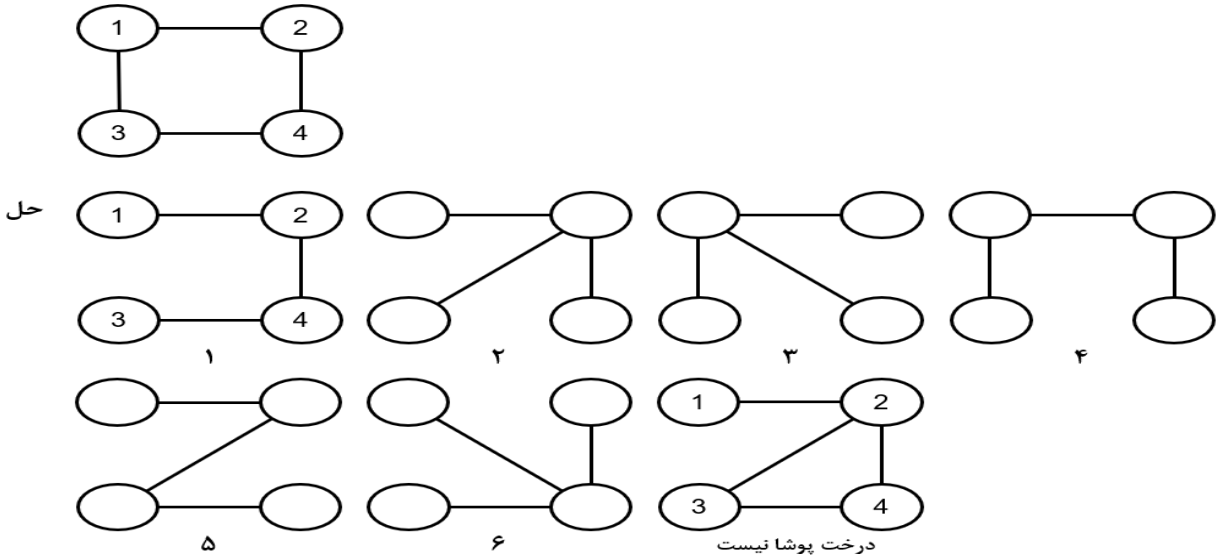
b.f.s : رأس شروع ← تمام رئوس

این نمایش منحصر به فرد نیست. $\mathbf{b.f.s\ (G_1) = V_1, V_2, V_3, V_4, V_5, V_6, V_7, V_8}$

نکته (d.f.s , b.f.s) : روش d.f.s درجه انحصار کمتری نسبت به درجه b.f.s داریم.

درخت پوشا (Spanning Tree) : (درخت یک گراف بدون سیکل است.) گراف بدون هیچ سیکلی یک درخت پوشا است اگر شامل تمام رئوس گراف اصلی باشد و بخشی از خطوط اصلی

مثال) درخت پوشای گراف روبه‌رو را مشخص کنید.



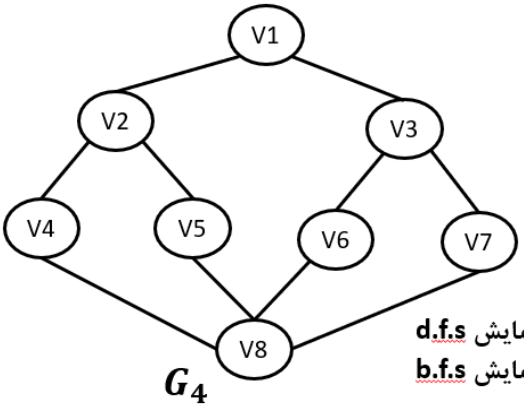
انواع درخت پوشا :

۱_ درخت پوشای حاصل از پیمایش d.f.s

۲_ درخت پوشای حاصل از پیمایش b.f.s

۳_ درخت پوشای عادی

۴_ درخت پوشای حداقل (هزینه)



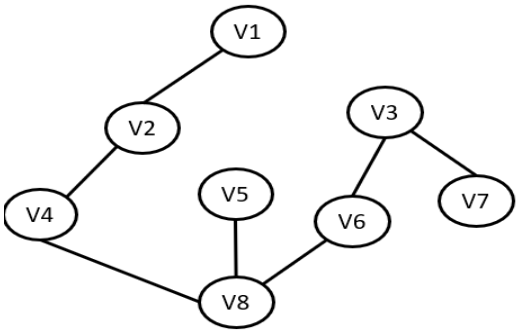
G_4

d.f.s پیمایش = $V_1, V_2, V_4, V_8, V_5, V_6, V_3, V_7$

b.f.s پیمایش = $V_1, V_2, V_3, V_4, V_5, V_6, V_7, V_8$

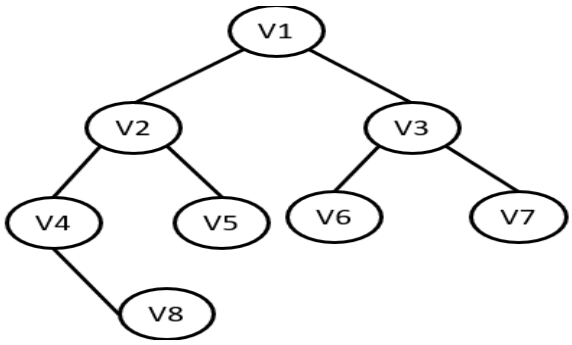
۱_ درخت پوشای حاصل از پیمایش d.f.s

درخت پوشا با توجه به لیست خطی خود منحصر به فرد است.



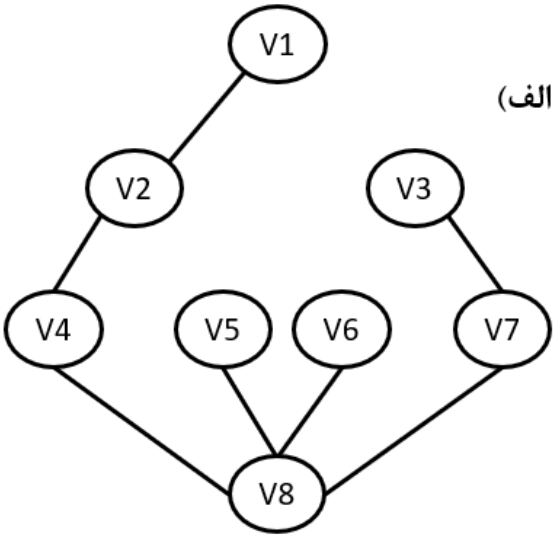
۲_ درخت پوشای حاصل از پیمایش b.f.s

نسبت به لیست خطی خود منحصر به فرد است.

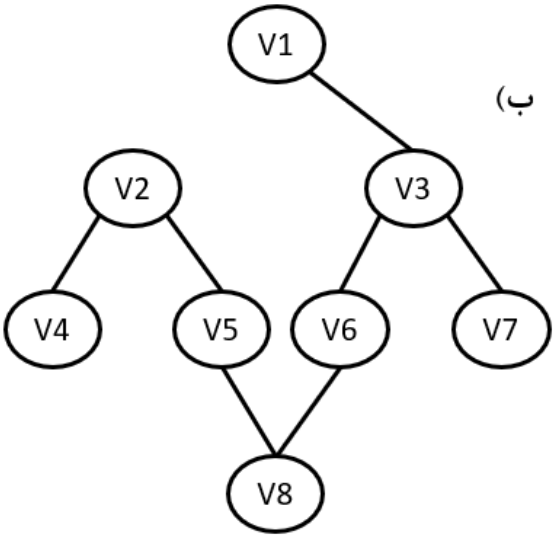


۳_ درخت پوشای عادی

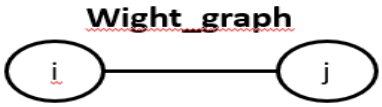
الف)



ب)

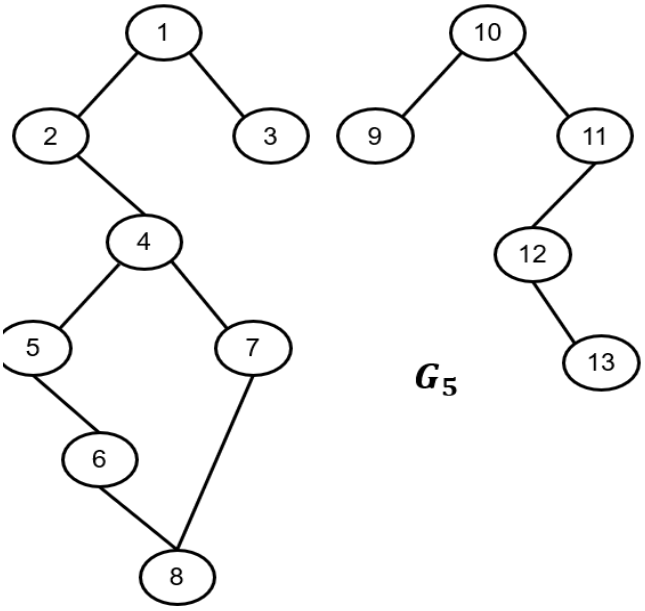


نکته : چنانچه در یک گراف برای خط لبه ارزش با مقدار خاص تعیین کنید، گراف ارزشش را مطرح نموده‌ایم.



“const value duration”

گراف متصل : گرافی است که بین هر دو رأس از گراف مسیری وجود داشته باشد.

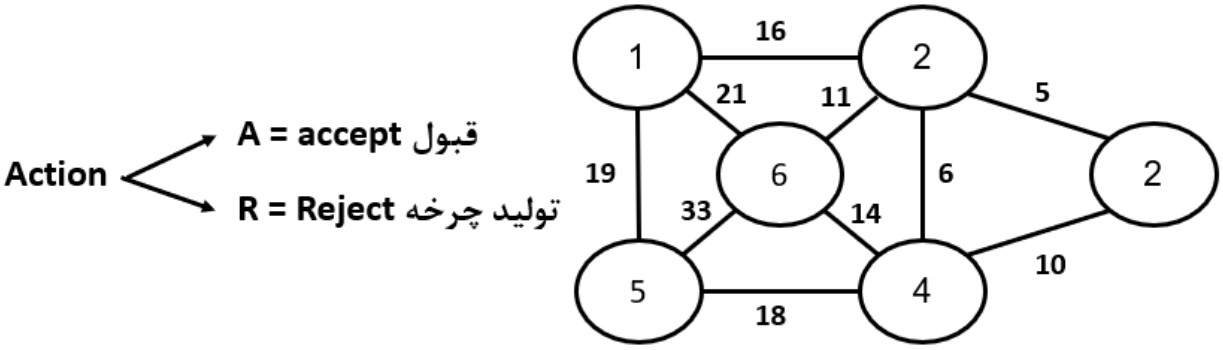


G_5

۱_ حداقل هزینه ارتباط یا اتصال رئوس گراف زیر را مشخص کنید.

۲_ درخت پوشای حداقل هزینه گراف را مشخص کنید.

۳_ گراف متصل (connect graph)

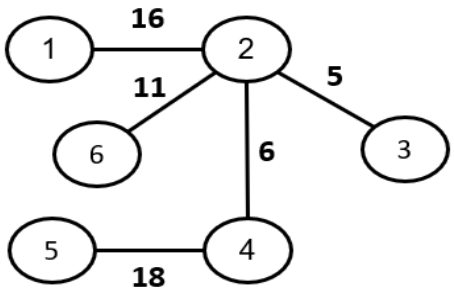


گراف ارزش G_6

نکته : برای گراف جهت‌دار قادر به بیان درخت پوشای حداقل هزینه نیستیم چون جهت در گراف دارای اهمیت است.

Const	edge	Action	graph
*5	(2,3)	A	
*6	(2,4)	A	
10	(3,4)	R	
*11	(2,6)	A	
14	(6,4)	R	
*16	(1,2)	A	
*18	(4,5)	A	
19	(1,5)	R	
21	(1,6)	R	
33	(5,6)	R	

Total : 56 مورد قبول



درخت پوشا با حداقل هزینه :

هزینه یک درخت پوشای یک گراف جهت‌دار دارای وزن مجموع هزینه‌ها (وزن‌ها) لبه‌ها در درخت پوشا میباشد. درخت پوشای حداقل هزینه درخت پوشایی است که دارای کمترین هزینه باشد. در اینجا برای بدست آوردن درخت پوشای حداقل هزینه یک گراف جهت‌دار متصل الگوریتم کروزکال و پریم را داریم.

روش ما برای تعیین درخت پوشا با حداقل هزینه باید سه شرط زیر را داشته باشد:

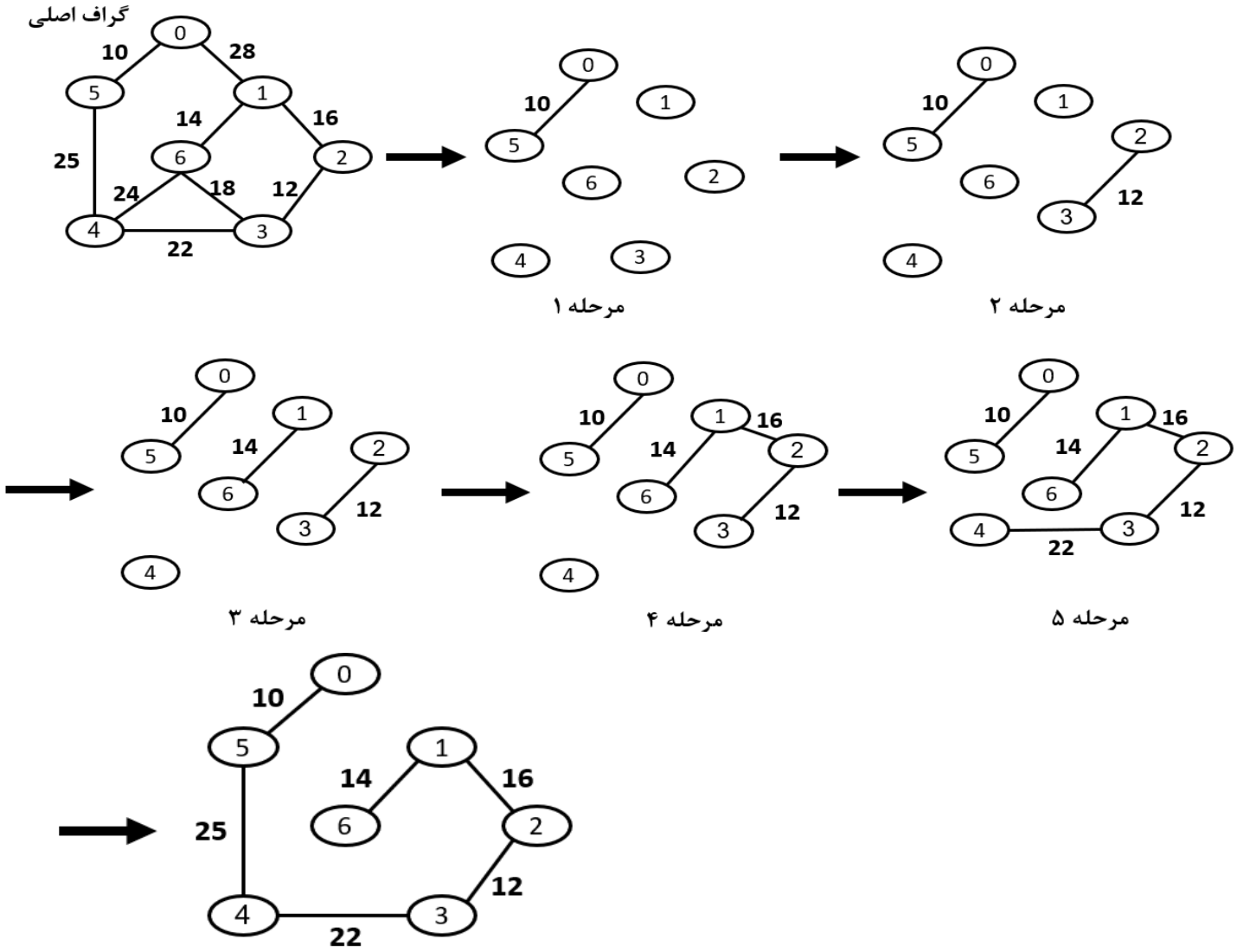
۱_ باید فقط از لبه‌های داخل گره استفاده کنیم.

۲_ باید دقیقا از n-۱ لبه استفاده کنیم.

۳_ نباید از لبه‌هایی که ایجاد یک حلقه میکنند استفاده کنیم.

الگوریتم کروزکال : در این روش درخت پوشا با کمترین هزینه T لبه به لبه ساخته میشود. لبه‌های مورد استفاده در T به ترتیب صعودی وزن میباشد یک لبه در T خواهد بود اگر با لبه‌های قبل که در T بوده‌اند تشکیل یک حلقه ندهند.

مثال) درخت پوشای با کمترین هزینه گراف زیر را بدست آورید؟



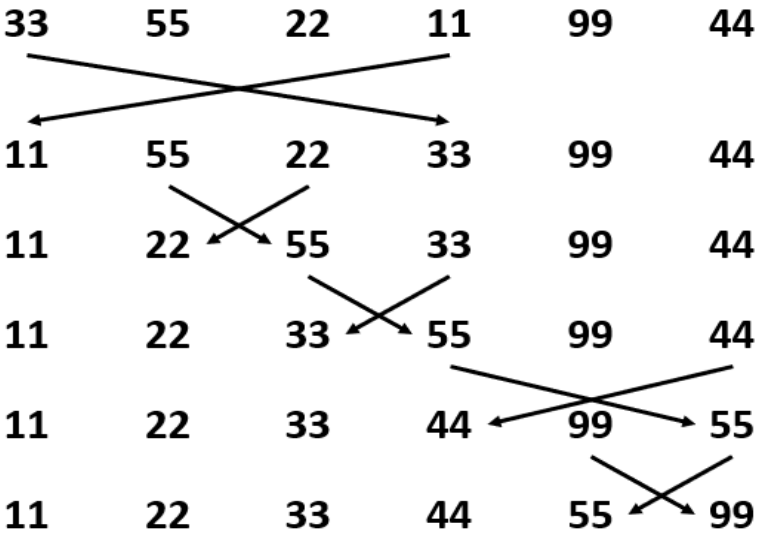
مرتب‌سازی :

به طور کلی الگوریتم‌های مرتب‌سازی به ۲ دسته تقسیم میشود :

۱ــ الگوریتم‌های مرتب‌کننده داخلی "Internal" : که عناصر مرتب‌شونده همگی در حافظه بوده و مقایسه کلید مهم‌ترین عمل در الگوریتم‌ها میباشد. تمام الگوریتم‌های این مبحث از این نوع میباشند.

۲ــ الگوریتم‌های مرتب‌کننده خارجی "External" : که عناصر مرتب‌شونده همه در حافظه اصلی نبوده و دسترسی به عناصر مهم‌ترین عمل و در نتیجه تعیین‌کننده زمان اجرا میباشد.

مرتب‌سازی انتخابی (Selection Sort) : در این الگوریتم در هر مرحله پیمایش آرایه، محل درست یک عنصر (بزرگترین، کوچکترین) پیدا شده و سپس با یک جابجایی در محل درست خود قرار میگیرند.



فرض کنید عناصر در آرایه A ذخیره شده است.

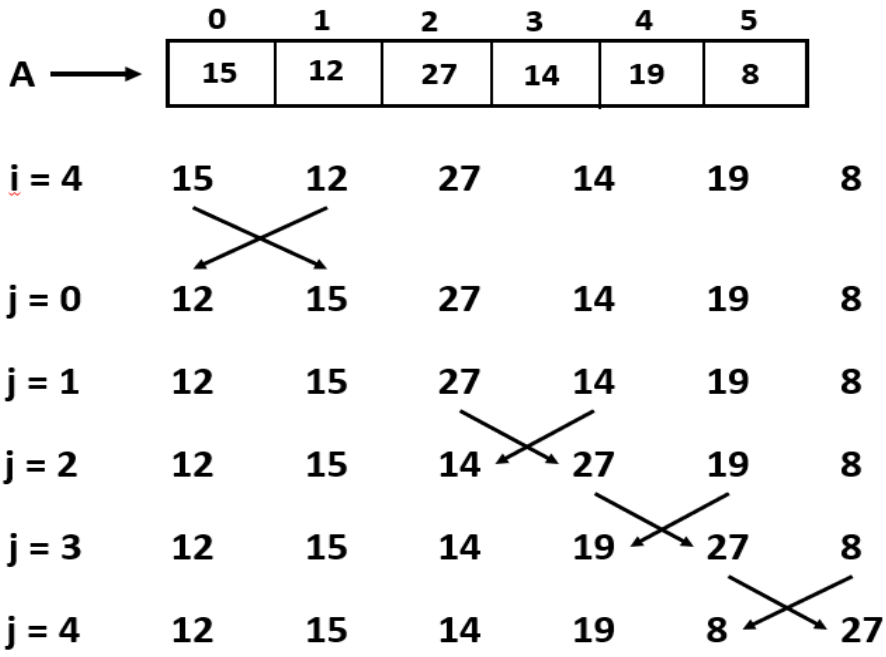
	بهترین	متوسط	بدترین
تعویض	O (1)	O (n)	O (n)
مقایسه	O (n^2)	O (n^2)	O (n^2)

مرتبه اجرایی مقایسه و تعویض

```
for(i = ۰; i < n-۱; i++)
{
   index = i;
   for (j = i + ۱; j < n; j++)
      if (A[index] > A[j])
         index = j;
   t = A[index]; A[index] = A[i]; A[i] = t;
}
```

مرتب‌سازی حبابی (Bubble Sort) : در مرتب‌سازی حبابی باید چندین بار آرایه را پیمایش کرد و دو بار عنصری را با عنصر بعدی خودش مقایسه نمود و در صورتی که عنصر اول از عنصر دوم بزرگتر باشد در مرتب‌سازی صعودی، جای آنها را عوض میکنیم.

```
for (i = n-١; i >= ٠; i++)
   for (j = ٠; j <= i; j++)
      if (A[j] > A[j + ١])
      {
         temp = A[j];
         A[j] = A[j + ١];
         A[j + ١] = temp;
      }
```



	بهترین	متوسط	بدترین
تعویض	O (1)	O (n^2)	O (n^2)
مقایسه	O (n)	O (n^2)	O (n^2)

مرتبه اجرایی الگوریتم
مرتب‌سازی حبابی

مرتب‌سازی سریع (Quick Sort) : در این روش در هر گذر یک عنصر محدودی یا لولا (**Pivot**) انتخاب شده و سایر عناصر بردار به گونه‌ی جابجا میشوند که کلیه عناصر بزرگتر از آن در یک طرف و کلیه عناصر کوچکتر از آن در طرف دیگر عنصر لولا قرار میگیرند به این ترتیب عنصر لولا در مکان درست خود قرار گرفته است. سپس دو بردار در طرف این عنصر بصورت بازگشتی به همین روش مرتب میشوند.

اگر A[n] را داشته باشیم و بخواهیم بصورت صعودی مرتب نمائیم Quick sort (A, ۰, n-۱)

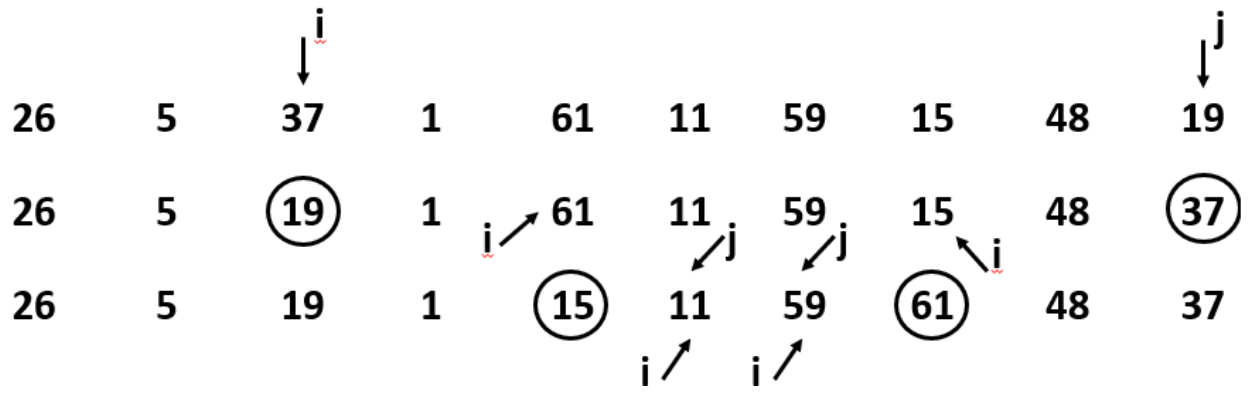
```
void QuickSort(int A[], int left, int right)
{
   int i, j;
   int pivot;
   if (left < right)
   {
      i = left; j = right; pivot = A[left];
      do
      {
         do
         {
            i++;
         }
         while (A[i] < pivot);
         do
         {
            j--;
         }
         while (A[j] > pivot);
         if (i < j) swap(A[i], A[j]);
      }
      while (i < j);
```

```
    swap(A[left], A[i]);
    QuickSort(A, left, j - ۱);
    QuickSort(A, j + ۱, right);
  }//if
}//QuickSort
```

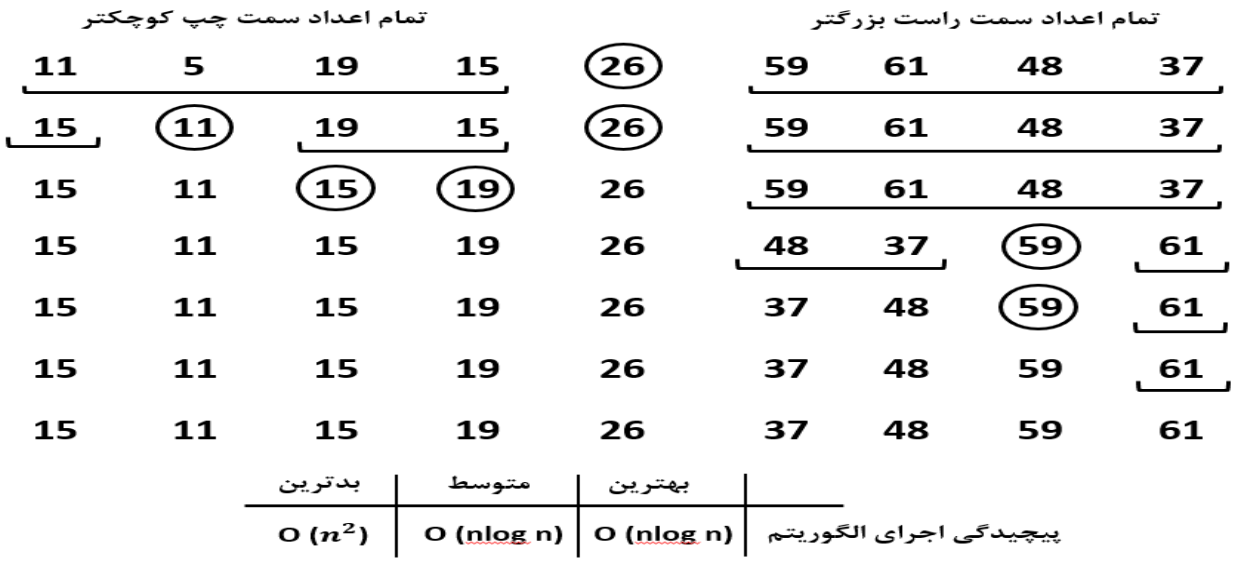
0	1	2	3	4	5	6	7	8	9
26	5	37	1	61	11	59	15	48	19

در ابتدا Pivot = 26, j = 10, i = 0, right = 9, left = 0

پس از سمت چپ مرتباً **i** زیاد شده تا هنگامی که به اولین عنصر بزرگتر از ۲۶ (یعنی ۳۷) برسیم. به همین ترتیب از سمت راست مرتباً **j** کم میشود تا به اولین عنصر کوچکتر ۲۶ (یعنی ۱۹) برسیم. در اینجا جای این دو خانه یعنی ۳۷ و ۱۹ عوض میشود.



از حلقه خارج میشود.



مرتب‌سازی درجی (Insertion Sort) : در این الگوریتم به ترتیب عناصر A[۰] تا A[n-۱] خوانده شده و هر عنصر A[k] در مکان صحیح در زیر آرایه مرتب شده A[۰], A[۱], ..., A[k-۲] درج میشود.

مرحله ۱ : A[۰] خودش به طور بدیهی مرتب است.

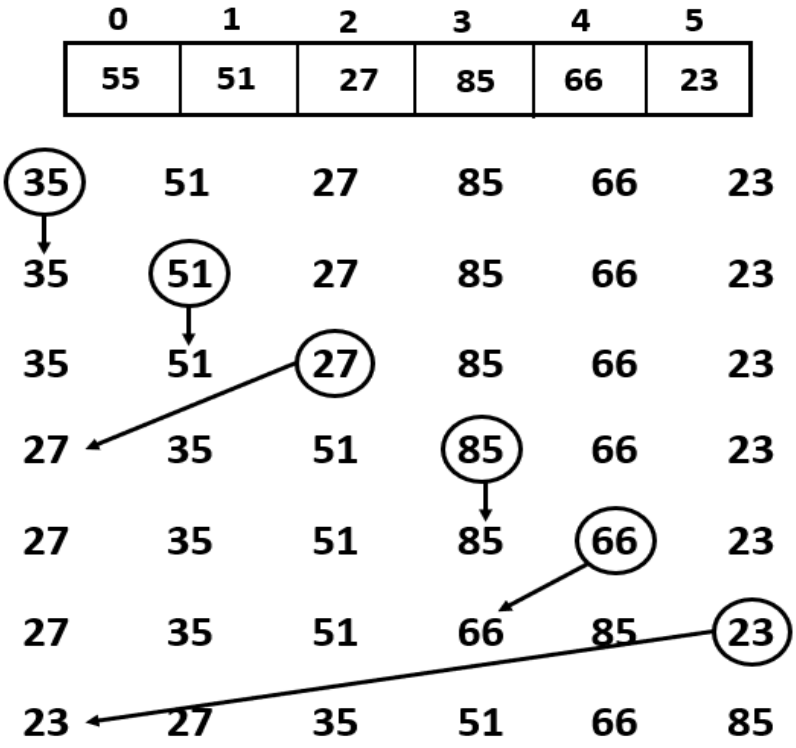
مرحله ۲ : A[۱] را یا قبل از یا بعد از A[۰] درج میکنیم طوری که A[۰] و A[۱] مرتب شوند.

مرحله ۳ : A[۲] را در مکان صحیح در A[۰] و A[۱] بگونه‌ای که A[۰] و A[۱] و A[۲] مرتب شده باشند و به همین ترتیب...

مرحله n : A[n-۱] را در مکان صحیح خود در A[۰], A[۱], ..., A[n-۲] به گونه‌ای درج میکنیم که کل آرایه مرتب شده باشد.

```
void InsertionSort(int A[], int n)
{
   int i, j, y;
   for(i = ۰; i < n; i++)
   {
      y = A[i];
```

```
        j = i - ۱;
        while(j = ۰ && y < A[j])
        {
           A[j + ۱] = A[j];
           j--;
        }
        A[j + ۱] = y;
    }//for
}//Insertion
```



	بهترین	متوسط	بدترین
تعویض	O (1)	O (n^2)	O (n^2)
مقایسه	O (n)	O (n^2)	O (n^2)

مرتب‌سازی ادغام یا ترکیب (Merge Sort) : در این روش دو لیست مرتب شده داریم که بایستی آنها بصورت در یک آرایه دیگر قرار دهیم. از الگوریتم زیر استفاده می‌کنیم.

فرض کنید B[n], A[m] را در اختیار داریم که میخواهیم در آرایه Z[m+n] ذخیره کنیم که الگوریتم آن بصورت زیر است :

```
viod mergeSort(int A[], int b[], int c[], int m, int n)
{
   int i, j, t;
   i = ۰; j = ۰; K = ۰;
   while(i < m && j < n)
   {
      if (A[i] <= b[j])
      {
         c[k] = A[i];
         i++;
      }
      else
      {
         c[k] = b[j];
         j++;
      }
      K++;
   }//while
   if (i >= m)
      for (; j < n; j++, k++)
         z[k] = B[j];
```

```
        else
            z[k] = A[i];
}//mergeSort
```

مثال) برنامه‌ای بنویسید که تعدادی عدد صحیح را از صفحه کلید دریافت کند و هر درخت باینری را مرتب نمایش دهد.

```
struct Tree
{
   Tree* left;
   int a;
   Tree* right;
};
Tree* start = ۰, * p;
void addTree()
{
   Tree* h۱ = start;

   if (start == ۰)
   {
      start = p;
      return;
   }
   while (h۱)
```

```
    {
        if (p->a < h\->a)
        {
            if (h\->left == ·)
            {
                h\->left = p;
                return;
            }
            else
                h\ = h\->left;
        }
        else
        {
            if (h\->right == ·)
            {
                h\->right = p;
                return;
            }
            else
                h\ = h\->right;
        }
    }
}
```

```
void printTree(Tree* h丶)
{
   if (h丶 == NULL)
      return;
   printTree(h丶->left);
   cout << h丶->a << " ";
   printTree(h丶->right);
}
int main()
{
   int i;
   cin >> i;
   while (i)
   {
      p = new Tree;
      p->a = i;
      p->left = ·;
      p->right = ·;
      addTree();
      cin >> i;
   }
   printTree(start);
   return ·;
```

```
}
```

چاپ محتویات لیست پیوندی با رویه بازگشتی :

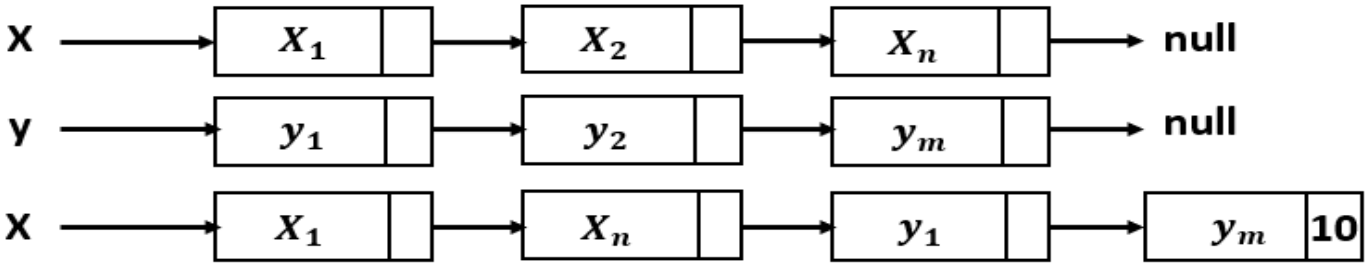
```
void print(linklist* start)
{
   if(start != null)
   {
      count << start->data;
      print(start->next);
   }
}
```

معکوس کردن یک بیست پیوندی : اگر آدرس شروع لیست پیوندی strat باشد. برای معکوس کردن لیست که با هر لیست start شروع میشود از سه اشاره‌گر کمکی g,s,p استفاده میشود و پنجره‌ای به طول ۳ گام در نظر گرفته میشود که در آن هربار یک پیوند معکوس شده و سپس پنجره یک گره جلو میرود.

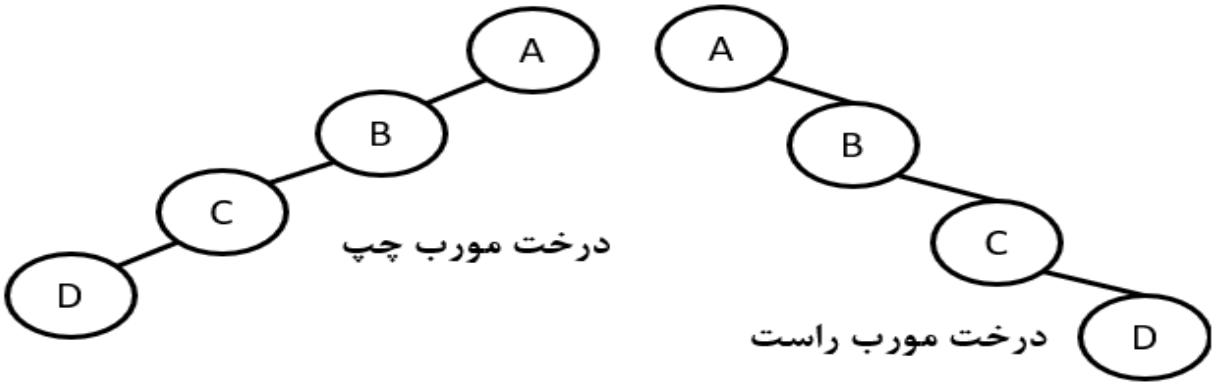
```
linklist* linkviwes(linklist* start)
{
   p = start;
   q = p->next;
   while (q)
   {
      s = q->next;
      q->next = p;
      p = q;
      q = s;
```

```
    }
    start->next = null;
    start = p;
    return start;
}
```

اتصال دو لیست پیوندی به یکدیگر : اگر دو لیست پیوندی $X = (x_1, x_2, ..., x_n)$ و $Y = ((y_1, y_2, ..., y_m)$ وجود داشته باشند لیست $Z = (x_1, x_2, ..., x_n, y_1, ..., y_n)$ را ایجاد میکنیم.



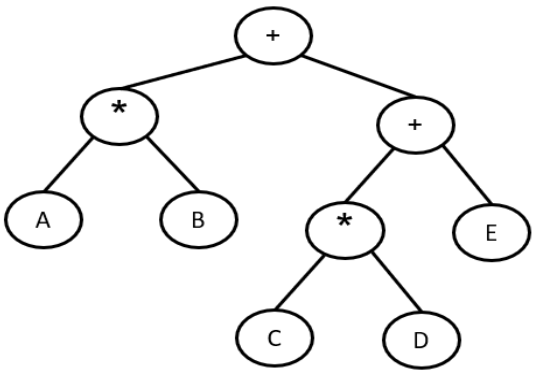
درخت مورب : در درخت مورب چپ هر گره فرزند چپ والد خود میباشد. و درخت مورب راست هر گره فرزند راست والد خود میباشد.



نکته : در یک درخت دودویی با n گره عمق برابر $Log_2 (n+1)$ میباشد.

تمرین ۱ : درخت باینری متناظر با عبارت (A+B*C)(A-۱) را بنویسید.

تمرین ۲ : پیمایش postorder درخت مقابل را بنویسید.



تمرین ۳ : در الگوریتم روبه‌رو در صورتی که n = m باشد مرتبه اجرایی آن را بدست آورید؟

```
for (i = ۱; i <= n; i++)
   for (j = ۱; j <= m; j++)
      for (k = ۱; k <= j; k++)
         x = x + ۱;
```

$$S = \sum_{i=1}^{n}\sum_{j=1}^{m}\sum_{k=1}^{j} 1 = \sum_{i=1}^{n}\sum_{j=1}^{m} j = \sum_{i=1}^{n} \frac{m(m+1)}{2} = \frac{nm(m+1)}{2}$$

$$m = n \Longrightarrow S = \frac{n^2(n+1)}{2} \Longrightarrow \text{مرتبهی زمانی } O(n^3)$$

تمرین ۴ : زمان اجرای الگوریتم T(n) را بنویسید؟

$$T(n) = \begin{cases} ۱ & n=۱ \\ n + T(n-۱) & n>=۲ \end{cases}$$

T(۱) = ۱

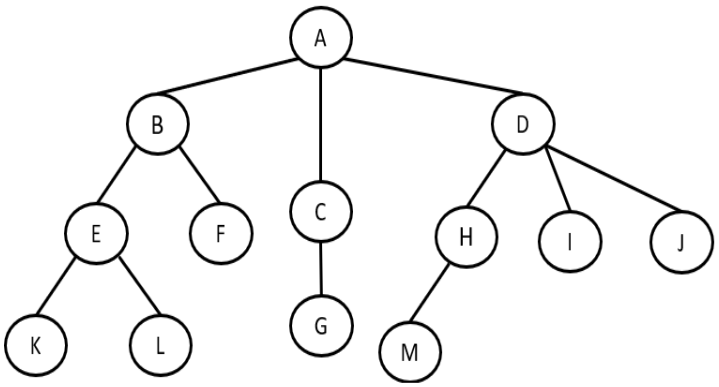
T(۲) = ۲ + T(۱) + T(۳) = ۳ + T(۲) = ۳ + ۲ + ۱...

$$T(n) = n + (n-1) + \dots + 3 + 2 + 1 \longrightarrow T(n) = \frac{n(n+1)}{2} = \frac{n^2+n}{2} = O(n^2)$$

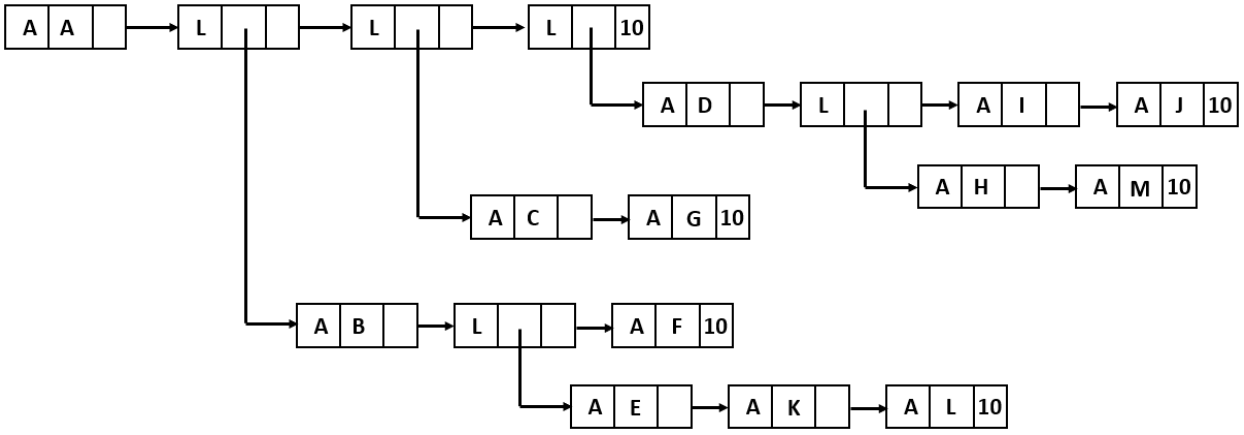
پیمایش درخت :

۱ــ پیمایش سطحی : در این پیمایش‌ها تا زمانی که یک سطح در درخت بطور کامل ملاقات و بازیابی شود پیمایش سطح لبه شروع نمیشود در هر سطح میتوان به شیوه‌های مختلف عمل کرد مثلاً کلیه گره‌ها از چپ یا از راست به چپ میتوانند ملاقات شوند.

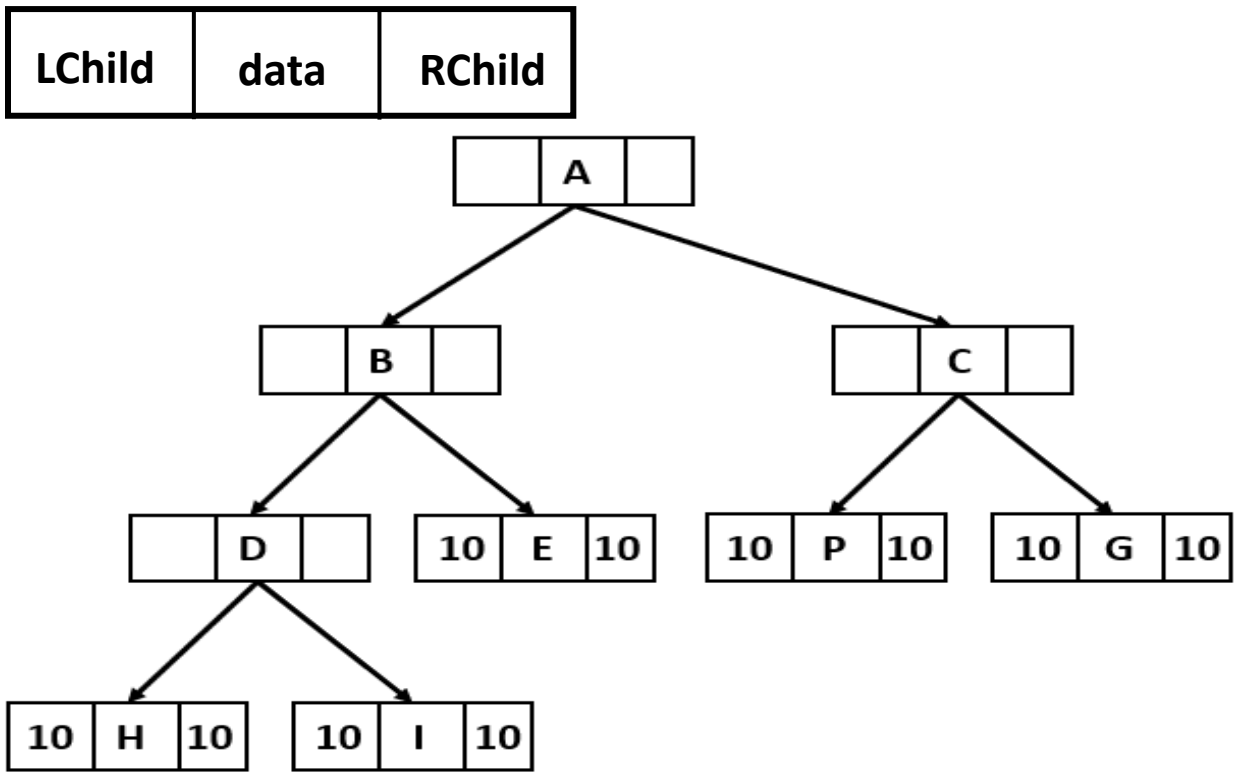
۲ــ پیمایش عمقی : در این پیمایش ها براساس قاعده‌ای پیمایش یک زیر درخت شروع شده و تا قبل از اتمام آن پیمایش زیر درخت دیگر شروع نمیشود. بعنوان مثال اگر برای یک پیمایش پیشوندی ابتدا ملاقات کنیم و سپس کلیه زیر درخت‌ها از چپ به راست تعریف شود. با رسیدن به هر زیر درخت نیز روش فوق تا آخرین سطح اجرا شد و سپس پیمایش زیر درخت سمت راست آن شروع میشود.



مثال) در این روش فرزندان هر گره در سمت راست آن ذکر میشوند و هر فرزندی که خود برگ نباشد با افزودن یک سطح بر لیست بصورت یک زیر لیست نمایش داده میشود، گره‌های لیست بنام اتم با حرف A و گره‌های لیست با حرف L مشخص میشوند.



نمایش درخت دودویی : در این روش نمایش درخت، هر گره سه بخش بصورت زیر میباشد.



پیمایش پیشوندی به شیوه غیر بازگشتی :

در این شیوه از یک پشته و یک متغیر منطقی استفاده میشود.

```
void preorder(Tree *t)
{
   p = t;
   done = ٠;
   do
   {
      while (p != NULL)
      {
```

```
            cout << p->data << " ";
            if (p->rchild != NULL)
               push(p->rchild, s);
            p = p->lchild;
         }
         if (stackEmpty(s))
            done = ١;
         else
            pop(p, s);
      } while (!done);
}
```

پیمایش میانوندی به شیوه غیر بازگشتی :

```
void inorder(Tree *t)
{
   p = t;
   done = ٠;
   do
   {
      while (p != NULL)
      {
         push(p, s);
         p = p->lchild;
      }
```

```
        if (!stackEmpty(s))
        {
            pop(p, s);
            cout << p->data << " ";
            p = p->rchild;
        }
        else
            done = ١;
    } while (!done);
}
```

ایجاد کپی از درخت دودویی :

```
Tree* Copy(Tree *t)
{
    if (t != NULL)
    {
        q = Copy(t->lchild);
        p = Copy(t->rchild);
        k = getnode();
        k->data = t->data;
        k->lchild = q;
        k->rchild = p;
    }
    else
```

```
    {
        k = NULL;
    }
    return k;
}
```

بررسی برابر دو درخت دودویی :

```
int equal(Tree *t۱, Tree *t۲)
{
    int res = ۰; // همان متغیری که در عکس equal نام داشت
    if (t۱ == NULL && t۲ == NULL)
        res = ۱;
    else if (t۱ != NULL && t۲ != NULL)
    {
        if (t۱->data == t۲->data)
        {
            if (equal(t۱->lchild, t۲->lchild))
                res = equal(t۱->rchild, t۲->rchild);
        }
    }
    return res;
}
```

درخت نخی دودویی : در یک درخت دودویی با n گره، ۲n اشاره‌گر وجود دارد که از این تعداد n-۱ اشاره‌گر مورد استفاده قرار گرفته‌اند و n+۱ اشاره‌گر مقدار null دارند. با استفاده از این اشاره‌گرهای بدون استفاده میتوان به عناصر قبلی یا لبه در یک پیمایش خاص اشاره نمود و در نتیجه در هنگام پیمایش به نحو سریعتری درخت را پیمایش کرد، درختی که اشاره‌گرهایی بدون استفاده آن به این صورت مورد استفاده قرار گرفته است، درخت نخی یا درخت نخ‌کشی شده نام دارد.

بسته با وجود این مزیت اکنون باید اشاره‌گرهای واقعی از اشاره‌گرهای نخی به نحوی متمایز شوند و به این منظور باید در هر گره بخشی برای مشخص کردن نوع اشاره‌گر موجود باشد.

معمولاً اشاره‌گر سمت چپ در صورتی که بلااستفاده باشد جهت اشاره به عنصری استفاده میشود که در یک پیمایش مورد نظر قبل از این عنصر قرار دارد و اشاره‌گر سمت راست در صورت بدون استفاده بودن برای اشاره به عنصر لبه‌ی در پیمایش مورد نظر مورد استفاده قرار میگیرد.

برای امکان تشخیص اشاره‌گرهای واقعی از اشاره‌گرهای نخی دو فیلد منطقی به هر گره درخت افزوده میشود که Ltag, Rtag نامیده میشود. ساختار گره در چنین درختی بصورت زیر است :



اگر Ltag = ۱ باشد Lchild دارای یک اشاره‌گر نخی است و در غیر اینصورت اشاره‌گر عادی به فرزند چپ است و به همین صورت اگر Rtag = ۱ باشد Rchild از نوع نخ و در غیراینصورت (Rtag = ۰) Rchild اشاره‌گر عادی به فرزن راست است.

پیمایش میانوندی با استفاده از درخت نخی :

رویه زیر عنصری را پیدا میکند که پس از گره مفروض t در پیمایش میانوندی درخت قرار دارد.

```
struct tree* Insvc(struct tree *t) {
   struct tree *temp;
   if (t->Rtag == ۱) {
      return t->Rchild;
```

```
    }
    temp = t->Rchild;

    while (temp->Ltag == ۰) {
        temp = temp->Lchild;
    }
    return temp;
}
```

رویه نمایش میانوندی درخت را با استفاده از روش قبل به همراه چاپ محتوای درخت انجام میدهند.

پیمایش پیشوند درخت نخی:

```
void Tinorder(tree *t)
{
    temp = t;
    do {
    temp = Invscl(temp);
    if (temp != t)
    cout << temp -> data;
    } while (temp != t);
}
struct tree *Invscl(tree *t)
{
    temp = t;
    if (t -> ltag != ۱)
```

```
    temp = temp -> Lchild;
    else
    temp = temp -> Rchild;
    return temp;
}
```

تشکیل درخت نخی براساس پیمایش پیشوندی :

در این حالت برای ایجاد نخ‌ها از اتصالات تهی بصورت زیر استفاده میشود.

P →Rchild : به عنصری که در پیمایش پیشوندی بعد از p می‌آید اشاره میکند.

P → Lchild : به عنصری که در پیمایش پیشوندی قبل از p می‌آید اشاره می‌نماید.

```
void Tpreorder(tree *t)
{
    temp = t;
    do
    {
    temp = InSvc(temp);
    if (temp != t,
    cout << temp -> data,
    while (temp != t);
    }
}
```

تشکیل درخت نخی براساس پیشمایش پسوندی : در این حالت نیز اتصالات تهی بصورت زیر استفاده میشود :

P → Rchild : به عنصری که در پیمایش پسوند بعد از p می‌آید اشاره میکند.

P → Lchild : به عنصری که در پیمایش پسوندی قبل از p می‌آید اشاره میکند.

